B. TECH. PROJECT REPORT

On

Accelerating Diffusion Models with Training-Free Caching: ReSPect and KalSea

with Applications in Mechanical Engineering

BY

Shreeyut Maheshwari

230003071



DISCIPLINE OF MECHANICAL ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY INDORE

November 2026

Accelerating Diffusion Models with Training-Free Caching: ReSPect and KalSea, with Applications in Mechanical Engineering

A PROJECT REPORT

Submitted in partial fulfillment of the
requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY
in

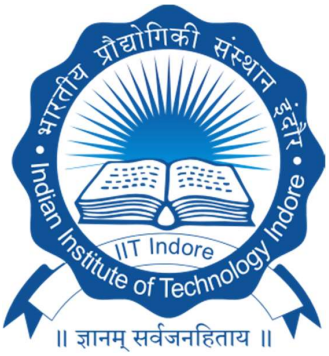
MECHANICAL ENGINEERING

Submitted by:

Shreeyut Maheshwari

Guided by:

Dr. Mohd. Arshad



INDIAN INSTITUTE OF TECHNOLOGY INDORE
November 2026

CANDIDATE'S DECLARATION

I hereby declare that the project entitled **"Accelerating Diffusion Models with Training-Free Caching: ReSPect and KalSea, with Applications in Mechanical Engineering,"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Mechanical Engineering' completed under the supervision of **Dr. Mohd. Arshad, Assistant Professor,** IIT Indore is an authentic work.

Further, I declare that I have not submitted this work for the award of any other degree elsewhere.

Signature and name of the student with the date
Shreeyut Maheshwari (230003071)

CERTIFICATE by BTP Guide

It is certified that the above statement made by the student is correct to the best of my knowledge.

Signature of BTP Guide with date and designation
Dr. Mohd. Arshad
Assistant Professor
Department of Mathematics
IIT Indore

Preface

This report on "Accelerating Diffusion Models with Training-Free Caching: ReSPect and KalSea, with Applications in Mechanical Engineering" was prepared under the guidance of Dr. Mohd. Arshad. It describes my B.Tech. project on making diffusion models, the generative models behind current image and video generators and an increasing number of engineering design tools, faster to sample without retraining them. The project started with a literature review of training-free feature caching, in particular SeaCache and Spectrum, and led to two methods of my own: *ReSPect* (Residual Spectral Prediction), which forecasts the skipped transformer residuals with a Chebyshev–Taylor predictor behind a spectrally aligned gate, and *KalSea* (Kalman-Corrected Spectral-Evolution-Aware Caching), which replaces the open-loop forecast by a Kalman filter and corrects every skipped gap with predictor-corrector gap smoothing.

The report explains the theory behind both methods, their implementation for several open-weight models, their evaluation, and how they carry over to diffusion models used in mechanical engineering, namely structural topology optimization and the reconstruction of flow fields governed by partial differential equations. I have tried, to the best of my abilities, to explain the content in a lucid manner and have added figures, tables and algorithms to make it more illustrative.

Shreeyut Maheshwari
B.Tech. IV Year
Discipline of Mechanical Engineering
IIT Indore

Acknowledgments

I extend my sincere gratitude to Dr. Mohd. Arshad for his unwavering support and invaluable guidance throughout the course of this project. His encouragement to pursue a research problem in depth, and his insistence on careful analysis and honest comparison, shaped both methods described in this report. Without his mentorship this work would not have been possible.

I thank my co-author Abhishek Varshney (IIT Indore) for the many discussions on the methods and experiments, and Monty Anderson and Caleb Case (Prodia), co-authors of the ReSPect and KalSea papers, for their collaboration. I am also grateful to the authors of SeaCache, Spectrum, FLUX, HunyuanVideo, Wan2.1, Optimize Any Topology and DiffusionPDE for releasing their code and model weights, on which all experiments in this report are built. This experience has been both enlightening and rewarding.

Shreeyut Maheshwari
B.Tech. IV Year
Discipline of Mechanical Engineering
IIT Indore

Abstract

Diffusion and flow-matching transformers generate high-quality images and videos and are increasingly used in engineering design, but sampling them requires tens to thousands of sequential evaluations of a large network. Training-free feature caching reduces this cost by skipping the network on some steps; every cache must decide *when* to compute and *what* to write on the skipped steps. This project develops two caches that improve both decisions.

ReSPect (Residual Spectral Prediction) couples a spectrally aligned gate, derived from the linear minimum-mean-square-error (Wiener) denoiser, with a forecaster of the transformer *residual* that blends a ridge-regularised Chebyshev fit of the residual history with a first-order discrete Taylor correction. We show that its forecast error admits a bound independent of the skip length, whereas the bounds of copy-reuse and Taylor extrapolation grow with it, and we derive the blend weight and warm-up analytically. ReSPect achieves speedups of 3.30×, 2.63× and 2.77× on FLUX.1-dev, Wan2.1-1.3B and HunyuanVideo with lower output error than existing caching baselines at matched or lower compute, improving PSNR by up to 3.46, 11.4 and 9.9 dB.

KalSea (Kalman-Corrected Spectral-Evolution-Aware Caching) observes that all existing fillers are open loop: the error made on skipped steps is integrated into the sample and never corrected. Behind the same gate, a damped-trend Kalman filter forecasts the residual, and *predictor–corrector gap smoothing* (PCGS) uses the Rauch–Tung–Striebel smoother to correct every skipped gap once the next step is computed, re-integrating it through the output layers only, with no additional transformer evaluations. On FLUX.1-dev, KalSea reaches 27.72 dB PSNR with respect to the 50-step sampler at 18.1 of 50 evaluations (SeaCache: 26.29 dB at 18.4) and 22.53 vs. 21.33 dB at 13 evaluations; PCGS alone contributes 0.81 and 0.61 dB, and KalSea improves over SeaCache on HunyuanVideo and Wan2.1 by up to 3.2 and 2.3 dB in the same setup.

Finally, we carry both caches over to two open-weight diffusion models from mechanical engineering, Optimize Any Topology for structural topology optimisation and DiffusionPDE for the reconstruction of a flow past a cylinder, by generalising the residual split to U-Net denoisers and the corrector to DDIM with classifier-free guidance and to gradient-guided second-order samplers. On DiffusionPDE's Navier–Stokes forward and inverse problems, KalSea needs 1.7–2.6× fewer network evaluations than the 2000-step sampler, recovers the flow with 3.3% error on the forward problem of the first released case, and is more accurate than ReSPect in all eight comparisons, showing that the closed loop is essential when guidance gradients are approximated on skipped steps.

Table of Contents

Preface **4**

Acknowledgments **5**

Abstract **6**

List of Figures **11**

List of Tables **12**

List of Abbreviations **14**

1 Introduction **15**
1.1 Background and motivation 15
1.2 Problem statement 16
1.3 Objectives and contributions 16
1.4 Organisation of the report 17

2 Literature Review **18**
2.1 Diffusion and flow-matching generative models 18
2.2 Approaches to faster sampling 19
2.3 Training-free feature caching 20
2.3.1 Caching in U-Nets and diffusion transformers 20
2.3.2 Dynamic gates: deciding when to compute 20
2.4 SeaCache: a spectrally aligned gate 21
2.5 Forecasting and correcting skipped features 22
2.5.1 From reuse to forecasting 22
2.5.2 Spectrum: global Chebyshev forecasting 22
2.5.3 Correcting what is written 22
2.6 Filtering, smoothing and predictor–corrector integration 23
2.7 Diffusion models in mechanical engineering 23
2.8 Research gaps 24

3 Background and Preliminaries **26**
3.1 Sampling diffusion and flow-matching models 26
3.2 Anatomy of a diffusion transformer 27
3.3 Residual caching 27

3.4 The spectrally aligned (SEA) gate . . . 28
3.4.1 Gate input . . . 28
3.4.2 Derivation of the Wiener response . . . 28
3.4.3 Accumulate-and-reset rule . . . 29
3.5 Evaluation methodology . . . 30

4 ReSPect: Residual Spectral Prediction 32
4.1 Motivation . . . 32
4.2 Problem setup . . . 33
4.3 Predictors on the sampling trajectory . . . 34
4.3.1 Order zero: copy-reuse . . . 34
4.3.2 Order P: Taylor extrapolation . . . 35
4.4 Residual forecasting with Chebyshev bases . . . 36
4.5 Why forecast the residual? . . . 38
4.6 Analytic blend and warm-up . . . 38
4.7 Implementation . . . 39
4.8 Experiments . . . 40
4.8.1 Settings . . . 40
4.8.2 Text-to-image results . . . 40
4.8.3 Text-to-video results . . . 41
4.8.4 Cost of the forecaster and comparison with SeaCache . . . 41
4.8.5 Ablations . . . 42
4.8.6 Qualitative clips . . . 44
4.9 Discussion and limitations . . . 45

5 KalSea: Kalman-Corrected Spectral-Evolution-Aware Caching 49
5.1 From ReSPect to KalSea: the open-loop problem . . . 49
5.2 The Kalman residual forecaster (predictor) . . . 50
5.2.1 State-space model . . . 50
5.2.2 Parametrisations . . . 52
5.3 Predictor–corrector gap smoothing (corrector) . . . 52
5.3.1 Step 1: smoothing . . . 52
5.3.2 Step 2: re-integration . . . 53
5.3.3 Step 3: continuation . . . 54
5.3.4 Step 4: gate refresh . . . 54
5.3.5 Remarks and extensions . . . 54
5.4 Cost . . . 55
5.5 Implementation . . . 55
5.6 Experiments . . . 57

5.6.1 Setup . . . 57
5.6.2 Results . . . 58
5.6.3 Ablations . . . 60
5.6.4 Limitations . . . 61
5.7 Comparison of ReSPect and KalSea . . . 62

6 Applications in Mechanical Engineering 64
6.1 Carrying the caches over to engineering denoisers . . . 64
6.1.1 Residual caching on a U-Net . . . 65
6.1.2 Gate input . . . 65
6.1.3 Signal and noise coefficients . . . 65
6.1.4 Classifier-free guidance and the corrector . . . 66
6.1.5 Implementation . . . 66
6.1.6 Validation . . . 67
6.2 Application I: structural topology optimisation with OAT . . . 68
6.2.1 Problem . . . 68
6.2.2 The OAT model . . . 68
6.2.3 Experimental protocol . . . 68
6.2.4 Results . . . 69
6.3 Application II: flow past a cylinder with DiffusionPDE . . . 69
6.3.1 Problem . . . 69
6.3.2 The DiffusionPDE sampler . . . 70
6.3.3 Experimental protocol . . . 70
6.3.4 Results . . . 70
6.4 Reproduction . . . 72
6.5 Discussion . . . 72

7 Conclusion and Future Work 73
7.1 Conclusion . . . 73
7.2 Future work . . . 74

References 76

A Code 84
A.1 Repository layout . . . 84
A.2 The SEA filter . . . 84
A.3 The gate in the patched FLUX.1 forward . . . 86
A.4 The ReSPect forecaster . . . 87
A.5 The KalSea forecaster . . . 90
A.6 Predictor–corrector gap smoothing . . . 91

A.7 Caching core for the engineering applications 93

List of Figures

2.1 Chebyshev polynomials used by Spectrum and ReSPect 22

3.1 Residual caching 28
3.2 Wiener responses of the SEA filter 30

4.1 Images generated with ReSPect on FLUX.1-dev 33
4.2 Overview of ReSPect 34
4.3 ReSPect on FLUX.1-dev: qualitative comparison 42
4.4 ReSPect: PSNR versus compute 43
4.5 ReSPect on video: opening frames 44
4.6 ReSPect: reconstructor ablations 45
4.7 ReSPect: skip rates per prompt 45
4.8 ReSPect: per-prompt SSIM and LPIPS 46
4.9 ReSPect: HunyuanVideo frame strip 47
4.10 ReSPect: Wan2.1 frame strip 47

5.1 Images generated with KalSea on FLUX.1-dev 50
5.2 Overview of KalSea 51
5.3 KalSea: fidelity versus compute 59
5.4 KalSea: qualitative comparison 60

6.1 Caching OAT: worked examples 69
6.2 Caching DiffusionPDE: forward problem, case 1 71
6.3 Caching DiffusionPDE: inverse problem, case 2 72

List of Tables

2.1 Summary of training-free caching methods 25

4.1 ReSPect on FLUX.1-dev 41
4.2 ReSPect on HunyuanVideo and Wan2.1 41
4.3 Cost of the ReSPect forecaster 42
4.4 ReSPect versus SeaCache at matched schedules 42
4.5 ReSPect: sweeps of M and λ 43
4.6 ReSPect: per-clip comparison 48

5.1 KalSea backbones and their integration 56
5.2 KalSea on FLUX.1-dev 58
5.3 KalSea on HunyuanVideo and Wan2.1 59
5.4 KalSea ablations 61
5.5 Conceptual comparison of ReSPect and KalSea 62

6.1 Mapping of the caches onto the two engineering models 67
6.2 Caching OAT: results on the OpenTO test split 69
6.3 Caching DiffusionPDE: results 71

List of Abbreviations

Abbreviation	Meaning
adaLN	Adaptive layer normalisation
CE	Compliance error
CFG	Classifier-free guidance
CI	Confidence interval
DCT	Discrete cosine transform
DDIM	Denoising diffusion implicit model
DDPM	Denoising diffusion probabilistic model
DiT	Diffusion transformer
DPS	Diffusion posterior sampling
EDM	Elucidated diffusion model (Karras et al.)
FEA	Finite-element analysis
FM	Flow matching
IoU	Intersection over union
KalSea	Kalman-corrected spectral-evolution-aware caching
LDM	Latent diffusion model
LMMSE	Linear minimum mean-square error
LPIPS	Learned perceptual image patch similarity
MAE	Mean absolute error
NFAE	Neural-field autoencoder
NFE	Number of (network) function evaluations
OAT	Optimize Any Topology
ODE	Ordinary differential equation
PCGS	Predictor–corrector gap smoothing
PDE	Partial differential equation
PSNR	Peak signal-to-noise ratio
ReSPect	Residual spectral prediction (for efficient caching of transformers)
RTS	Rauch–Tung–Striebel (smoother)
SEA	Spectral-evolution-aware (filter / gate)
SIMP	Solid isotropic material with penalisation
SNR	Signal-to-noise ratio
SSIM	Structural similarity index
TFLOPs	Tera floating-point operations
UniPC	Unified predictor–corrector solver

Chapter 1

Introduction

1.1 Background and motivation

Diffusion models [1–3] and their flow-matching counterparts [4, 5] are the dominant family of generative models today. They produce images and videos of remarkable quality, and they are increasingly used in science and engineering: to propose optimal structures in topology optimisation [6, 7], to design mechanical metamaterials [8] and CAD models [9], and to reconstruct flow fields governed by partial differential equations [10, 11]. Their strength comes at a price. A diffusion model generates a sample by starting from random noise and removing it gradually: a numerical solver integrates a learned differential equation over tens to thousands of steps, and every step requires a full evaluation of a large neural network. In the current text-to-image and text-to-video models, this network is a diffusion transformer (DiT) [12] with billions of parameters that processes thousands of tokens; FLUX.1-dev, for example, needs about five seconds on a state-of-the-art GPU for a single 1024×1024 image, and HunyuanVideo more than a minute for a short video clip.

Many approaches reduce this cost, from faster solvers to distillation into few-step students and quantisation (Section 2.2). Most of them require additional training or change the model's outputs. *Feature caching* is different: it exploits the observation that the network's internal activations change only slightly between consecutive steps. On some steps, the expensive part of the network is skipped and its output is approximated from the steps that were computed. Caching is training-free, leaves the weights and the sampler unchanged, and can be combined with the other approaches.

Every cache has to make two decisions at every step:

1. **When to compute:** should the network be evaluated at this step, or can it be skipped?
2. **What to write:** if the step is skipped, what should be written in place of the missing network output?

The quality of a cache depends on both. A cache that skips at the wrong moments, for example when the content of the image is still forming, loses detail however good its approximation; a cache that skips at the right moments but writes a stale copy of an old output cannot skip many steps in a row.

1.2 Problem statement

At the start of this project, the most relevant prior methods each addressed one of the two decisions. SeaCache [13] decides *when* to compute with a gate that measures the change of the network input after a spectral filter derived from the optimal linear denoiser, so that the gate responds to the content rather than to the noise; but on skipped steps it simply reuses the last computed output. Spectrum [14] decides *what* to write by fitting the whole history of the network's features with Chebyshev polynomials; but it forecasts the full hidden state and is used with simple schedules. Moreover, every filler in the literature, whether it reuses or forecasts, is *open loop*: when the next computed step reveals how wrong the filler was, its error has already been integrated into the sample and is never corrected. Finally, caching had been developed almost exclusively for text-to-image and text-to-video models, although engineering diffusion models are often sampled with even more steps.

This project therefore addresses the following questions:

- How should a spectrally aligned gate be combined with a long-horizon forecaster, and what exactly should be forecast?
- Can the forecaster's design choices be derived rather than tuned, and what can be proven about its error as a function of the skip length?
- Can the observation made at each computed step be used to correct the steps that were skipped before it, without additional network evaluations?
- Do these techniques carry over to diffusion models used in mechanical engineering?

1.3 Objectives and contributions

The objectives of the project were to develop training-free caches that improve the quality–compute trade-off of diffusion transformers at matched compute, to justify their design theoretically, to evaluate them on large open image and video models, and to apply them to mechanical-engineering diffusion models. The contributions are:

1. **ReSPect** (Residual Spectral Prediction, Chapter 4): a cache that couples the spectrally aligned gate with a forecaster of the transformer *residual*, combining a ridge-regularised Chebyshev fit of the residual history with a first-order Taylor correction. We prove error bounds that rank copy-reuse, Taylor extrapolation and Chebyshev forecasting by their dependence on the skip length, show why residuals are better regression targets than hidden states, and derive the blend weight and warm-up analytically. ReSPect achieves speedups of 3.30×, 2.63× and 2.77× on FLUX.1-dev, Wan2.1 and HunyuanVideo with lower error than existing caches at matched or lower compute.
2. **KalSea** (Kalman-Corrected Spectral-Evolution-Aware Caching, Chapter 5): a cache

whose damped-trend Kalman forecaster is followed by *predictor–corrector gap smoothing* (PCGS), which, once the next step is computed, smooths the residuals of the skipped gap with the Rauch–Tung–Striebel smoother, re-integrates the gap through the cheap output layers and continues from the corrected sample. PCGS needs no extra network evaluations and closes the loop that all previous caches leave open. On FLUX.1-dev KalSea reaches 27.72 dB PSNR at 18.1 of 50 evaluations, compared with 26.29 dB published for SeaCache, and it improves on SeaCache on both video models.

3. **Open implementations** of both methods for FLUX.1-dev, HunyuanVideo and Wan2.1, and for KalSea additionally FLUX.2 [dev] and MiniMax-H3, verified against exact invariants.

4. **Applications in mechanical engineering** (Chapter 6): a generalisation of both caches to U-Net denoisers, DDIM with classifier-free guidance and gradient-guided EDM samplers, and their evaluation on two recent open-weight models, Optimize Any Topology for structural topology optimisation (full OpenTO test split, evaluated with the model's own finite-element compliance analysis) and DiffusionPDE for flow-field reconstruction around a cylinder (forward and inverse Navier–Stokes problems), where KalSea reduces the network evaluations of the 2000-step guided sampler by 1.7–2.6×.

The two methods are also described in two research papers that I co-authored with Abhishek Varshney, Dr. Mohd. Arshad, Monty Anderson and Caleb Case.

1.4 Organisation of the report

Chapter 2 reviews diffusion models, acceleration methods, training-free caching, Kalman filtering and smoothing, and diffusion models in mechanical engineering, and identifies the research gaps. Chapter 3 introduces the notation, the residual-caching formulation, the derivation of the spectrally aligned gate and the evaluation methodology. Chapters 4 and 5 present ReSPect and KalSea with their theory, implementation and experiments, and compare the two. Chapter 6 applies both methods to topology optimisation and flow-field reconstruction. Chapter 7 concludes and outlines future work. Appendix A reproduces the key parts of the code.

Chapter 2

Literature Review

This chapter reviews the work on which the project builds. Section 2.1 summarises diffusion and flow-matching models and the transformer backbones that are accelerated in this report. Section 2.2 places training-free caching among the other families of acceleration methods. Sections 2.3 to 2.5 review caching in detail, organised by the two decisions every cache makes: *when* to evaluate the network and *what* to write on the steps it skips. SeaCache and Spectrum, the two methods closest to this project, are discussed in depth. Section 2.6 covers the estimation theory (Kalman filtering and smoothing) on which KalSea rests, and Section 2.7 surveys diffusion models in mechanical engineering, the target of the applications in Chapter 6. The chapter closes with the research gaps that motivated ReSPect and KalSea (Section 2.8).

2.1 Diffusion and flow-matching generative models

Denoising diffusion. Diffusion models were introduced by Sohl-Dickstein et al. [1] as a sequence of small Gaussian perturbations that gradually destroys the structure of the data, together with a learned reverse process that restores it. Ho et al. [2] made the approach practical with denoising diffusion probabilistic models (DDPM): a network is trained to predict the noise added to a clean sample at a randomly chosen noise level, and sampling runs the learned reverse chain over hundreds to a thousand steps. Song et al. [3] unified diffusion and score matching through stochastic differential equations and showed that every diffusion process has a deterministic *probability-flow* ordinary differential equation (ODE) with the same marginals. Deterministic samplers integrate this ODE: DDIM [15] reduces the number of steps to tens, and the EDM framework of Karras et al. [16] separates the design choices of noise schedule, network preconditioning and solver (with a second-order Heun sampler). Improvements to the noise schedule, such as the cosine schedule of Nichol and Dhariwal [17] and the zero-terminal-SNR rescaling of Lin et al. [18], are used by several of the models in this report.

Latent diffusion and guidance. Latent diffusion models [19] run the diffusion process in the compressed latent space of an autoencoder, which makes high-resolution generation affordable; all image and video models in this report, as well as the topology-optimisation model of Chapter 6, are latent models. Conditional generation is usually sharpened by classifier-free guidance [20], which combines a conditional and an unconditional network

evaluation at every step and therefore doubles the sampling cost, or is distilled into a single guidance-conditioned evaluation, as in FLUX.1-dev.

Flow matching and rectified flow. Flow matching [4], rectified flow [5] and stochastic interpolants [21] learn a velocity field that transports noise to data along (nearly) straight paths $x_\sigma = (1-\sigma)x_\star + \sigma\epsilon$. Sampling integrates the learned velocity with Euler steps over a decreasing schedule of σ. Scaled to large transformers [22], this formulation is the basis of the current open text-to-image and text-to-video models.

Diffusion transformers. Early diffusion models used convolutional U-Nets [23, 24]. Peebles and Xie [12] replaced the U-Net by a transformer [25] operating on a sequence of latent patches, with the timestep (and class) injected by adaptive layer normalisation (adaLN), and showed that quality improves steadily with model size. The models accelerated in this report are diffusion transformers (DiTs):

- **FLUX.1-dev** [26], a rectified-flow text-to-image model with 19 double-stream blocks (separate image and text streams with joint attention) followed by 38 single-stream blocks, and guidance distilled into the network;
- **HunyuanVideo** [27], a large text-to-video DiT with a similar dual-/single-stream design;
- **Wan2.1** [28], a text-to-video DiT family (the 1.3B model is used here) sampled with classifier-free guidance and a multistep UniPC solver [29].

Generating one image or video with these models requires tens of sequential evaluations of a network with billions of parameters over thousands of tokens, which is the cost that this project reduces.

2.2 Approaches to faster sampling

The cost of sampling is (number of network evaluations) $\times$ (cost per evaluation). Existing acceleration methods attack one or both factors.

Fast solvers. Higher-order and multistep ODE solvers reduce the number of steps needed for a given accuracy: DDIM [15], the Heun sampler of EDM [16], DPM-Solver and DPM-Solver++ [30, 31], and the predictor–corrector framework UniPC [29]. Below roughly ten to twenty steps, however, the truncation error of any solver becomes visible.

Distillation. Progressive distillation [32], consistency models [33], latent consistency models [34], adversarial diffusion distillation [35] and distribution matching distillation [36]

train a student that samples in one to a few steps. They are very effective but require an expensive training run per model and change the model's outputs.

Quantisation and efficient architectures. Post-training quantisation such as Q-Diffusion [37] and ViDiT-Q [38] lowers the cost of each evaluation, as do efficient attention kernels.

Feature caching. Caching exploits the strong similarity of the network's internal activations at consecutive sampling steps: on some steps, all or part of the network is skipped and its output is approximated from steps that were computed. Caching is *training-free*, leaves the weights and the sampler unchanged, and is complementary to the approaches above. Every cache makes two coupled decisions [13]:

1. a **schedule** (or gate) decides *when* the network is evaluated, and
2. a **reconstructor** (or filler) decides *what* is written to the features on the steps it skips.

The two are coupled through the length of the skipped runs: a content-aware gate cannot preserve quality on long skips if the skipped features are a stale copy, and an accurate long-horizon predictor cannot be used aggressively if the gate responds to noise rather than to content.

2.3 Training-free feature caching

2.3.1 Caching in U-Nets and diffusion transformers

DeepCache [39] observed that the high-level features of a U-Net change slowly across steps and recomputed only the shallow branch on most steps; block caching [40] reuses the outputs of individual blocks whose change is predicted to be small. For diffusion transformers, Δ-DiT [41] caches the deviation of block outputs at the front or back of the network depending on the sampling stage, FORA [42] reuses the attention and MLP outputs at a fixed interval, and ToCa [43] selects individual tokens to recompute. Video models add a temporal axis of redundancy: Pyramid Attention Broadcast (PAB) [44] broadcasts attention outputs at different rates for spatial, temporal and cross attention, and FasterCache [45] reuses features and the guidance branch. Learning-to-Cache [46] learns which layers to cache, and OmniCache [47] distributes reuse globally over the sampling trajectory and filters the noise that reuse introduces.

2.3.2 Dynamic gates: deciding when to compute

Fixed-interval caches spend computation uniformly, although the network output changes quickly at some stages of sampling and slowly at others. Dynamic gates decide online. AdaCache [48] adapts the caching schedule to each video using a motion-aware signal.

TeaCache [49] accumulates the relative ℓ_1 change of the timestep-modulated input of the first transformer block, which is available before the expensive blocks run, and recomputes once the sum exceeds a threshold δ; on skipped steps it adds the last computed *residual* of the block stack. MagCache [50] models the magnitude ratio of successive residuals, DiCache [51] probes a shallow layer to estimate the change of the deep features, and FreqCa [52] treats frequency bands differently. These gates use the raw change of a feature as the signal. As argued in the ReSPect paper, this raw distance is a noise-contaminated drift: at early steps the noise contribution can dominate it, so the gate reacts to stochastic variation rather than to the evolving content.

2.4 SeaCache: a spectrally aligned gate

SeaCache [13] keeps TeaCache's accumulate-and-reset rule but measures the change *after* a spectral-evolution-aware (SEA) filter. Modelling the latent at step t as $x_t = a_t x_0 + b_t \epsilon$ with white noise ϵ, the linear minimum-mean-square-error estimator of x_0 from x_t is a convolution whose frequency response is the Wiener filter

$$G_t(f) = \frac{a_t S_x(f)}{a_t^2 S_x(f) + b_t^2}, \tag{2.1}$$

where S_x is the power spectrum of clean latents. Under a power-law prior $S_x(f) \propto |f|^{-\beta}$, G_t is a low-pass filter at early (noisy) steps and admits progressively higher frequencies as sampling proceeds, reproducing the well-known coarse-to-fine evolution of diffusion sampling. SeaCache filters the first block's modulated input with the gain-normalised G_t for the current noise level and accumulates the relative ℓ_1 change of the filtered input, so that the gate responds to content rather than noise. It uses $\beta = 2$ for images and $\beta = 3$ for videos, always computes the first and last steps, and on skipped steps *reuses* the last computed residual. The full derivation is given in Section 3.4.

During this project two observations about SeaCache became important. First, its reconstructor is plain reuse, whose error grows linearly with the skip length (Chapter 4); SeaCache improves *when* to compute but not *what* to write. Second, its released HunyuanVideo script binds the scheduler that exists when the model is loaded, whereas the HunyuanVideo sampler creates a fresh scheduler for sampling. The gate therefore reads a stale noise level close to one instead of the sampling schedule, which skips step 1 once $\delta \gtrsim 0.21$ and costs 12–14 dB on the affected prompts. KalSea reads the live noise level of the sampler (Chapter 5).

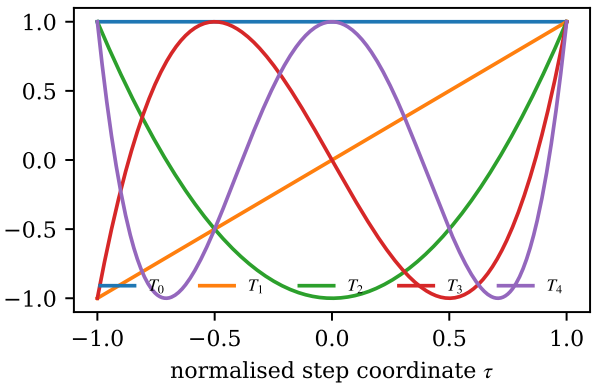


Figure 2.1: The Chebyshev polynomials $T_0, \ldots, T_4$ on the normalised step coordinate $\tau \in [-1, 1]$. Spectrum and ReSPect fit each feature channel with a ridge-regularised expansion in this basis ($M = 4$).

2.5 Forecasting and correcting skipped features

2.5.1 From reuse to forecasting

Reuse is a zero-order prediction. TaylorSeer [53] replaces it by a Taylor extrapolation built from finite differences of the features cached at the most recent computed steps, which is accurate for short skips but uses only local information and amplifies noise at higher orders. HiCache [54] replaces the Taylor basis by scaled Hermite polynomials. FoCa [55] treats feature caching as an ODE and combines a second-order backward-differentiation predictor with a Heun-type calibration.

2.5.2 Spectrum: global Chebyshev forecasting

Spectrum [14] forecasts the hidden-state trajectory globally. The solver index is mapped affinely to $\tau \in [-1, 1]$, every feature channel is expanded in Chebyshev polynomials $T_0, \ldots, T_M$ (Fig. 2.1), and the coefficients are fitted by ridge regression [56] to all features recorded so far. The forecast at a skipped step evaluates the fitted expansion at its coordinate, and is blended with a first-order discrete Taylor step. Spectrum's analysis gives three results that ReSPect builds on: an order-$(P{+}1)$ remainder for an ideal degree-P Taylor predictor, a truncation error of the Chebyshev expansion that decays as ρ^{-M} for functions analytic in a Bernstein ellipse and is independent of the skip length [57], and a stability bound for the ridge estimator. Spectrum forecasts the full hidden state and is used with simple schedules.

2.5.3 Correcting what is written

Several recent methods check or repair the values written on skipped steps. SpeCa [58] accepts or rejects each forecast with a lightweight verification, OmniCache [47] filters the noise introduced by reused features, ERTACache [59] adjusts the integration intervals and

rectifies cache-induced errors with a closed-form model profiled offline, and GP-Refiner [60] regresses the deviation of forecast features online with a Gaussian process. Contemporaneous with KalSea, SpecKCast [61] tracks the DCT coefficients of the *model output* with damped-trend Kalman filters along band-specific warped time and schedules computation by the filter's uncertainty. All of these methods improve the value used at the current or upcoming steps; none of them revises the latent of steps that have already been skipped. This observation is the starting point of KalSea (Chapter 5).

2.6 Filtering, smoothing and predictor–corrector integration

The Kalman filter [62] is the exact recursive minimum-mean-square-error estimator of the state of a linear-Gaussian state-space model; the Rauch–Tung–Striebel (RTS) smoother [63] computes the estimates of past states given all observations, including later ones. Both are covered in the textbook of Särkkä [64]. In target tracking, the integrated Wiener process (constant-velocity) model with state (position, velocity) is standard [65]; damping the velocity, as in the damped-trend models of exponential smoothing [66], prevents forecasts over long horizons from overshooting, and the process-noise intensity can be adapted online so that the observed innovations match their predicted variance [67]. Probabilistic ODE solvers [68, 69] infer the solution of an ODE from evaluations of its vector field by Gauss–Markov filtering and smoothing. In diffusion sampling, predictor–corrector schemes add Langevin steps [3] or a second evaluation [16], and UniPC [29] corrects a multistep solver by reusing the next network evaluation, which the sampler performs anyway. KalSea uses the RTS smoother for a different purpose: to infer the network evaluations that a cache *skipped*.

2.7 Diffusion models in mechanical engineering

Deep generative models have become a design tool in mechanical engineering [70]. Two application areas are relevant to this report.

Structural topology optimisation. Topology optimisation distributes material in a design domain so as to minimise compliance (maximise stiffness) under loads, supports and a volume constraint. The homogenisation method of Bendsøe and Kikuchi [71] and density-based SIMP implementations such as the 99-line and 88-line codes [72, 73] require a finite-element analysis at each of many iterations, which makes the optimisation expensive for large or repeated problems. Generative models learn to propose near-optimal structures directly. TopoDiff [6] showed that a conditional diffusion model guided by physical fields and surrogate models produces better structures than GAN-based approaches. Giannone et al. [74] align the sampling trajectory of the diffusion model with the trajectory of the optimiser, and NITO [75] uses neural implicit fields for resolution-free topology optimisation. Optimize Any

Topology (OAT) [7] combines a resolution- and shape-agnostic neural-field autoencoder with a conditional latent diffusion model trained on OpenTO, a dataset of 2.2 million optimised structures, and predicts minimum-compliance layouts for arbitrary aspect ratios, resolutions, volume fractions, loads and fixtures. Its denoiser is a conditional U-Net sampled with DDIM, and its code and weights are public, which makes it the first application in Chapter 6. Diffusion models are also used for other design tasks, such as boundary-representation CAD models [9] and mechanical metamaterials with a prescribed nonlinear response [8].

Fluid mechanics and PDE-governed fields. Physics-informed neural networks [76] and neural operators such as the Fourier neural operator [77] learn solutions of partial differential equations (PDEs). Diffusion models add the ability to represent uncertainty and to reconstruct fields from incomplete data: Shu et al. [10] reconstruct high-fidelity flow fields from low-fidelity inputs with a physics-informed diffusion model, and Kohl et al. [78] benchmark autoregressive conditional diffusion models for turbulent flow simulation. DiffusionPDE [11] learns the joint distribution of PDE coefficients (or initial states) and solutions with an EDM model and solves forward and inverse problems from sparse observations by guiding the sampler with the observation loss and the PDE residual, in the manner of diffusion posterior sampling [79]. Its sampler uses 2000 guided second-order steps, so sampling is the dominant cost, which makes it the second application in Chapter 6.

In both areas the generative model is evaluated many times per design or per solution, and design studies require many designs. Acceleration that is training-free and leaves the model unchanged is therefore directly useful, yet caching methods have so far been developed and evaluated almost exclusively on text-to-image and text-to-video models.

2.8 Research gaps

Table 2.1 summarises the caching methods along the two decisions of a cache. The review identifies the following gaps.

1. **Gate and reconstructor are improved in isolation.** SeaCache improves the gate but keeps reuse; Spectrum improves the reconstructor but uses simple schedules. Since the two decisions are coupled through the skip length, neither can realise its benefit alone.
2. **The regression target is larger than necessary.** Forecasting methods predict the full hidden state, although on a skipped step the block input is known exactly and only the block-stack residual is missing.
3. **Hyper-parameters of forecasters are tuned empirically,** for example the blend between a local and a global predictor and the number of warm-up steps.
4. **All fillers are open loop.** Reuse and every forecaster predict the skipped steps from the past only. When the next computed step reveals the error, the sampler has already

Table 2.1: Training-free caching methods by their two decisions. "Loop": whether the error of the filler on skipped steps is corrected once the next computed step is observed.

Method	When to compute (schedule)	What to write (filler)	Loop
DeepCache, block caching	fixed / block-wise	reuse of U-Net features	open
Δ-DiT, FORA, ToCa, PAB	fixed interval, block / token level	reuse	open
TeaCache, MagCache, DiCache	accumulated raw change	reuse of residual	open
SeaCache	accumulated SEA-filtered change	reuse of residual	open
TaylorSeer, HiCache	fixed interval	Taylor / Hermite extrapolation	open
Spectrum	fixed / simple	Chebyshev ridge + Taylor (hidden state)	open
FoCa	fixed interval	BDF2 predictor + Heun calibration	open
SpecKCast	filter uncertainty	Kalman forecast of DCT output	open
ReSPect (ours)	accumulated SEA-filtered change	Chebyshev ridge + Taylor (*residual*), analytic blend	open
KalSea (ours)	accumulated SEA-filtered change	Kalman forecast of residual + *PCGS*	**closed**

integrated it into the latent, and no existing method revises the latent of the skipped steps.

5. **Engineering diffusion models have not been accelerated by caching,** even though they are sampled with many steps and used in loops over many designs.

ReSPect (Chapter 4) addresses gaps 1–3: it couples the SEA gate with a residual forecaster whose blend and warm-up are derived analytically. KalSea (Chapter 5) addresses gap 4 with predictor–corrector gap smoothing, and Chapter 6 addresses gap 5.

Chapter 3

Background and Preliminaries

This chapter fixes the notation used in the rest of the report. It describes how the models in this report are sampled (Section 3.1), the anatomy of a diffusion transformer (Section 3.2), the residual-caching formulation shared by ReSPect and KalSea (Section 3.3), the spectrally aligned gate (Section 3.4), and the metrics and protocol used to evaluate a cache (Section 3.5).

3.1 Sampling diffusion and flow-matching models

Flow matching. Rectified-flow models [4, 5, 21, 22] interpolate linearly between a data sample $x_\star$ and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$,

$$x_\sigma = (1-\sigma)\, x_\star + \sigma\, \epsilon, \qquad \sigma \in [0,1], \tag{3.1}$$

and train a network $v_\theta(x_\sigma, \sigma, c)$ to predict the velocity $\epsilon - x_\star$ for a condition c (e.g. a text prompt). Sampling starts from pure noise and integrates the learned velocity over a decreasing schedule $\sigma_0 > \sigma_1 > \cdots > \sigma_N = 0$ with Euler steps,

$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i)\, v_i, \qquad v_i = v_\theta(x_i, \sigma_i, c), \qquad i = 0, \ldots, N-1. \tag{3.2}$$

FLUX.1-dev uses this sampler with $N = 50$ and a resolution-dependent shift of the schedule. Each step costs one evaluation of the network, so a sample costs N network function evaluations (NFE).

Variance-preserving diffusion and DDIM. DDPM-type models [2] define $x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\, \epsilon$ for a decreasing signal level $\bar\alpha_t$. With the network predicting $\hat\epsilon$, $\hat x_0$, or the "velocity" $v = \sqrt{\bar\alpha_t}\epsilon - \sqrt{1-\bar\alpha_t} x_0$ [32], the deterministic DDIM update [15] from t to the previous timestep t' is

$$x_{t'} = \sqrt{\bar\alpha_{t'}}\, \hat x_0 + \sqrt{1-\bar\alpha_{t'}}\, \hat\epsilon, \tag{3.3}$$

which, for every one of these parametrisations, is an *affine* function of the network output: $x_{t'} = A_t x_t + B_t\, v_\theta(x_t, t)$. This linearity is all that KalSea's corrector needs (Chapter 5). Multistep solvers such as UniPC [29] combine several past outputs, but remain linear in the outputs for a fixed solver state.

Signal and noise coefficients. Both families can be written as $x_t = a_t x_0 + b_t \epsilon$, with $(a_t, b_t) = (1-\sigma, \sigma)$ for flow matching and $(\sqrt{\bar\alpha_t}, \sqrt{1-\bar\alpha_t})$ for variance-preserving diffusion.

These coefficients are read from the sampler and determine the spectral filter of the gate (Section 3.4).

3.2 Anatomy of a diffusion transformer

A diffusion transformer [12] computes the network output in three stages:

$$h_i^{\text{in}} = \text{Emb}(x_i), \qquad h_i^{\text{out}} = F\big(h_i^{\text{in}}, c_i\big), \qquad v_i = \text{Out}\big(h_i^{\text{out}}, c_i\big), \tag{3.4}$$

where Emb embeds the latent patches into tokens, F is the stack of transformer blocks, Out is a final adaptive normalisation followed by a linear projection back to latent patches, and c_i embeds the timestep (and, for FLUX.1-dev, the guidance scale and pooled text). Each block modulates its inputs with adaptive layer normalisation:

$$\text{AdaLN}(h, c) = \big(1 + \gamma(c)\big) \odot \text{LayerNorm}(h) + \beta(c), \tag{3.5}$$

with scale γ and shift β regressed from c. In FLUX.1-dev, F consists of 19 double-stream blocks, which keep separate image and text streams that interact through joint attention, followed by 38 single-stream blocks over the concatenated sequence; a 1024×1024 image corresponds to $64 \times 64 = 4096$ image tokens. Almost all of the computation is in F; Emb and Out together cost a fraction of a percent of one evaluation. This asymmetry is what makes residual caching, and KalSea's corrector, cheap.

3.3 Residual caching

A residual cache stores the block-stack residual of the image tokens on computed steps,

$$r_i = h_i^{\text{out}} - h_i^{\text{in}}, \tag{3.6}$$

and on skipped steps runs only Emb and Out:

$$\hat{h}_j^{\text{out}} = h_j^{\text{in}} + \hat{r}_j, \qquad \hat{v}_j = \text{Out}\big(\hat{h}_j^{\text{out}}, c_j\big), \tag{3.7}$$

where the *filler* $\hat{r}_j$ is the last computed residual (reuse, as in TeaCache and SeaCache) or a forecast (ReSPect, KalSea). Because h_j^{in} is computed exactly from the current latent, the reconstruction error of the block-stack output equals the error of the filler:

$$\hat{h}_j^{\text{out}} - h_j^{\text{out}} = \hat{r}_j - r_j. \tag{3.8}$$

Figure 3.1 illustrates one computed and one skipped step. Note that the sampler still advances the latent on skipped steps (Eq. (3.2)) with the approximate velocity $\hat{v}_j$; any error of the filler

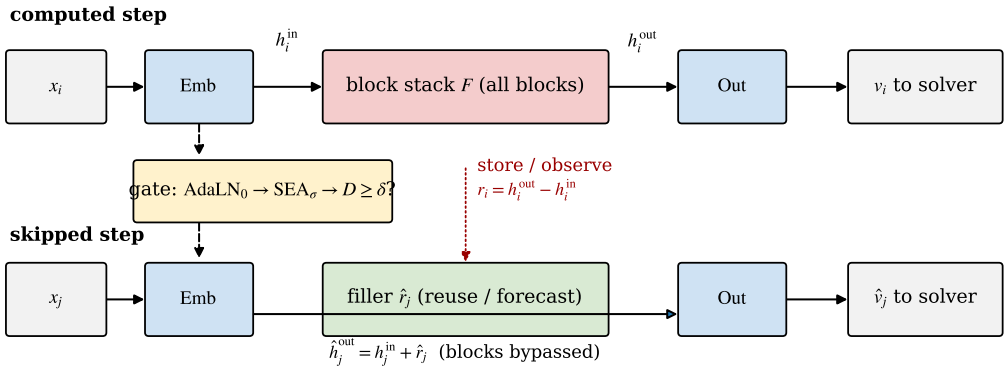


Figure 3.1: Residual caching. On a computed step the full block stack F runs and its residual $r_i = h_i^{\text{out}} - h_i^{\text{in}}$ is stored or observed. On a skipped step the blocks are bypassed: the filler $\hat{r}_j$ is added to the exactly computed input h_j^{in} and the result goes through the output layers. The gate decides which of the two paths a step takes.

is therefore integrated into the latent. This point is central to KalSea.

3.4 The spectrally aligned (SEA) gate

3.4.1 Gate input

A gate must be evaluated at every step and must therefore be cheap. Following TeaCache [49] and SeaCache [13], the gate reads the timestep-modulated input of the first block,

$$I_i = \text{AdaLN}_0\big(h_i^{\text{in}}, c_i\big), \tag{3.9}$$

which is available before any block runs and reflects both the latent and the noise level.

3.4.2 Derivation of the Wiener response

Model the latent at step t as the linear mixture

$$x_t = a_t x_0 + b_t \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{3.10}$$

with ϵ white and independent of the clean latent x_0. This is a signal/noise model for the gate, not a claim about the exact forward process of each model. Consider the linear-MMSE estimator of x_0 from x_t, $h_t^\star = \arg\min_h \mathbb{E}\|h * x_t - x_0\|_2^2$. Assuming wide-sense stationarity, convolution is diagonal in the Fourier basis, and by Parseval's identity the risk of a filter with

frequency response $H_t(f)$ splits per frequency:

$$J_t(H_t) = \sum_f \Big[|H_t(f)|^2 \big(a_t^2 S_x(f) + b_t^2\big) - 2\,\mathrm{Re}\big(H_t(f)\big)\, a_t S_x(f) + S_x(f) \Big], \tag{3.11}$$

where S_x is the power spectrum of x_0. Each summand is a scalar quadratic in $H_t(f)$; setting its derivative to zero gives $H_t(f)\big(a_t^2 S_x(f) + b_t^2\big) = a_t S_x(f)$, i.e. the Wiener response [80]

$$G_t(f) = \frac{a_t\, S_x(f)}{a_t^2\, S_x(f) + b_t^2}, \tag{3.12}$$

which is a minimum because the second derivative $a_t^2 S_x(f) + b_t^2$ is positive.

Spectral evolution. Natural images follow a power law $S_x(f) \propto |f|^{-\beta}$ with $\beta \approx 2$. Inserting this prior into Eq. (3.12): at early steps (a_t small) the denominator is dominated by the noise except near $f = 0$, so G_t is low-pass; as $a_t \to 1$ the passband widens towards high frequencies (Fig. 3.2a). This reproduces the coarse-to-fine evolution of diffusion sampling, in which global structure forms first and details later. SeaCache uses $\beta = 2$ for images and $\beta = 3$ for videos; for multi-dimensional token grids the filter is separable over the spatial (and temporal) axes.

Gain normalisation. The raw responses differ by orders of magnitude in overall gain across steps, which would let high-gain steps dominate the accumulated distance. Each response is therefore normalised to unit mean gain over the frequency bins,

$$G_t^{\mathrm{norm}}(f) = \nu_t\, G_t(f), \qquad \nu_t^{-1} = \frac{1}{L} \sum_{\ell} G_t(f_\ell), \tag{3.13}$$

so that the accumulated distance measures changes of spectral shape rather than changes of filter gain (Fig. 3.2b).

3.4.3 Accumulate-and-reset rule

Let $\mathrm{SEA}_i(I) = \mathcal{F}^{-1}\big[G_{t_i}^{\mathrm{norm}} \odot \mathcal{F}(I)\big]$ denote the filtered gate input of step i on its token grid. The gate computes the relative ℓ_1 change between consecutive filtered inputs and accumulates it:

$$d_i = \frac{\big\|\mathrm{SEA}_i(I_i) - \mathrm{SEA}_{i-1}(I_{i-1})\big\|_1}{\big\|\mathrm{SEA}_{i-1}(I_{i-1})\big\|_1}, \qquad D_i = D_{i-1} + d_i. \tag{3.14}$$

Step i is computed if $D_i \geq \delta$, after which D_i is reset to zero, or if it is the first or the last step; otherwise it is skipped. The reference $\mathrm{SEA}_{i-1}(I_{i-1})$ is refreshed at *every* step, computed or skipped, so that each d_i measures only the incremental drift between consecutive solver states. The single threshold δ sets the compute budget: a larger δ skips more steps. Because

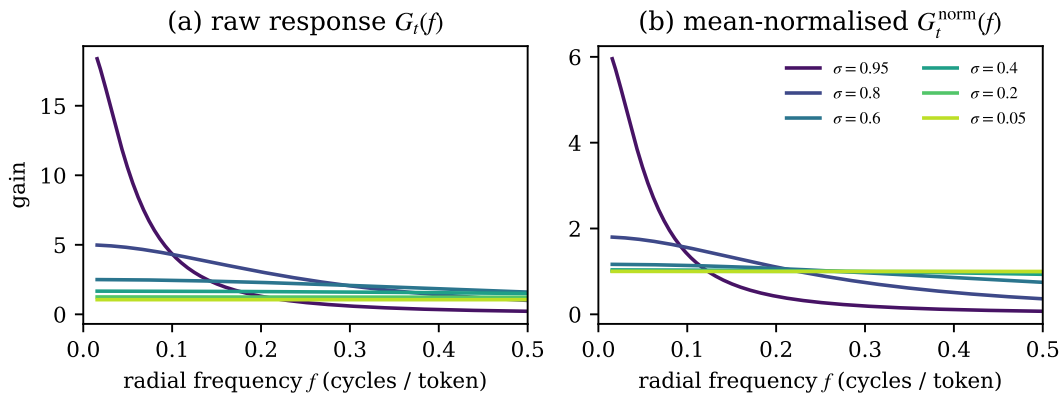


Figure 3.2: Wiener responses of the SEA filter along a flow schedule ($a_t = 1 - \sigma$, $b_t = \sigma$, $S_x(f) = |f|^{-2}$), computed from the closed form of Eq. (3.12). (a) The raw response is low-pass at high noise and widens as $\sigma \to 0$. (b) After the mean-gain normalisation of Eq. (3.13), the responses share a common scale and differ only in shape.

the gate reads only the first block's modulated input, replacing the filler (reuse, ReSPect, KalSea) does not change where the gate fires, which allows fillers to be compared at an identical schedule.

3.5 Evaluation methodology

Compute. The compute of a cache is reported as the mean number of full block-stack evaluations per sample (NFE, out of 50 for the 50-step samplers), counted by hooks on the blocks. TFLOPs follow SeaCache's convention, and latency is the wall-clock time per sample on a single NVIDIA B200 GPU in bf16. Because a content-aware gate need not realise the same skip rate as another method at a shared δ, methods are compared at *matched or lower* compute, never at matched δ.

Fidelity. The goal of a cache is to reproduce the output of the uncached sampler. Every accelerated sample is therefore compared with the output of the uncached 50-step sampler for the same prompt and seed, using

- **PSNR** on 8-bit RGB, $10 \log_{10}(255^2/\text{MSE})$ (higher is better);
- **SSIM** [81], the structural similarity index (higher is better);
- **LPIPS** [82], a learned perceptual distance on AlexNet features (lower is better).

For videos the metrics are averaged over frames. In the KalSea study, differences between methods run in the same setup are reported with paired bootstrap 95% confidence intervals over prompts (10,000 resamples, percentile method) and the fraction of prompts on which a method is better.

Benchmarks and models. Text-to-image experiments use FLUX.1-dev [26] at 1024×1024 with 50 steps, embedded guidance 3.5 and the 200 DrawBench prompts [83], with the prompt index as seed. Text-to-video experiments use HunyuanVideo [27] and Wan2.1-T2V-1.3B [28] with 65 frames and 50 steps on VBench prompts [84]. The exact settings of each study are given in Chapters 4 and 5.

Chapter 4

ReSPect: Residual Spectral Prediction

This chapter presents ReSPect (Residual Spectral Prediction for Efficient Caching of Transformers), the first method developed in this project. ReSPect is a training-free cache for diffusion transformers that combines the spectrally aligned gate of Section 3.4 with a forecaster of the skipped *residuals*. The forecaster is a weighted sum of a ridge-regularised Chebyshev regression over the accumulated residual history, which captures the global trajectory, and a first-order discrete Taylor correction, which captures local dynamics. We show that its forecast error admits a skip-length-independent bound, whereas the bounds of copy-reuse and Taylor extrapolation grow with the skip length, and we derive its blend weight and warm-up length analytically. On FLUX.1-dev, Wan2.1-1.3B and HunyuanVideo, ReSPect achieves speedups of 3.30×, 2.63× and 2.77× over full evaluation with lower output error than existing caching baselines at matched or lower compute. The chapter follows our ReSPect paper, with the derivations given in full and an additional description of the implementation.

4.1 Motivation

As discussed in Section 2.2, a cache makes two coupled decisions at every step: a schedule decides whether the network is evaluated, and a reconstructor decides what is written to the features when it is not. Improving either one in isolation leaves the other as the bottleneck. The literature review identified two limitations:

- **Noisy schedules.** Many gates use the raw distance between consecutive features [49, 51]. Under the mixture $x_t = a_t x_0 + b_t \epsilon$ the noise contribution can dominate this distance at early steps, so the gate responds to stochastic variation. A timestep-dependent spectral filter isolates the content-sensitive drift (Section 3.4).
- **Local reconstructors.** Most reconstructors use local temporal information, copying the last computed features [42, 44] or extrapolating from finite differences [53]. Their approximation error grows with the skip length, and sample quality degrades at high speedups. Global trajectory fitting [14] still predicts the full hidden state.

ReSPect specifies both decisions together (Fig. 4.2). It keeps the spectrally aligned gate and forecasts only the transformer residual, because the block input at skipped steps is known exactly. This removes the predictable component from the regression target and yields a smaller, centred trajectory.



Figure 4.1: Images generated by FLUX.1-dev with ReSPect, which keeps high image fidelity at a 2.43× acceleration.

4.2 Problem setup

Consider a latent generative model whose sampling trajectory $\{x_t\}_{t=0}^{T}$ is produced by a discrete solver, where $t \in \{0, \dots, T\}$ indexes solver steps from the initial noise towards the clean output. At step t the latent x_t is mapped to the input $I_t = \phi(x_t, t)$ of a transformer block, whose output $h^{\text{out}}(t)$ is produced from the input $h^{\text{in}}(t)$. The block residual

$$r(t) = h^{\text{out}}(t) - h^{\text{in}}(t) \tag{4.1}$$

induces a second trajectory $\{r(t)\}$ alongside $\{x_t\}$. On an evaluated step $h^{\text{out}}(t) = h^{\text{in}}(t) + r(t)$ holds by definition. On a skipped step the input is known and the reconstructor supplies $\hat{r}(t)$, yielding $\hat{h}^{\text{out}}(t) = h^{\text{in}}(t) + \hat{r}(t)$, so that the reconstruction error coincides with the residual error,

$$\hat{h}^{\text{out}}(t) - h^{\text{out}}(t) = \hat{r}(t) - r(t) \tag{4.2}$$

(Eq. (3.8)). The reconstruction problem is therefore reduced to forecasting the residual trajectory at unevaluated steps.

The gate. ReSPect uses the spectrally aligned gate of Section 3.4 unchanged in form: the first-block input is filtered with the gain-normalised Wiener response G_t^{norm} of Eqs. (3.12) and (3.13), each adjacent pair (I_t, I_{t+1}) is compared with its own pair of responses,

$$\tilde{\Delta}_t = \mathrm{L1}_{\text{rel}}\big(\mathcal{P}(G_t^{\text{norm}}, I_t),\, \mathcal{P}(G_{t+1}^{\text{norm}}, I_{t+1})\big), \qquad \mathcal{P}(G, I) = \mathcal{F}^{-1}\big[G \odot \mathcal{F}(I)\big], \tag{4.3}$$

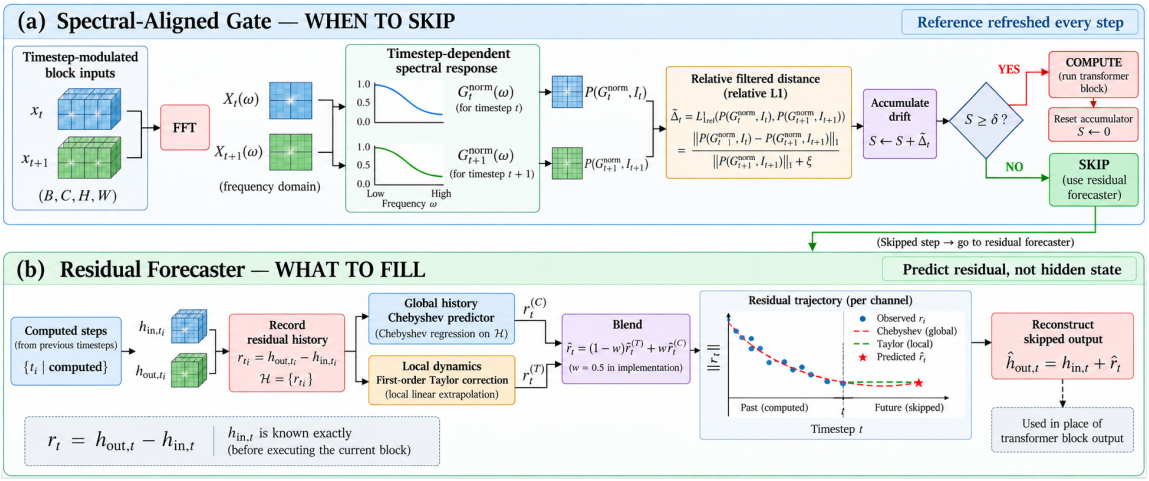


Figure 4.2: Overview of ReSPect. (a) The spectrally aligned gate decides adaptively when to evaluate the network. (b) The residual forecaster combines a global Chebyshev prediction with a local first-order Taylor correction and writes $h^{\text{in}} + \hat{r}$ on skipped steps.

and the network is evaluated when the accumulated $\tilde{\Delta}_t$ exceeds δ. The reference is refreshed at every step, including skipped steps, so each $\tilde{\Delta}_t$ measures only the incremental drift; on HunyuanVideo this choice is worth +2.50 dB at matched compute (Section 4.8.5). The rest of this chapter specifies what is written on the skipped steps.

4.3 Predictors on the sampling trajectory

We state the analysis for one residual channel; the predictor is applied independently across channels. Consider a skip from the last evaluated step t_k to the current step t_j. Every reconstructor is a predictor $\hat{r}(t_j)$ built from the residuals recorded at evaluated steps; the candidates differ in the smoothness they assume of r.

4.3.1 Order zero: copy-reuse

Copy-reuse sets $\hat{r}(t_j) = r(t_k)$. If r is L-Lipschitz on the sampling interval, then directly from the definition

$$|r(t_j) - r(t_k)| \le L\,|t_j - t_k|. \tag{4.4}$$

The bound is tight, since $r(t) = Lt$ attains equality for every skip, and Lipschitz continuity is necessary in the worst case: for a non-Lipschitz residual the ratio $|r(t_j) - r(t_k)|/|t_j - t_k|$ is unbounded as the skip shrinks. Copy-reuse uses no derivative information, so additional smoothness ($r \in C^P$, $P \ge 1$) cannot improve the worst-case rate. *The error of reuse grows linearly with the skip length.*

4.3.2 Order P: Taylor extrapolation

Assume $r \in C^{P+1}$. The degree-P Taylor expansion of r about t_k is

$$r(t_j) = \sum_{p=0}^{P} \frac{r^{(p)}(t_k)}{p!}(t_j - t_k)^p + R_P(t_j), \qquad R_P(t_j) = \frac{r^{(P+1)}(\xi)}{(P+1)!}(t_j - t_k)^{P+1} \tag{4.5}$$

for some ξ between t_k and t_j. Hence $|R_P(t_j)| \le \|r^{(P+1)}\|_\infty |t_j - t_k|^{P+1}/(P+1)!$, an order-$(P{+}1)$ dependence on the skip length [14, Thm. 3.1]. This is the *ideal* predictor, because it uses the exact derivatives at t_k, which a training-free cache never computes.

The implemented (discrete) predictor. In practice the derivatives are replaced by backward differences formed from $P{+}1$ neighbouring evaluated residuals $s_0 = t_k, s_1, \ldots, s_P$, so the predictor is the degree-P interpolant p_P through those nodes, with remainder

$$r(t_j) - p_P(t_j) = \frac{r^{(P+1)}(\xi)}{(P+1)!} \prod_{q=0}^{P} (t_j - s_q) \tag{4.6}$$

for some ξ in the convex hull of $\{t_j, s_0, \ldots, s_P\}$. Only the $q = 0$ factor shrinks with the skip length $\ell = |t_j - t_k|$; the remaining factors are fixed by the positions of the earlier evaluated nodes. The $(P{+}1)$-order rate therefore requires a node-spacing condition $\max_q |t_j - s_q| \le C_{\text{node}}\, \ell$, under which $|r(t_j) - p_P(t_j)| \le C_{\text{node}}^{P+1} \|r^{(P+1)}\|_\infty \ell^{P+1}/(P+1)!$. A dynamic gate does not guarantee a uniform C_{node}: with the earlier nodes held fixed and $\ell \to 0$, the error is $O(\ell)$, not $O(\ell^{P+1})$.

Proposition 4.1 (The failure is sharp)**.** *Take $P = 1$, $r(t) = t^2$, nodes $s_0 = t_k = 0$ and $s_1 = -H$ with $H > 0$, and forecast point $t_j = \ell > 0$. Then $|r(\ell) - p_1(\ell)|/\ell^2 = 1 + H/\ell$, which is unbounded as $H/\ell \to \infty$.*

Proof. The linear interpolant through $(0, 0)$ and $(-H, H^2)$ is $p_1(t) = -Ht$, so $|r(\ell) - p_1(\ell)| = \ell^2 + H\ell = \ell(\ell + H)$. Dividing by ℓ^2 gives the claim. □

Even uniform backward spacing does not recover the ideal rate: for $t_j = t_k + \ell$ and $s_q = t_k - qh$, $\prod_{q=0}^{P} |t_j - s_q| = \prod_{q=0}^{P} (\ell + qh)$ is a degree-$(P{+}1)$ polynomial in ℓ that coincides with ℓ^{P+1} only in the limit $h/\ell \to 0$. In addition, the difference operator amplifies observation noise by binomial coefficients of order 2^P. A higher order therefore tightens the bound only for short skips and becomes unstable as $|t_j - t_k|$ grows, and the predictor discards all but the $P{+}1$ most recent evaluations. These bounds motivate a *global* predictor that exploits the full observed trajectory.

4.4 Residual forecasting with Chebyshev bases

To predict $r(t_j)$ we fit the accumulated residual trajectory. The solver-step index is mapped affinely from the fixed solver-step range to $\tau \in [-1, 1]$, and each residual channel (of feature dimension F) is expanded in the Chebyshev basis $T_0, \ldots, T_M$ of degree M (Fig. 2.1). Stack the K recorded residuals row-wise into the observation matrix $H \in \mathbb{R}^{K\times F}$, let $\Phi \in \mathbb{R}^{K\times(M+1)}$ evaluate the basis at the recorded steps, and regress the coefficient matrix $C \in \mathbb{R}^{(M+1)\times F}$ with ridge penalty λ [56]:

$$C^\star = \arg\min_C \|\Phi C - H\|_F^2 + \lambda\|C\|_F^2 = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top H. \tag{4.7}$$

The fit is re-solved whenever a new residual is computed; the window retains the last K observations, which exceeds the number of solver steps in all reported runs. The forecast at a skipped step evaluates the fitted expansion at its coordinate, $\hat{r}(t_j) = \sum_{m=0}^{M} \hat{C}_m T_m(\tau_j)$.

Truncation error. Let $p_M^\star$ be the exact degree-M Chebyshev truncation of r. Assume that r extends analytically to the Bernstein ellipse E_ρ ($\rho > 1$, foci ± 1, semi-axis sum ρ) on which $|r| \le B$. The Chebyshev coefficients have the contour representation $c_m = \frac{1}{\pi i}\oint_{E_\rho} r(z)\, T_m(z)^{-1}\, dz/z$ (up to the standard factor for $m = 0$), and on E_ρ one has $|T_m| \ge (\rho^m - \rho^{-m})/2$, which yields $|c_m| \le 2B\rho^{-m}$ for $m \ge 1$ and $|c_0| \le B$. Since $|T_m(\tau)| \le 1$ on $[-1, 1]$, truncating at degree M leaves [14, Thm. 3.2]

$$\|r - p_M^\star\|_\infty \le \sum_{m=M+1}^{\infty} 2B\rho^{-m} = \frac{2B}{\rho - 1}\rho^{-M}. \tag{4.8}$$

The right-hand side depends on M, ρ and B, but not on the evaluation point: *the approximation-theoretic term is independent of the skip length*. Because the coefficients are estimated from finitely many observations, we also control the deviation of the estimator from the ideal truncation.

Lemma 4.2 (Ridge stability; restated from 14, Thm. 3.3)**.** *Let $\hat{r}$ be the Chebyshev component from Eq.* (4.7)*, $p_M^\star$ the exact degree-M truncation and $\sigma_{\min} = \sigma_{\min}(\Phi)$ (zero whenever $K < M + 1$). At any forecast coordinate τ_j,*

$$|\hat{r}(\tau_j) - p_M^\star(\tau_j)| \le \varepsilon_M \frac{(M+1)K}{\sigma_{\min}^2 + \lambda} + \frac{\lambda\sqrt{M+1}}{\sigma_{\min}^2 + \lambda} \cdot \frac{2B}{\sqrt{1 - \rho^{-2}}}, \tag{4.9}$$

where ε_M is the right-hand side of Eq. (4.8)*.*

Proof. Fix one channel and take $\lambda > 0$. Write the K observations as $y = \Phi c^\star + e$, where $c^\star$ are the exact Chebyshev coefficients of $p_M^\star$ and, by Eq. (4.8), $|e_k| \le \varepsilon_M$, so $\|e\|_2 \le \sqrt{K}\varepsilon_M$.

The ridge estimate is $\hat{c} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y$, hence

$$\hat{c} - c^\star = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top e - \lambda (\Phi^\top \Phi + \lambda I)^{-1} c^\star. \tag{4.10}$$

The forecast error is the inner product of $\hat{c} - c^\star$ with the basis row $\varphi(\tau_j) = [T_0(\tau_j), \dots, T_M(\tau_j)]$, and $\|\varphi(\tau_j)\|_2 \le \sqrt{M+1}$.

Variance term. With the singular values $\sigma_1 \ge \cdots \ge \sigma_{M+1} = \sigma_{\min}$ of Φ, $\|(\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top\|_2 = \max_i \sigma_i / (\sigma_i^2 + \lambda) \le \sigma_1 / (\sigma_{\min}^2 + \lambda)$, and since every entry of Φ is bounded by one, $\sigma_1 \le \|\Phi\|_F \le \sqrt{(M+1)K}$. Therefore

$$\left|\varphi(\tau_j)(\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top e\right| \le \sqrt{M+1} \cdot \frac{\sqrt{(M+1)K}}{\sigma_{\min}^2 + \lambda} \cdot \sqrt{K}\varepsilon_M = \frac{(M+1)K}{\sigma_{\min}^2 + \lambda}\, \varepsilon_M. \tag{4.11}$$

Bias term. The coefficient bound $|c_m^\star| \le 2B\rho^{-m}$ gives $\|c^\star\|_2 \le 2B\big(\sum_{m \ge 0} \rho^{-2m}\big)^{1/2} = 2B/\sqrt{1-\rho^{-2}}$, independent of M. The eigenvalues of $(\Phi^\top \Phi + \lambda I)^{-1}$ are $1/(\sigma_i^2 + \lambda)$, so its spectral norm is $1/(\sigma_{\min}^2 + \lambda)$ and

$$\left|\varphi(\tau_j)\, \lambda (\Phi^\top \Phi + \lambda I)^{-1} c^\star\right| \le \frac{\lambda \sqrt{M+1}}{\sigma_{\min}^2 + \lambda} \cdot \frac{2B}{\sqrt{1-\rho^{-2}}}. \tag{4.12}$$

Adding the two bounds proves the lemma. □

Adding ε_M once more by the triangle inequality bounds the total error $|r(\tau_j) - \hat{r}(\tau_j)|$. The two terms expose a trade-off. Each observation adds a positive semi-definite rank-one term to $\Phi^\top \Phi$, so $\sigma_{\min}$ is non-decreasing in K and the factor $K/(\sigma_{\min}^2 + \lambda)$ stays bounded rather than growing; before K reaches $M{+}1$ the design is rank-deficient and λ alone controls the bound, at the price of bias. Section 4.6 turns this condition into an explicit warm-up threshold.

Node spacing versus design conditioning. The node-spacing condition of the Taylor analysis and the design conditioning $\sigma_{\min}(\Phi)$ of Theorem 4.2 are distinct properties of the realised schedule, and neither implies the other. If the forecast distance ℓ stays fixed while the earlier nodes coalesce onto t_k, the spacing condition holds with $C_{\text{node}} \to 1$, yet the rows of Φ become nearly identical and $\sigma_{\min} \to 0$. Conversely, a fixed well-spread design keeps $\sigma_{\min}$ bounded away from zero, while a forecast with $\ell \to 0^+$ against a fixed distant node drives $\max_q |t_j - s_q|/\ell \to \infty$. Each analysis therefore controls a separate failure mode.

4.5 Why forecast the residual?

On a skipped step the block input h^{in} is known exactly from the current sampling state. For a state forecast $\hat{h}^{\text{out}}$ and a residual forecast $\hat{r}$,

$$\hat{h}^{\text{out}} - h^{\text{out}} = (\hat{r} + h^{\text{in}}) - (r + h^{\text{in}}) = \hat{r} - r. \tag{4.13}$$

Once the predictor is fixed, state and residual formulations therefore have identical reconstruction error; they differ only in their regression target. Stacking the observed rows gives the exact identity $H_{\text{out}} = H_{\text{in}} + R$, where H_{in} varies with the step and is not rank one, so no low-rank structure is assumed. Each hidden-state observation carries the large input component, which the sampler already knows; fitting the full state inflates the regression target without adding anything that needs predicting. The ridge map is linear in its target, so scaling the target scales the fitted coefficients and the absolute shrinkage bias, while the shrinkage factor $\lambda/(\sigma_{\min}^2 + \lambda)$ is unchanged. The constants of Theorem 4.2 scale with the target norm through B. Forecasting R directly keeps the target small-normed and tightens every constant in proportion. This is a target-scale advantage rather than a conditioning one, since all fits share the same design matrix Φ. The analysis assumes that the residual trajectory satisfies the same analyticity condition with comparable ρ; consistently, the M-sweep of Table 4.5 saturates at the same small M used throughout. The skipped-step update is simply $h \leftarrow h + \hat{r}$.

4.6 Analytic blend and warm-up

Blend. The forecast is a convex combination of the discrete Taylor predictor $\hat{r}_T$ and the Chebyshev predictor $\hat{r}_C$,

$$\hat{r}_w = (1 - w)\,\hat{r}_T + w\,\hat{r}_C. \tag{4.14}$$

For the risk model we assign the Taylor predictor's systematic error to a squared bias b_T^2 and the Chebyshev estimator's finite-history uncertainty to a variance σ_C^2, and treat the two errors as uncorrelated. Omitting w-independent terms, the risk is

$$R(w) = (1 - w)^2 b_T^2 + w^2 \sigma_C^2, \tag{4.15}$$

and setting $R'(w) = 0$ gives $w^\star = b_T^2/(b_T^2 + \sigma_C^2)$. Early in sampling the drift is fast (b_T large) and the fit has seen few points (σ_C large); later both decrease as the trajectory settles and observations accumulate. Measured trajectories keep the two terms within a factor of two of each other, which places $w^\star$ near $1/2$ throughout. Because the bias and variance are not observable at inference time, we use the symmetric fixed weight $w = 0.5$, which needs no adaptation or hyper-parameter search. Section 4.8.5 shows that this choice is robust across

Algorithm 1 ReSPect sampling (one sample)

Require: initial noise x_0, solver with N steps, threshold δ; M=4, K=100, λ=0.1, w=0.5

$D \leftarrow 0$; history $\mathcal{H} \leftarrow \emptyset$; $I_{\text{prev}} \leftarrow$ none
for $t = 0, \ldots, N-1$ **do**
  $h^{\text{in}} \leftarrow \text{Emb}(x_t)$; $\tilde{I} \leftarrow \mathcal{P}(G_t^{\text{norm}}, \text{AdaLN}_0(h^{\text{in}}, c_t))$
  if $t \in \{0, N-1\}$ **or** I_{prev} is none **then**
    compute $\leftarrow$ true; $D \leftarrow 0$
  else
    $D \leftarrow D + \text{L1}_{\text{rel}}(\tilde{I}, I_{\text{prev}})$; compute $\leftarrow (D \geq \delta)$; **if** compute **then** $D \leftarrow 0$
  $I_{\text{prev}} \leftarrow \tilde{I}$ ▷ reference refreshed at every step
  if compute **then**
    $r \leftarrow F(h^{\text{in}}) - h^{\text{in}}$; append (t, r) to $\mathcal{H}$ (keep last K); refit $C^\star$ by Eq. (4.7)
  else
    $\hat{r}_T \leftarrow$ first-order discrete Taylor step from the last two entries of $\mathcal{H}$
    if $|\mathcal{H}| \geq 4$ **then** $r \leftarrow (1-w)\hat{r}_T + w \sum_m C_m^\star T_m(\tau_t)$
    else $r \leftarrow \hat{r}_T$
  $v_t \leftarrow \text{Out}(h^{\text{in}} + r, c_t)$; $x_{t+1} \leftarrow \text{SolverStep}(x_t, v_t)$
return x_N

models and budgets, whereas $w = 1$ removes the low-bias local term precisely when the polynomial fit is least determined.

Warm-up. With $n < M + 1$ observations, Eq. (4.7) is rank-deficient ($\sigma_{\min} = 0$), the observations cannot identify all coefficient directions, and the fit is dominated by the penalty (the bias term of Theorem 4.2). The first-order discrete Taylor step is exact on the last two observations by construction and therefore strictly dominates in this regime: it reduces to copy-reuse at $n = 1$ and extrapolates linearly from $n = 2$ onward. In practice the regularised fit is determined one observation early, so blending starts at $n \geq 4$ (with $M = 4$) rather than at $M + 1$. The warm-up is thus the degrees-of-freedom crossover rather than a tuned choice.

Compute and memory. Skip decisions accumulate per sample, so prompts in a batch never share a schedule. Each sample stores one $(M+1) \times F$ coefficient matrix and K recorded residuals, i.e. $O((M+1)F + KF)$ memory, with fitting cost $O(K(M+1)F)$ for $M \ll K \ll F$ (the $(M+1)^3$ Cholesky term is negligible) and prediction cost $O((M+1)F)$. The measured overhead is about 0.09 s per FLUX.1-dev image.

Algorithm 1 summarises the complete procedure.

4.7 Implementation

The ReSPect code release follows the layout of SeaCache: one self-contained folder per backbone, each with a copy of the SEA filter (`util_seacache.py`), the forecaster

(`spectrum_forecaster.py`) and a generate script (`respect_generate.py`). Key excerpts are reproduced in Appendix A.

- **FLUX.1-dev (diffusers).** The script replaces `FluxTransformer2DModel.forward` at import time. The gate takes the first double-stream block's `norm1` output, lays it out on its (H, W) token grid using the rotary image ids, applies the SEA filter with $\beta = 2$ over the two spatial axes, and accumulates the relative ℓ_1 distance. A fresh `SpectrumResidualForecaster` is created for every image, so the forecaster state never leaks between prompts. Command-line flags: `-seacache_thresh` (δ) and `-spectrum_m/k/lam/w/min_cheb_obs`.
- **Wan2.1.** The script is a copy of the official `generate.py` with patched `t2v_generate`, `i2v_generate` and model forward. Classifier-free guidance calls the transformer twice per step; the two calls alternate, so the gate and the forecaster keep separate states for the even (conditional) and odd (unconditional) calls, with step index $\lfloor \mathrm{cnt}/2 \rfloor$.
- **HunyuanVideo.** The forward of `HYVideoDiffusionTransformer` is replaced on the loaded sampler. Because HunyuanVideo residuals are large, the ridge solve is chunked over the feature dimension (`fit_chunks=8`); each chunk is an independent solve with the same Cholesky factor, so the result is identical while peak memory drops.

The forecaster stores the residual history in bf16, performs the ridge solve in fp32 with a Cholesky factorisation of $\Phi^\top \Phi + \lambda I$ (with jitter as a fallback), and maps the step index to τ over the fixed range $[0, 50]$ rather than the buffer's minimum and maximum.

4.8 Experiments

4.8.1 Settings

We evaluate FLUX.1-dev on the 200 DrawBench prompts (1024×1024, 50 steps, guidance 3.5, bf16, seeds $0 + i$), and HunyuanVideo and Wan2.1-1.3B on 946 VBench prompts (480p, 65 frames, 50 steps). Metrics are PSNR, SSIM and LPIPS against the uncached same-seed reference (Section 3.5). All qualities and latencies are measured on a single NVIDIA B200 GPU. Each ReSPect operating point uses equal or less compute than the baseline rows it is compared against. The forecaster uses $M = 4$, $K = 100$, $\lambda = 0.1$, $w = 0.5$, a first-order discrete Taylor term and a Taylor-only warm-up until $n \geq 4$; the first and last steps are always computed.

4.8.2 Text-to-image results

Table 4.1 reports FLUX.1-dev. At $\delta = 0.37$ ReSPect attains 26.24 dB, +3.46 dB over TaylorSeer at lower latency (2.19 s vs. 2.52 s); at $\delta = 0.6$ it attains 21.63 dB, +1.66 dB while

Table 4.1: FLUX.1-dev, DrawBench-200, 1024×1024, 50 steps. Speedup is the latency of the original sampler (5.31 s) over the row latency. ReSPect spends equal or less compute than every baseline in its block.

Method	PSNR↑	SSIM↑	LPIPS↓	Lat. (s)	Speedup
Original (50 steps)	–	–	–	5.31	1.00×
Vanilla, 25 steps	15.55	0.668	0.409	2.72	1.95×
TeaCache ($\delta = 0.3$)	20.76	0.810	0.211	2.85	1.86×
TaylorSeer ($S = 3$)	22.78	0.828	0.163	2.52	2.11×
ReSPect ($\delta = 0.37$)	**26.24**	**0.895**	**0.093**	**2.19**	**2.43×**
Vanilla, 15 steps	17.84	0.740	0.305	1.68	3.16×
ToCa	18.40	0.700	0.324	2.25	2.36×
TeaCache ($\delta = 0.6$)	17.21	0.714	0.348	1.72	3.09×
TaylorSeer ($S = 5$)	19.97	0.762	0.236	1.95	2.72×
ReSPect ($\delta = 0.6$)	**21.63**	**0.823**	**0.178**	**1.61**	**3.30×**

Table 4.2: Text-to-video results on VBench (480p, 65 frames, 50 steps). Latencies on one NVIDIA B200 (bf16); speedup is relative to the original latency (HunyuanVideo 77.3 s, Wan2.1 28.8 s).

HunyuanVideo (orig. 77.3 s)					Wan2.1-1.3B (orig. 28.8 s)				
Method	PSNR↑	SSIM↑	LPIPS↓	Lat./Speedup	Method	PSNR↑	SSIM↑	LPIPS↓	Lat./Speedup
Vanilla 25	19.97	0.731	0.263	38.7 / 2.00×	TeaCache 0.09	20.84	0.721	0.171	14.9 / 1.93×
TeaCache 0.12	23.40	0.805	0.133	42.0 / 1.84×	TaylorSeer S2	16.15	0.543	0.336	15.3 / 1.88×
TaylorSeer S2	24.14	0.820	0.152	42.0 / 1.84×	**ReSPect 0.17**	**32.21**	**0.947**	**0.033**	14.8 / 1.95×
ReSPect 0.27	**34.07**	**0.946**	**0.046**	39.9 / 1.94×					
Vanilla 15	17.49	0.662	0.371	23.2 / 3.33×	TeaCache 0.15	18.88	0.645	0.245	10.4 / 2.77×
TeaCache 0.2	20.42	0.734	0.172	30.2 / 2.56×	TaylorSeer S3	14.18	0.453	0.455	11.2 / 2.57×
TaylorSeer S3	20.42	0.733	0.242	30.2 / 2.56×	**ReSPect 0.31**	**27.20**	**0.888**	**0.082**	10.9 / 2.63×
ReSPect 0.48	**29.20**	**0.891**	**0.100**	27.9 / 2.77×					

faster (1.61 s vs. 1.95 s). Both points dominate the published quality–compute frontier (Fig. 4.4). Figure 4.3 shows two DrawBench prompts at the aggressive budget.

4.8.3 Text-to-video results

The gains are largest on video (Table 4.2). On Wan2.1, ReSPect reaches 32.21 dB at $\delta = 0.17$ (+11.4 dB over TeaCache) and 27.20 dB at $\delta = 0.31$ (+8.3 dB), with LPIPS less than a third of any baseline. On HunyuanVideo the margins are +9.9 dB and +8.8 dB at $\delta = 0.27$ and 0.48. Figure 4.5 shows opening frames and Figs. 4.9 and 4.10 full frame strips.

4.8.4 Cost of the forecaster and comparison with SeaCache

A latency column combines the effects of schedule and reconstructor. Running copy-reuse and the forecast in one harness at the same δ fixes the schedule, because the gate reads the modulated block input and the reconstructor does not change where it fires; the realised NFE is then identical. Table 4.3 shows that the forecast adds 1.7–4.3% wall-clock time

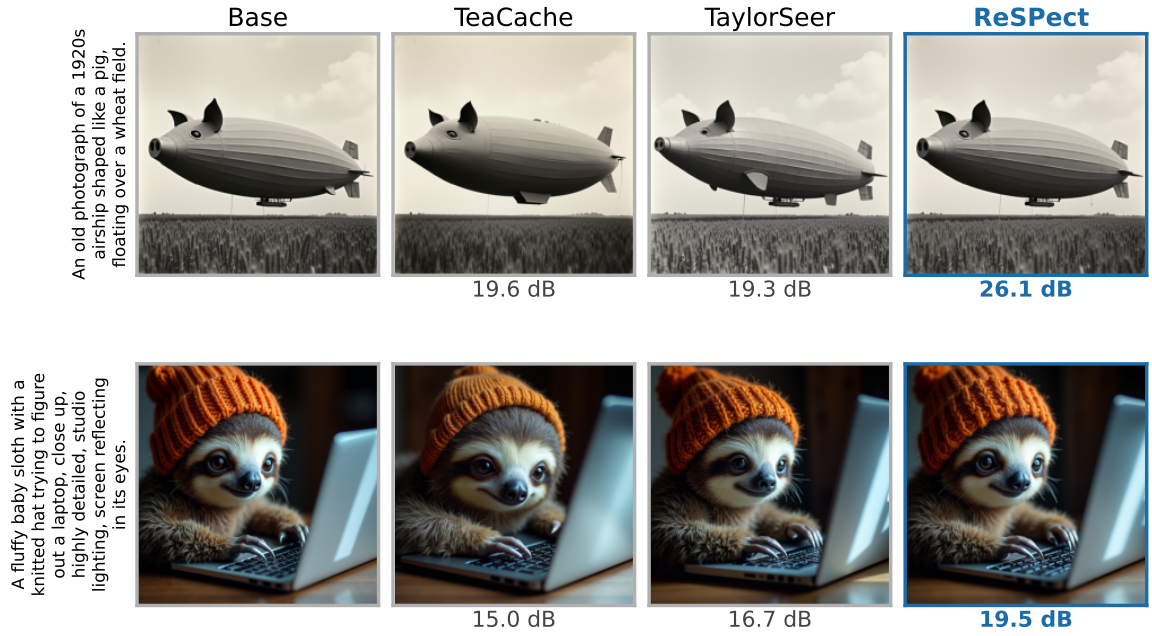


Figure 4.3: FLUX.1-dev at $\delta = 0.6$. The PSNR with respect to the uncached image is printed under each panel (top prompt: 19.6 / 19.3 / 26.1 dB; bottom prompt: 15.0 / 16.7 / 19.5 dB for TeaCache / TaylorSeer / ReSPect). TeaCache and TaylorSeer use equal or more compute.

Table 4.3: Cost of the reconstructor at a fixed schedule (same gate and δ, one B200). NFE is identical within each pair.

Model	δ	NFE	Reuse (s)	Forecast (s)	Δ
FLUX.1-dev	0.3	21	2.38	2.47	+3.8%
FLUX.1-dev	0.6	13	1.55	1.61	+3.9%
Wan2.1-1.3B	0.17	46	14.31	14.80	+3.4%
Wan2.1-1.3B	0.31	32	10.49	10.94	+4.3%
HunyuanVideo	0.27	24	39.20	39.86	+1.7%
HunyuanVideo	0.48	16	27.38	27.90	+1.9%

Table 4.4: ReSPect vs. SeaCache copy-reuse at matched schedules (same δ, prompts and gate; only the reconstruction differs).

Setting	Method	NFE	PSNR	SSIM
FLUX δ=0.37	SeaCache	18.1	25.45	0.880
(n=200)	ReSPect	18.1	26.24	0.895
FLUX δ=0.6	SeaCache	13.0	23.08	0.848
(n=20)	ReSPect	13.0	23.57	0.852
Wan δ=0.31	SeaCache	32.4	28.80	0.904
(n=20)	ReSPect	32.3	29.73	0.917

over copy-reuse. Table 4.4 compares ReSPect with SeaCache's copy-reuse at matched schedules: the forecast wins on PSNR and SSIM at every setting, by +0.79 dB on FLUX $\delta = 0.37$ ($n = 200$), +0.49 dB on FLUX $\delta = 0.6$ ($n = 20$) and +0.93 dB on Wan2.1 $\delta = 0.31$ ($n = 20$), with LPIPS down 18% where measured. We attribute this to the forecast tracking the trajectory between evaluations, whereas copy-reuse holds the last residual fixed.

4.8.5 Ablations

Degree and penalty. On FLUX.1-dev at $\delta = 0.6$ (first 20 prompts, one shared base run), varying M and λ around the defaults yields a flat optimum (Table 4.5): $M \in \{4, 6\}$ and $\lambda \in \{0.01, 0.1\}$ agree to within 0.15 dB.

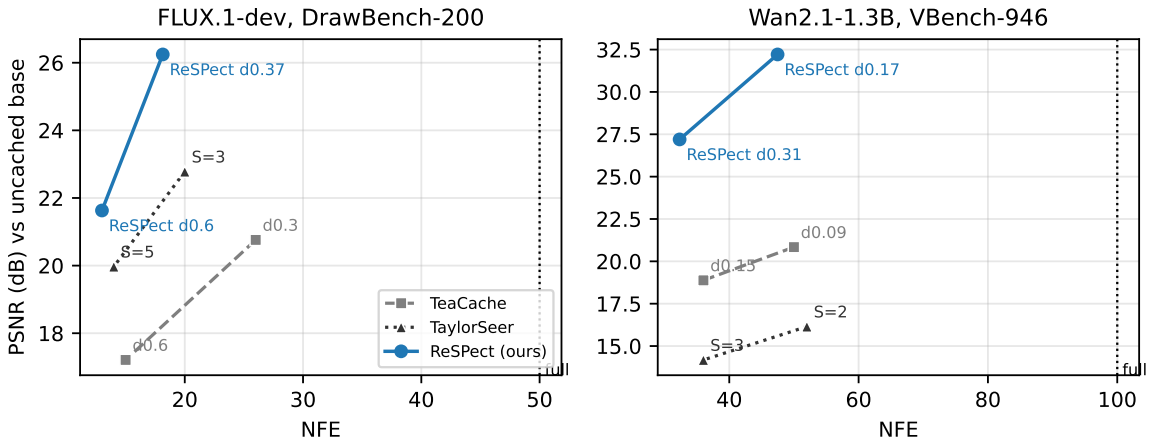


Figure 4.4: PSNR versus compute (NFE) on FLUX.1-dev, DrawBench-200 (left) and Wan2.1-1.3B, VBench-946 (right). ReSPect is compared with published TeaCache and TaylorSeer operating points; the dotted line marks uncached compute. The Wan2.1 gain at the aggressive budget is +8.32 dB.

Table 4.5: Sweeps of the Chebyshev degree M ($\lambda = 0.1$) and the ridge penalty λ ($M = 4$) on FLUX.1-dev, $\delta = 0.6$, $n = 20$. The default is the first row of each block.

Setting	PSNR↑	SSIM↑	Setting	PSNR↑	SSIM↑
Default ($M = 4$)	23.57	0.852	Default ($\lambda = 0.1$)	23.57	0.852
$M = 2$	23.02	0.845	$\lambda = 0.01$	23.56	0.853
$M = 6$	23.72	0.855	$\lambda = 1.0$	23.38	0.847

Reconstructor ablations against copy-reuse. All arms of Fig. 4.6 share one base run, so every ΔdB is paired against the same copy-reuse baseline, at the aggressive budgets (FLUX $\delta = 0.6$, Wan $\delta = 0.31$) on fixed 20-prompt subsets.

- *Blend weight w.* Pure Taylor ($w = 0$) is strong wherever neighbouring evaluations carry the signal: +2.30 dB on Wan2.1 and +0.50 dB on FLUX. Pure Chebyshev ($w = 1$) fails precisely where the fit is least determined: −1.68 dB on Wan2.1, with its long skip runs, and +0.09 dB (about copy-reuse) on FLUX. The fixed $w = 0.5$ remains at or above copy-reuse on either model (+0.49/+0.93 dB): a robust compromise rather than a per-subset optimum.

- *Warm-up threshold.* Blending before the fit is determined lets the penalty dominate the data. A warm-up of 8 attains the largest FLUX gain (+0.68 dB) but lies within noise of the default; longer Taylor-only spans (12–24) never accumulate enough observations to blend at this budget and coincide with pure Taylor (+0.50 dB).

- *Window K.* $K = 32$ matches $K = 100$ to three decimals (+0.491 vs. +0.491 dB): per-prompt buffers rarely exceed about 40 observations, so the $\sigma_{\min}$ gains of Theorem 4.2 saturate early and $K = 100$ is a conservative default.

- *Gate reference.* Refreshing the gate reference at every step rather than only at evaluated steps contributes +2.50 dB on HunyuanVideo at matched compute.

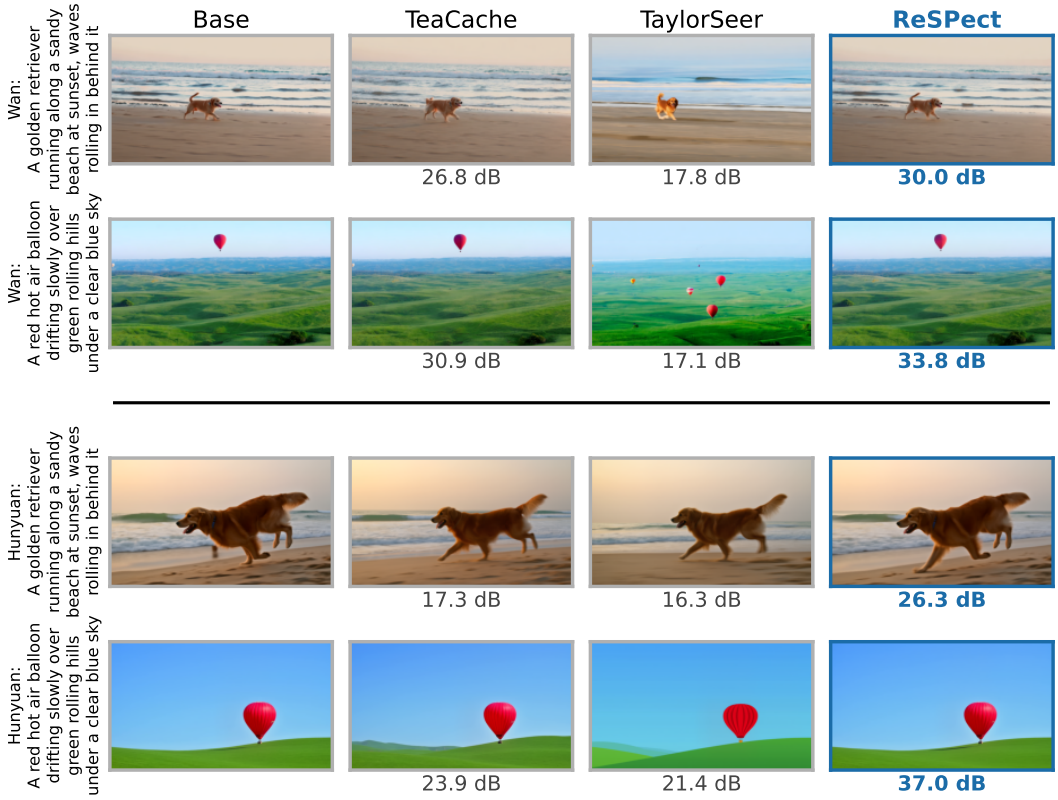


Figure 4.5: Opening frames on Wan2.1 and HunyuanVideo for two prompts (a golden retriever on a beach, a hot-air balloon over hills). ReSPect uses $\delta = 0.31$ (Wan2.1) and 0.48 (HunyuanVideo) with less compute than TeaCache and TaylorSeer; the PSNR of each panel is printed below it.

Where compute is spent. Refresh histograms concentrate compute early along the trajectory, in accordance with the spectral prior: evaluations are dense while G_t is narrow (content forming) and sparse as the passband widens. Skip rates concentrate tightly per budget (Fig. 4.7), and PSNR-versus-refresh curves follow the full-compute reference monotonically at both δ settings, with the gap to copy-reuse widening at lower refresh ratios, exactly where the reuse bound Eq. (4.4) loosens while the uniform bound Eq. (4.8) remains tight. Figure 4.8 shows per-prompt SSIM and LPIPS.

4.8.6 Qualitative clips

Table 4.6 gives frame-averaged metrics for two clips per video model, generated in our harness at $\delta = 0.31$ (Wan2.1, 32 forward passes) and $\delta = 0.48$ (HunyuanVideo, 16); TeaCache and TaylorSeer run at their published operating points and use more compute. ReSPect uses the least compute and leads on every clip and metric. On the HunyuanVideo balloon clip, TaylorSeer renders a different scene entirely and retains it for the whole clip; on Wan2.1 it hallucinates three additional balloons (Figs. 4.9 and 4.10). The retriever prompt is a difficult motion case in which all caches degrade while the ordering is preserved.

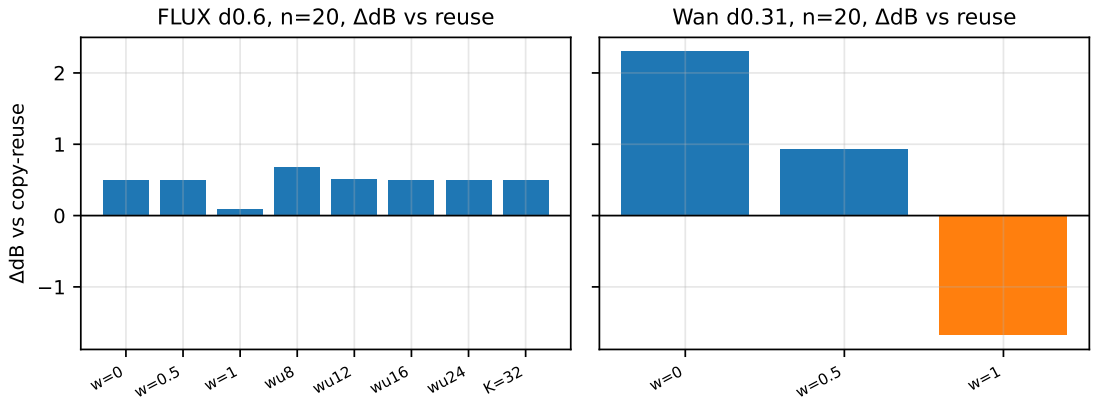


Figure 4.6: Reconstructor ablations (ΔdB against copy-reuse, paired on one shared base run per panel). Left: FLUX.1-dev, $\delta = 0.6$, $n = 20$ (blend weights, warm-ups "wu" of 8–24, window $K = 32$). Right: Wan2.1, $\delta = 0.31$, $n = 20$. Pure Taylor dominates on Wan2.1 while the blend remains at or above copy-reuse; pure Chebyshev fails on long skip runs.

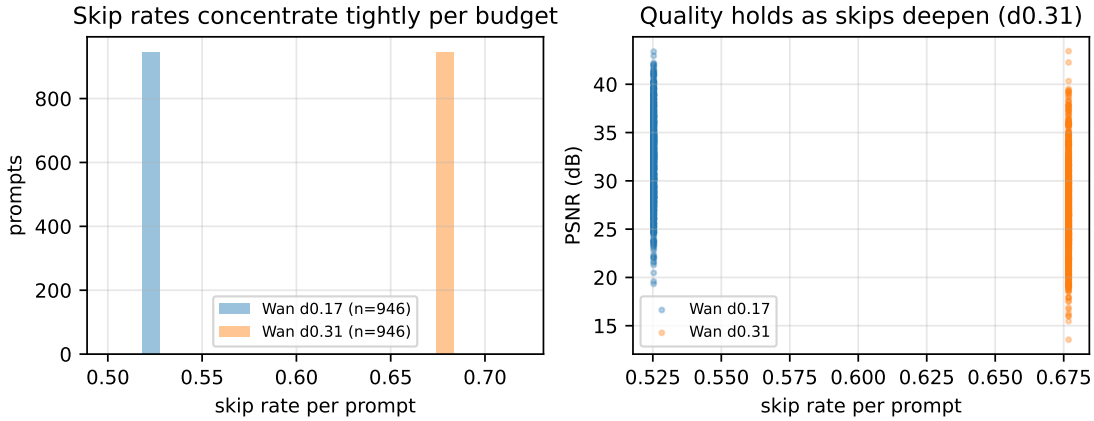


Figure 4.7: Compute concentrates early and quality tracks full compute (Wan2.1, VBench-946). Left: per-prompt skip-rate histograms at $\delta = 0.17$ and 0.31. Right: per-prompt PSNR versus skip rate.

4.9 Discussion and limitations

ReSPect couples a linear-MMSE-motivated gate with residual-space forecasting. At matched or lower compute it improves over the strongest compared baseline by up to 11.4 dB on Wan2.1, 9.9 dB on HunyuanVideo and 3.46 dB on FLUX.1-dev, with the largest gains under long skips, particularly for video. Its limitations are the following.

- Gains are largest when many steps are skipped; under rapid content drift the gate evaluates densely and little residual error remains for any predictor to reduce.
- The blend weight is fixed at $w = 0.5$ rather than scheduled.
- Content-aware gates need not realise equal skip rates at a shared δ, so comparisons must be made at matched compute.
- Most importantly for the next chapter, ReSPect is still an *open-loop* predictor: when the next computed step reveals the forecast error of a skipped run, that error has already

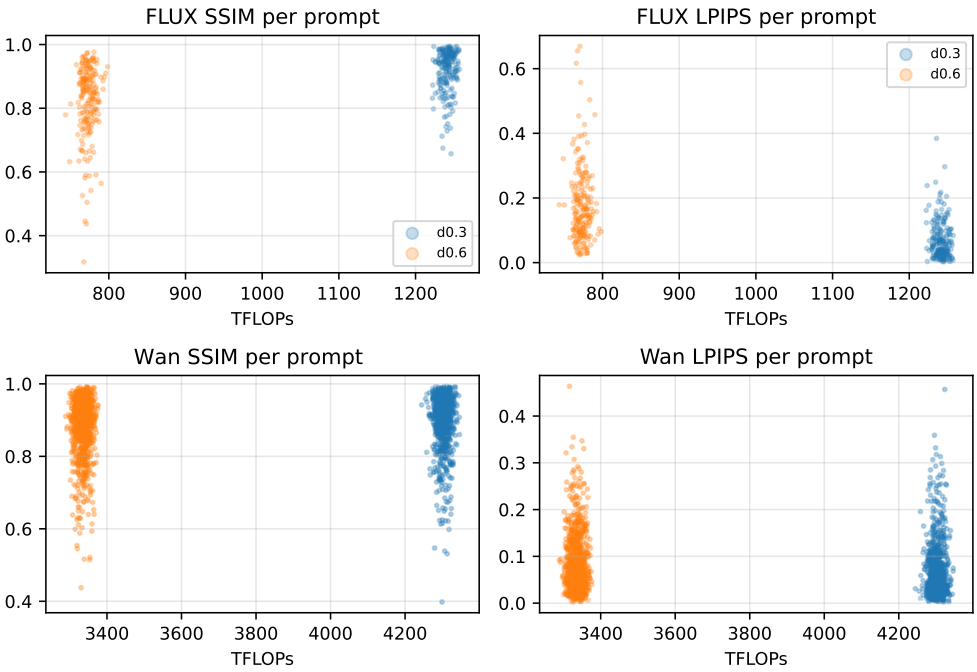


Figure 4.8: Per-prompt SSIM and LPIPS at each configuration's TFLOPs. Top: FLUX.1-dev, DrawBench-200. Bottom: Wan2.1, VBench-946.

been integrated into the latent, and ReSPect uses the new observation only to predict the *next* run. The forecast improves the upcoming steps but never revisits the past ones.

The last point led to KalSea, which replaces the regression by a state-space model whose smoother can correct the past gap (Chapter 5).



Figure 4.9: HunyuanVideo, balloon clip, $\delta = 0.48$ (16 NFE), frames 0, 16, 32, 48 and 64. Averaged over all 65 frames, ReSPect scores 37.41 dB against 25.80 dB (TeaCache) and 21.52 dB (TaylorSeer).

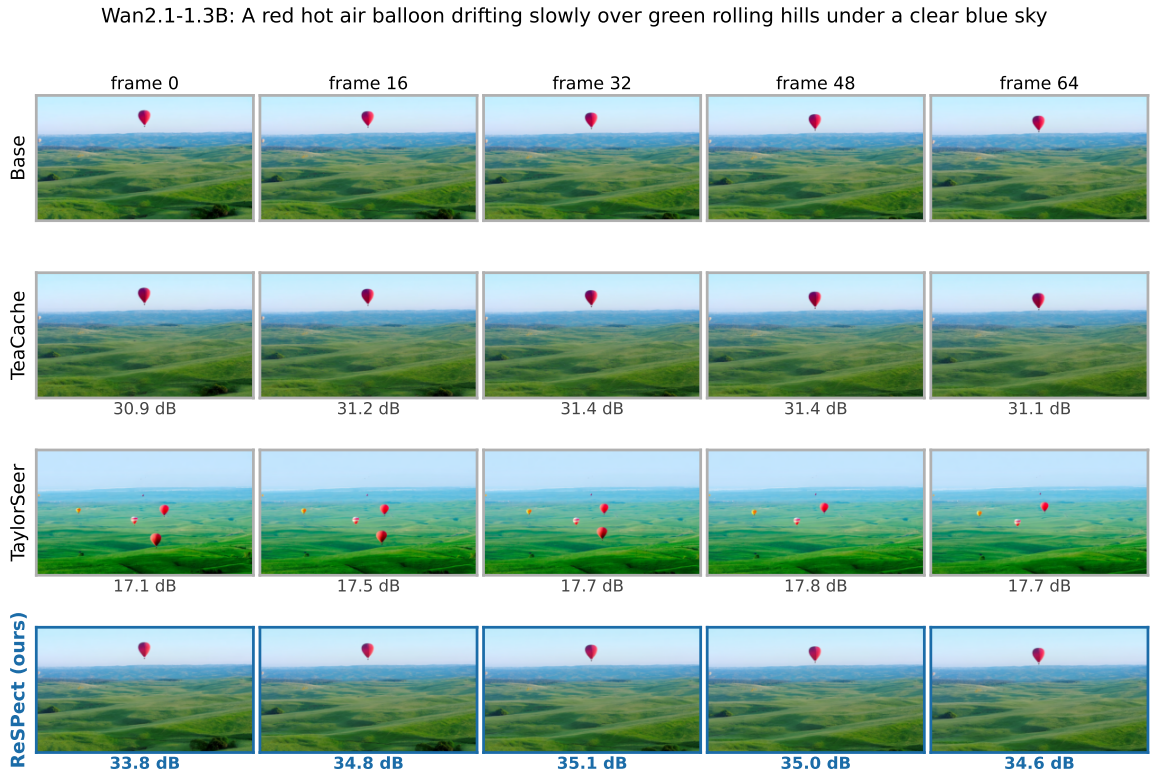


Figure 4.10: Wan2.1-1.3B, balloon clip, $\delta = 0.31$ (32 NFE). TaylorSeer hallucinates three additional balloons and retains them for the whole clip.

Table 4.6: Per-clip comparison: frame-averaged PSNR, SSIM and LPIPS against the uncached same-seed reference over all 65 frames. NFE is forward passes per clip (Wan2.1 base 100, HunyuanVideo base 50).

Clip	Method	PSNR↑	SSIM↑	LPIPS↓	NFE
Wan retriever	TeaCache	27.07	0.892	0.109	38
	TaylorSeer	17.91	0.656	0.384	36
	ReSPect	**30.96**	**0.937**	**0.066**	32
Wan balloon	TeaCache	31.29	0.918	0.064	38
	TaylorSeer	17.60	0.568	0.321	36
	ReSPect	**34.87**	**0.950**	**0.031**	32
Hunyuan retriever	TeaCache	16.32	0.669	0.322	18
	TaylorSeer	15.24	0.643	0.379	18
	ReSPect	**26.04**	**0.878**	**0.089**	16
Hunyuan balloon	TeaCache	25.80	0.914	0.046	18
	TaylorSeer	21.52	0.903	0.117	18
	ReSPect	**37.41**	**0.980**	**0.005**	16

Chapter 5

KalSea: Kalman-Corrected Spectral-Evolution-Aware Caching

ReSPect forecasts the skipped residuals better than reuse, but like every filler in the literature it is *open loop*: once the next computed step reveals how wrong the forecast was, that error has already been integrated into the latent and is never corrected. This chapter presents KalSea (**Kal**man + **SEA**), the second method of the project, which closes this loop. Behind the unchanged SEA gate of SeaCache, a damped-trend Kalman filter forecasts the block-stack residual on skipped steps (the *predictor*). When the next step is computed, a Rauch–Tung–Striebel smoother turns the filter's innovation into corrected residuals for the whole skipped gap, the gap is re-integrated through the output layers only, and sampling continues from the corrected latent (the *corrector*). We call this procedure predictor–corrector gap smoothing (PCGS). PCGS requires no additional transformer evaluations and no hyper-parameters of its own.

On FLUX.1-dev and held-out DrawBench prompts, KalSea reaches a PSNR of 27.72 dB with respect to the 50-step sampler at 18.1 of 50 transformer evaluations, against 26.29 dB reported for SeaCache at 18.4, and 22.53 against 21.33 dB at 13 evaluations. In paired comparisons PCGS alone contributes 0.81 dB (95% CI [0.63, 1.00]) and 0.61 dB [0.48, 0.76], and it also improves plain reuse and linear extrapolation. On HunyuanVideo and Wan2.1 at SeaCache's budgets, KalSea improves PSNR over SeaCache run in the same setup by 1.9–3.2 dB and 0.3–2.3 dB.

5.1 From ReSPect to KalSea: the open-loop problem

Reuse and forecasting share a structural limitation. The filler predicts the skipped steps from the past alone, and the sampler integrates its errors into the latent (Section 3.3). These errors grow with the distance to the last computed step: on six reference trajectories of our development set, the relative error of the velocity obtained from a Kalman forecast of the residual grows from 4–5% at the first skipped step to over 20% at the seventh. When the next step is computed, its block-stack output reveals how wrong the forecast was, but existing caches, including ReSPect, use this observation only to predict the *next* gap; the latent, already advanced along erroneous velocities, is never revisited. Forecasting reduces the drift but does not remove it, and as gaps grow its advantage over reuse vanishes (Section 5.6.2).

Our key observation is that *a skipped gap is a smoothing problem*. With a Kalman filter as the predictor of the block-stack residual, the innovation at the next computed step b (observed



Figure 5.1: Images generated by FLUX.1-dev (1024 × 1024, 50 steps) with KalSea at $\delta = 0.2$, i.e. 27 of 50 transformer evaluations (1.85× fewer). The tiles are only downscaled; the billboard is the central 2:1 band of its image.

minus forecast residual) and the covariances accumulated over the gap determine, through the Rauch–Tung–Striebel (RTS) smoother [63], the minimum-mean-square-error residuals of all skipped steps under the filter's model. Because only the cheap output layers depend on the residual, the gap can be re-integrated through them and sampling can continue from the corrected latent. Like the corrector of UniPC [29], PCGS reuses a network evaluation that the sampler performs anyway; unlike it, it corrects an entire gap of steps whose network outputs were never computed. The regression forecaster of ReSPect does not provide the covariances needed for such a correction, which is why KalSea moves to a state-space model.

Figure 5.2 gives an overview of the method. The gate is the SEA gate of Section 3.4, used unchanged with $\beta = 2$ for images and $\beta = 3$ for videos; KalSea changes only what is written on the steps it skips and then corrects it.

5.2 The Kalman residual forecaster (predictor)

5.2.1 State-space model

The residual evolves smoothly but nonlinearly along the trajectory, and the cache observes it only at the irregular steps chosen by the gate. Following the state-space model of the contemporaneous SpecKCast [61], which forecasts the model output, we model every element of the residual as a damped trend observed in noise. The state $s_i = [m_i, \nu_i]^\top$ holds a value and its rate of change with respect to a time coordinate s that increases monotonically along the schedule, with steps $\Delta s_i = s(i) - s(i-1)$. At *every* step, computed or skipped, the filter

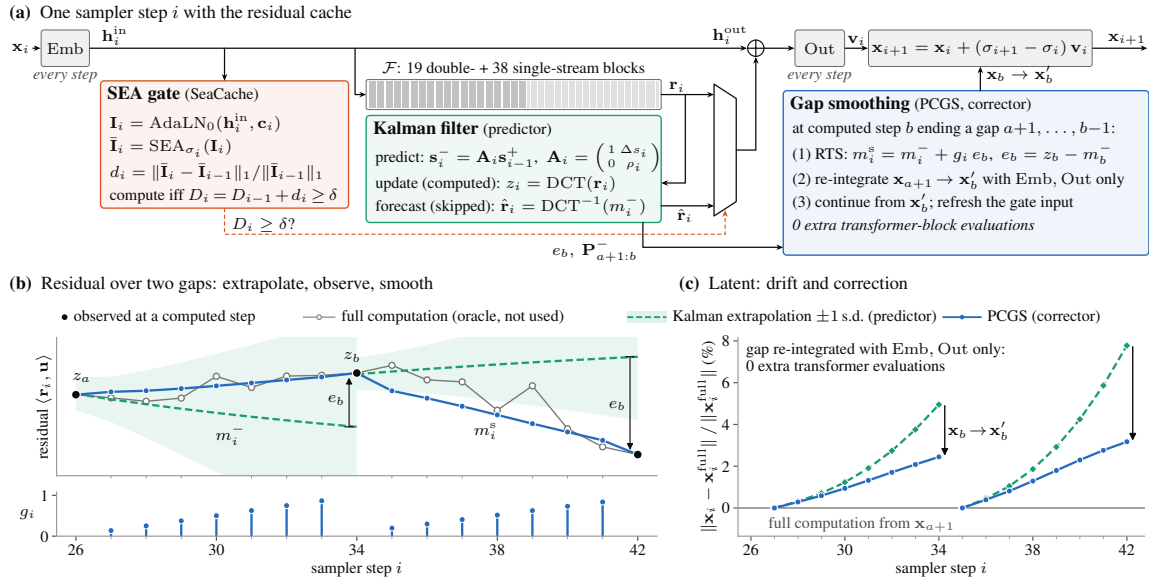


Figure 5.2: Overview of KalSea. (a) In each step the SEA gate accumulates the filtered relative change of the first block's modulated input and runs the blocks F only when the sum reaches δ; the residual of a computed step updates a damped-trend Kalman filter (the predictor), whose forecast replaces it on skipped steps. (b) When a computed step b ends the gap $a{+}1, \ldots, b{-}1$, PCGS spreads the innovation e_b over the gap with RTS gains g_i that grow towards b. (c) Re-integrating the gap from x_{a+1} with Emb and Out only lowers the latent deviation from full computation at b (by 25–59% in all ten gaps of this FLUX.1-dev sample at $\delta = 0.6$), without extra transformer evaluations. In (b, c) the residual is projected onto a fixed random direction in the lowest DCT band; grey: full computation, evaluated only for the figure.

predicts

$$s_i^- = A_i\, s_{i-1}^+, \qquad P_i^- = A_i P_{i-1}^+ A_i^\top + Q_i, \tag{5.1}$$

$$A_i = \begin{bmatrix} 1 & \Delta s_i \\ 0 & \rho_i \end{bmatrix}, \qquad Q_i = q\kappa \begin{bmatrix} \Delta s_i^3/3 & \Delta s_i^2/2 \\ \Delta s_i^2/2 & \Delta s_i \end{bmatrix}, \qquad \rho_i = e^{-\Delta s_i/\tau}. \tag{5.2}$$

Q_i is the process noise of an integrated Wiener process [65] with intensity $q\kappa$, and ρ_i damps the trend so that forecasts over long gaps do not overshoot [66]. Because the filter is predicted at every step and the gate's irregular schedule enters only through which steps are corrected, any pattern of computed and skipped steps is handled consistently.

On a computed step b the filter observes the residual element z_b with noise variance R and updates, with $h = [1, 0]^\top$,

$$e_b = z_b - m_b^-, \quad S_b = [P_b^-]_{00} + R, \quad k_b = P_b^- h / S_b, \quad s_b^+ = s_b^- + k_b e_b, \quad P_b^+ = (I - k_b h^\top) P_b^-. \tag{5.3}$$

On skipped steps the moments are not updated ($s_i^+ = s_i^-$, $P_i^+ = P_i^-$) and the cache writes the predicted value, $\hat{r}_i = m_i^-$, into $h_i^{\mathrm{out}} = h_i^{\mathrm{in}} + \hat{r}_i$; the output layers run as usual. The intensity κ is adapted online so that the mean squared innovation matches its predicted variance S_b [67]. In the implementation, the multiplicative factor is updated by $g^{0.2}$ with g the ratio of

measured to predicted innovation power, clipped to $[0.1, 10]$.

5.2.2 Parametrisations

The covariance P is shared by all elements (*per-element* variant) or by all coefficients of a frequency band (*DCT* variant), so the filter costs a few element-wise operations per step and its gains are scalars per band. All filter statistics are kept in relative units, i.e. the residual is divided by the square root of its (band) power, estimated from the first observation in the calibration-free mode.

- **DCT variant** (default for FLUX.1-dev, following SpecKCast). The residual of each channel is laid out on the 64×64 token grid and transformed with an orthonormal 2D DCT; the coefficients are grouped into seven radial frequency bands, each with its own covariance and its own warped time coordinate (a band-specific sigmoid warp of the log signal-to-noise ratio of the schedule). Noise intensities, initial trend variances and band powers are calibrated once per model from 16 uncached trajectories of prompts disjoint from all evaluation prompts (the file `kalsea_calib_flux.json`).
- **Per-element variant.** One filter per element with a single shared covariance; the time coordinate is a warped log-SNR or simply the step index. With the step index and online power estimation it is calibration-free and needs no token grid, which is the setting used for all video models.

Throughout, $\tau = 0.5$ and $R = 10^{-3}$. The two variants perform the same once PCGS is added (Section 5.6.3). With the corrector of Section 5.3, this predictor forms our full method, KalSea; the predictor alone is labelled *KalSea w/o PCGS*.

5.3 Predictor–corrector gap smoothing (corrector)

Let $a < b$ be consecutive computed steps, so that $a{+}1, \ldots, b{-}1$ is a gap of skipped steps. The predictor has written $\hat{r}_i = m_i^-$ on the gap, and the sampler has advanced the latent from x_{a+1} to x_b with the corresponding velocities. At step b the block stack runs and the filter observes z_b. The innovation e_b is the forecast error at the end of the gap; the forecast errors inside the gap are correlated with it, since they stem from the same trend error and process noise. PCGS exploits this correlation in four steps.

5.3.1 Step 1: smoothing

Proposition 5.1 (Smoother gain)**.** *Under the filter's model and given the observations up to step a, let $\Phi(b, i) = A_b A_{b-1} \cdots A_{i+1}$ for $a < i < b$. The minimum-mean-square-error*

estimate of the value m_i after observing z_b is

$$m_i^s = m_i^- + g_i\, e_b, \qquad g_i = \frac{\left[P_i^-\,\Phi(b,i)^\top\right]_{00}}{[P_b^-]_{00} + R}. \tag{5.4}$$

Proof. No observation falls inside the gap, so the state propagates as $s_b = \Phi(b,i)\, s_i + w_{i\to b}$, where the accumulated process noise $w_{i\to b}$ has zero mean and is independent of s_i. Conditioned on the observations up to step a, s_i has mean s_i^- and covariance P_i^- (inside the gap the filtered and predicted moments coincide), and $z_b = h^\top s_b + v_b$ with $v_b \sim \mathcal{N}(0, R)$ independent of everything else. Hence (s_i, z_b) is jointly Gaussian with

$$\begin{gathered}\mathbb{E}[z_b] = h^\top \Phi(b,i) s_i^- = m_b^-, \qquad \operatorname{Cov}(s_i, z_b) = P_i^-\,\Phi(b,i)^\top h, \\ \operatorname{Var}(z_b) = h^\top P_b^- h + R = [P_b^-]_{00} + R.\end{gathered} \tag{5.5}$$

Conditioning a joint Gaussian gives $\mathbb{E}[s_i \mid z_b] = s_i^- + \operatorname{Cov}(s_i, z_b)\operatorname{Var}(z_b)^{-1}(z_b - m_b^-)$. Its first component is Eq. (5.4), since $[P_i^-\Phi(b,i)^\top h]_0 = [P_i^-\Phi(b,i)^\top]_{00}$ and $z_b - m_b^- = e_b$. □

This is exactly the RTS smoother [63, 64] over $[a, b]$: its backward gains $J_i = P_i^+ A_{i+1}^\top (P_{i+1}^-)^{-1}$ telescope because $P_i^+ = P_i^-$ inside the gap, and their product reduces to the closed form above. The gains use only quantities that the filter already maintains, so the smoother has no parameters of its own. They increase across the gap: the steps closest to b, whose forecasts were the least reliable, absorb most of the innovation (Fig. 5.2b). In residual space the smoothed residual is the forecast plus a gain-weighted share of the forecast error revealed at b:

$$\tilde{r}_i = \hat{r}_i + G_i\big(r_b - \hat{r}_b\big), \qquad a < i < b, \tag{5.6}$$

where $\hat{r}_b = m_b^-$ is the forecast for step b and G_i multiplies by the scalar g_i (per-element variant) or applies the band gains $g_{i,\ell}$ in the DCT domain.

5.3.2 Step 2: re-integration

Only the output layers depend on the residual. We therefore replay the sampler over the gap with the smoothed residuals, starting from the latent that the sampler actually used at the first skipped step, $x'_{a+1} = x_{a+1}$ (exact, because step a was computed):

$$x'_{i+1} = x'_i + (\sigma_{i+1} - \sigma_i)\,\mathrm{Out}\big(\mathrm{Emb}(x'_i) + \tilde{r}_i,\ c_i\big), \qquad a < i < b. \tag{5.7}$$

Re-embedding the corrected latents x'_i, instead of reusing the stored inputs h_i^{in}, makes the replay an exact Euler integration with the smoothed residuals. Under the filter's model and to first order, re-integrating with the smoothed residuals minimises the expected squared error of x'_b given the observations up to step b.

5.3.3 Step 3: continuation

The block stack at step b was evaluated on the uncorrected latent x_b. Instead of evaluating it again, we keep its residual r_b, attach it to the corrected input, and absorb the latent correction into the velocity returned to the sampler:

$$v'_b = \mathrm{Out}\big(\mathrm{Emb}(x'_b) + r_b,\ c_b\big) + \frac{x'_b - x_b}{\sigma_{b+1} - \sigma_b}. \tag{5.8}$$

The unmodified Euler step Eq. (3.2) then produces

$$x_{b+1} = x_b + (\sigma_{b+1} - \sigma_b)\, v'_b = x'_b + (\sigma_{b+1} - \sigma_b)\, \mathrm{Out}\big(\mathrm{Emb}(x'_b) + r_b, c_b\big), \tag{5.9}$$

i.e. sampling continues from x'_b. PCGS is therefore a drop-in replacement of the network output: neither the pipeline nor the scheduler is modified.

5.3.4 Step 4: gate refresh

The gate's reference input of step b is recomputed from x'_b (block-0 normalisation with the step-b modulation followed by the SEA filter), so that the gate measures the change along the trajectory actually followed. This has no measurable effect on quality (Section 5.6.3) but keeps the gate consistent.

5.3.5 Remarks and extensions

PCGS does not change the predictor for the next gap, because smoothed and filtered estimates coincide at the latest observation. It only requires the solver update to be linear in the network output:

- For a **multistep solver** such as the UniPC sampler of Wan2.1, the solver state saved after step a is restored and the updates of steps $a{+}1, \ldots, b{-}1$ are replayed exactly with the corrected outputs.
- With **classifier-free guidance** [20], the conditional and unconditional residuals are filtered and smoothed separately under a shared gate, and the replay uses the corrected guided outputs $u' + g(c' - u')$.
- For the **video-and-audio** model MiniMax-H3, the video and the audio stream are corrected separately in steps 1–3, each with its own forecaster and its own scheduler.

Algorithm 2 summarises the procedure.

Algorithm 2 KalSea sampling. The second **if** block is PCGS; without it, the procedure is KalSea w/o PCGS.

Require: noise x_0, schedule $\sigma_0 > \cdots > \sigma_N = 0$, threshold δ

$D \leftarrow 0$; gap $\mathcal{S} \leftarrow \emptyset$
for $i = 0, \ldots, N-1$ **do**
  $h_i^{\text{in}} \leftarrow \text{Emb}(x_i)$; $D \leftarrow D + d_i$; Kalman predict ▷ Eqs. (3.14) and (5.1)
  if $i \in \{0, N-1\}$ **or** $D \geq \delta$ **then** ▷ computed step, $b = i$
    $r_i \leftarrow F(h_i^{\text{in}}) - h_i^{\text{in}}$; $D \leftarrow 0$; Kalman update ▷ Eq. (5.3)
    $v_i \leftarrow \text{Out}(h_i^{\text{in}} + r_i, c_i)$
  else ▷ skipped step
    $\hat{r}_i \leftarrow m_i^-$; $v_i \leftarrow \text{Out}(h_i^{\text{in}} + \hat{r}_i, c_i)$; $\mathcal{S} \leftarrow \mathcal{S} \cup \{i\}$
  if step i was computed **and** $\mathcal{S} \neq \emptyset$ **then**
    $\tilde{r}_j \leftarrow \hat{r}_j + G_j(r_i - \hat{r}_i)$ for $j \in \mathcal{S}$ ▷ Eq. (5.6)
    $x' \leftarrow x_{\min \mathcal{S}}$
    for $j \in \mathcal{S}$ in ascending order **do**
      $x' \leftarrow x' + (\sigma_{j+1} - \sigma_j)\,\text{Out}(\text{Emb}(x') + \tilde{r}_j, c_j)$ ▷ Eq. (5.7)
    $v_i \leftarrow \text{Out}(\text{Emb}(x') + r_i, c_i) + (x' - x_i)/(\sigma_{i+1} - \sigma_i)$ ▷ Eq. (5.8)
    recompute the gate reference from x'; $\mathcal{S} \leftarrow \emptyset$
  $x_{i+1} \leftarrow x_i + (\sigma_{i+1} - \sigma_i)\, v_i$ ▷ unchanged sampler
return x_N

5.4 Cost

KalSea performs exactly the block-stack evaluations of the gate's schedule; PCGS adds none. It adds one evaluation of Emb and Out per skipped step and, at each computed step that closes a gap, one more of each plus the gate refresh: for FLUX.1-dev at 1024^2 this is about 0.3% of one transformer evaluation per image. The filter state and the gap buffer raise the peak memory by 0.16 GiB over the uncached sampler (SeaCache: 0.05 GiB; KalSea w/o PCGS: 0.14 GiB). The filter adds element-wise work, so at equal NFE KalSea is 3–5% slower than SeaCache on FLUX.1-dev and at most 1.4% slower on video.

5.5 Implementation

The code release (`KalSea-v0.2`) contains inference code for five backbones, again one self-contained folder per backbone with copies of the SEA filter (`util_seacache.py`) and the Kalman forecaster (`kalsea_forecaster.py`); PCGS lives in `pcgs.py` (FLUX.1, FLUX.2, MiniMax-H3) or in the backbone module (`kalsea_hunyuan.py`, `kalsea_wan.py`). All five scripts share the same switches: KalSea + PCGS is the default, `-no_pcgs` runs the forecaster alone, `-reuse` runs SeaCache (last residual reused), `-no_cache` computes every step, and `-delta` sets the threshold. The method names `kalsea_pcgs`, `kalsea`, `seacache` and `reference` are written to a JSONL statistics file together with the latency, the number of full evaluations and the computed steps.

Table 5.1: Backbones supported by KalSea-v0.2 and how the cache is attached. Thresholds give the measured number of full evaluations per generation (50-step samplers; MiniMax-H3: 50 sigma points = 49 evaluations).

Backbone	Forecaster	Integration	$\delta \rightarrow$ evaluations
FLUX.1-dev	7 DCT bands, warped time, calibrated	import replaces the diffusers transformer forward	0.2→27, 0.3→21, 0.37→18, 0.5→15, 0.6→13, 0.8→11, 1.0→9
HunyuanVideo	per element, calibration-free	`install()` on the loaded sampler; live sampler σ	0.2575→24, 0.48→16
Wan2.1-1.3B	per element and CFG stream	cached `generate`/forward bound per instance; UniPC replay	0.2→24, 0.35→17
FLUX.2 [dev]	per element, calibration-free	patched class forward with a state object	0.47→18, 0.8→13
MiniMax-H3	per element and stream (video, audio)	patched class forward; two schedulers	0.3→23, 0.6→14

Table 5.1 lists the backbones. Notable implementation details are:

- **Patching.** FLUX.2 and MiniMax-H3 install the patch on the *class* forward, so that accelerate and diffusers' component-manager offload hooks, which wrap the instance forward, keep working; the undecorated original forward is kept so that a LoRA scale is applied exactly once. HunyuanVideo and Wan2.1 modify nothing in the official packages: the forward is replaced at run time.
- **Live noise level.** HunyuanVideo's sampler creates a fresh scheduler for sampling. KalSea's gate reads the σ of the scheduler the sampler actually uses; `-stale_sigma` reproduces the behaviour of SeaCache's released script (Section 2.4).
- **Exact replay.** On FLUX.2 the re-integration uses the scheduler's own sigma tensors and the arithmetic of `FlowMatchEulerDiscreteScheduler.step` (a float32 latent plus dt times the bf16 velocity, cast back to bf16), so with a zero innovation the replay reproduces the pipeline's latents bitwise. On Wan2.1 the UniPC solver state is saved after step a and replayed exactly; PCGS therefore requires the UniPC solver.
- **Determinism.** The DCT mode of FLUX.1 is not bit-reproducible run to run, because its band reductions use `scatter_add_` (CUDA atomic additions). The per-element mode and `-reuse` are bit-reproducible, and on HunyuanVideo and Wan2.1 the forecaster's two scalar reductions run in deterministic mode.
- **Memory.** On FLUX.2 the KalSea state is two fp32 copies of the $[1, 4096, 6144]$ residual plus the previous gate input (0.23 GiB at 1024^2); PCGS additionally holds the bf16 forecast of every skipped step of the open gap, released at the next computed step. All state is freed before the VAE decode.

Verification. The implementation was checked against invariants that must hold exactly, using tiny random-weight models built from the real diffusers classes: with no state attached or with `-no_cache` the original forward runs unchanged; with $\delta = 0$ (nothing skipped) and PCGS on, every transformer output and the final latent are bitwise identical to the original forward; with the innovation forced to zero, every gap is still re-integrated but each replayed latent is bitwise equal to the latent the pipeline used, and the whole generation is bitwise identical to `-no_pcgs`; the smoother gains equal $\mathrm{Cov}(\mathrm{value}_i, \mathrm{value}_b)/(P_{b,00} + R)$ from an independent forward propagation of the filter's covariance to a relative error below 10^{-15}. For FLUX.2 [dev] (32B parameters) and MiniMax-H3 (text-to-video with audio), the code was additionally run on the real checkpoints as a functional check (16 and 3 prompts respectively); no quality or speed evaluation was performed for these two models, and their thresholds are not calibrated.

5.6 Experiments

5.6.1 Setup

We follow the protocol of SeaCache. For text-to-image generation we use FLUX.1-dev at 1024×1024 with 50 Euler steps of its default flow-matching schedule, embedded guidance 3.5 and the 200 DrawBench prompts, with the prompt index as seed. For text-to-video generation we use HunyuanVideo at 480×640 and Wan2.1-T2V-1.3B at 832×480, both with 65 frames, 50 steps and seed 42, on the 944 unique VBench prompts. Each accelerated sample is compared with the output of the uncached 50-step sampler for the same prompt and seed (Section 3.5). All runs use bf16 on NVIDIA B200 GPUs. All design decisions (the smoother, the re-integration and continuation rules, τ and R) were made on 64 COCO captions [85], and the forecaster was calibrated on 16 others, so DrawBench is a held-out test set.

Budgets and baselines. We report the mean number of block-stack evaluations per image (NFE, of 50) and compare only at matched NFE. The SEA gate computes 13.0 steps at $\delta = 0.6$ both in SeaCache's table and in our setup, where our re-run of SeaCache reproduces the published PSNR (21.26 vs. 21.33 dB). At $\delta = 0.3$ it computes 20.9 steps in our setup but 18.4 according to the published TFLOPs, so we run our cache at $\delta = 0.37$ (18.1 NFE) to match. For controlled comparisons, four fillers run behind the same gate with the same code, seeds and hardware: reuse (SeaCache), linear extrapolation of the residual from the last two computed steps (the first-order forecast of TaylorSeer), the Kalman forecast (KalSea w/o PCGS), and KalSea.

Table 5.2: Text-to-image results on FLUX.1-dev (DrawBench, 200 held-out prompts). †Published by SeaCache (other hardware); NFE = 50×TFLOPs/2976. ‡The two rows appear swapped in SeaCache's table. Lat.: s/image on one B200, measured by us (*SeaCache's method in our code at the NFE of KalSea; Δ-DiT and ToCa not run).

Method	NFE	TFLOPs	Lat.	PSNR↑	SSIM↑	LPIPS↓
Original (50 steps)	50	2976	5.25	–	–	–
≈18 NFE (baselines: 18–33 NFE)						
Δ-DiT†	33.3	1984	–	17.40	0.710	0.336
TeaCache ($\delta = 0.3$)†	26.0	1547	2.81	20.76	0.810	0.211
Vanilla, 25 steps†‡	25.0	1487	2.68	15.55	0.668	0.409
ToCa†	21.2	1263	–	18.40	0.700	0.324
TaylorSeer ($S = 3$)†	20.0	1191	2.64	22.78	0.828	0.163
SeaCache ($\delta = 0.3$)†	18.4	1098	2.05*	26.29	0.893	0.106
KalSea ($\delta = 0.37$)	18.1	1072	2.12	**27.72**	**0.916**	**0.070**
13 NFE (baselines: 13–15 NFE)						
TeaCache ($\delta = 0.6$)†	15.0	892	1.71	17.21	0.714	0.348
Vanilla, 15 steps†‡	15.0	892	1.66	17.84	0.740	0.305
TaylorSeer ($S = 5$)†	14.0	834	2.09	19.97	0.762	0.236
SeaCache ($\delta = 0.6$)†	13.0	773	1.53*	21.33	0.798	0.226
KalSea ($\delta = 0.6$)	13.0	775	1.60	**22.53**	**0.842**	**0.154**

5.6.2 Results

Text-to-image. Table 5.2 compares KalSea with published results. At 18.1 NFE KalSea reaches 27.72 dB, 1.43 dB above the 26.29 dB reported for SeaCache at 18.4 NFE, with better SSIM and LPIPS, and it exceeds all baselines that use more evaluations. At 13 NFE it reaches 22.53 vs. 21.33 dB. In our same-setup runs, KalSea improves over SeaCache by 2.39 dB [2.03, 2.75] (better on 88% of the prompts) at 18.1 NFE and by 1.27 dB [1.02, 1.52] (79%) at 13 NFE.

Budget sweep. Figure 5.3 varies δ from 0.2 to 1.0 in the same setup. KalSea attains the highest PSNR and the lowest LPIPS at every budget. Its PSNR gain over our re-run of SeaCache grows from 0.74 dB [0.46, 1.04] at 11 NFE to 3.89 dB [3.49, 4.28] at 27 NFE; at 9 NFE the PSNR gain (+0.21 dB [−0.005, +0.42]) is not significant and only LPIPS improves significantly (0.043 [0.033, 0.053]).

Text-to-video. We apply KalSea with the per-element filter to HunyuanVideo and Wan2.1-1.3B at SeaCache's two budgets (Table 5.3); on Wan2.1 (UniPC, classifier-free guidance) PCGS replays the solver. Because SeaCache's released HunyuanVideo script gates on a stale noise level near one, every gated method reads the sampler's noise level, with δ set for 24.0 and 16.0 NFE. Our re-run of SeaCache then reproduces its published result at 24 NFE (32.43 vs. 32.39 dB) but not at 16.4 (24.53 dB at 16.0 vs. 26.46 dB; its released gate gives 20.52 dB),

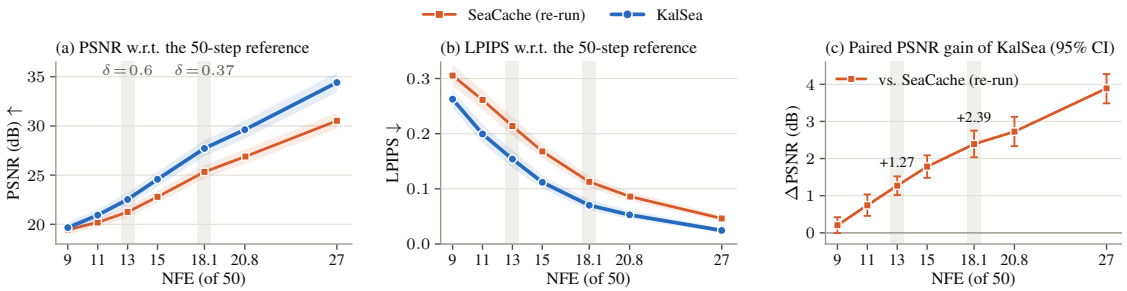


Figure 5.3: Fidelity versus compute on FLUX.1-dev (DrawBench, 200 held-out prompts). (a) PSNR and (b) LPIPS with respect to the 50-step reference versus the mean NFE; the SEA-gate threshold $\delta \in \{0.2, 0.3, 0.37, 0.5, 0.6, 0.8, 1.0\}$, shared by all methods, sets the budget. All methods share code, seeds and hardware. Lines: means over prompts; shaded: 95% bootstrap CIs. (c) Paired PSNR gain of KalSea over SeaCache with 95% CIs.

Table 5.3: Text-to-video results (944 VBench prompts, 65 frames, fidelity to the 50-step output). †Published by SeaCache (other GPUs); NFE = 50×TFLOPs/TFLOPs of the full run. §Our re-run of SeaCache's released code with the corrected gate (same code, seeds and GPUs as KalSea). Lat.: s/video on one B200, measured by us (*SeaCache's method in our code at the NFE of KalSea).

	HunyuanVideo, 480 × 640, 65 frames						Wan2.1 1.3B, 832 × 480, 65 frames					
Method	NFE	TFLOPs	Lat.	PSNR↑	SSIM↑	LPIPS↓	NFE	TFLOPs	Lat.	PSNR↑	SSIM↑	LPIPS↓
Reference (50 steps)	50	14043	74.5	–	–	–	50	6649	52.0	–	–	–
≈50% budget (24–26 NFE)												
Fewer steps†	25.0	7019	39.9	19.97	0.731	0.263	–	–	–	–	–	–
TeaCache†	24.9	6994	40.2	23.40	0.805	0.133	25.0	3324	27.6	20.84	0.721	0.171
TaylorSeer†	26.0	7299	43.3	24.14	0.820	0.152	25.5	3391	29.1	16.15	0.543	0.336
SeaCache	24.0§	6744	38.4	32.43	0.929	0.048	24.0†	3191	26.1*	26.60	0.873	0.075
KalSea	24.0	6745	38.6	**35.60**	**0.951**	**0.028**	24.0	3193	26.5	**31.67**	**0.953**	**0.020**
≈30% budget (15–18 NFE)												
Fewer steps†	15.0	4211	25.9	17.49	0.662	0.371	–	–	–	–	–	–
TeaCache†	17.1	4794	28.9	20.42	0.734	0.172	18.0	2393	20.6	18.88	0.645	0.245
TaylorSeer†	18.0	5053	32.6	20.42	0.733	0.242	18.0	2393	20.9	14.18	0.453	0.455
SeaCache	16.0§	4498	27.4	24.53	0.813	0.155	17.0†	2261	19.1*	21.78	0.740	0.170
KalSea	16.0	4499	27.5	**26.45**	**0.846**	**0.117**	17.0	2263	19.4	**23.85**	**0.851**	**0.090**

so the table reports the re-run for HunyuanVideo. There, KalSea improves on SeaCache by 3.17 dB $[2.99, 3.35]$ at 24 NFE and by 1.92 dB $[1.70, 2.14]$ at 16 NFE (better on 90% and 72% of the prompts), with better SSIM and LPIPS. On Wan2.1 it exceeds SeaCache's published results by 5.07 and 2.07 dB at 24 and 17 NFE; in our setup the paired gains are 2.27 dB $[2.13, 2.42]$ and 0.33 dB $[0.16, 0.49]$, the latter with clearly better SSIM (+0.023) and LPIPS (−0.029). Part of the distance to the published Wan2.1 numbers therefore comes from the setup (our clips have 65 frames).

Qualitative results. Figure 5.4 shows one example per model at the lower budget, chosen by a fixed rule: among the prompts on which SeaCache and KalSea computed identical steps, the one whose PSNR gain of KalSea over SeaCache is closest to the upper quartile. SeaCache alters details that KalSea keeps, such as the street lamp added at the top right of the FLUX.1-dev image.

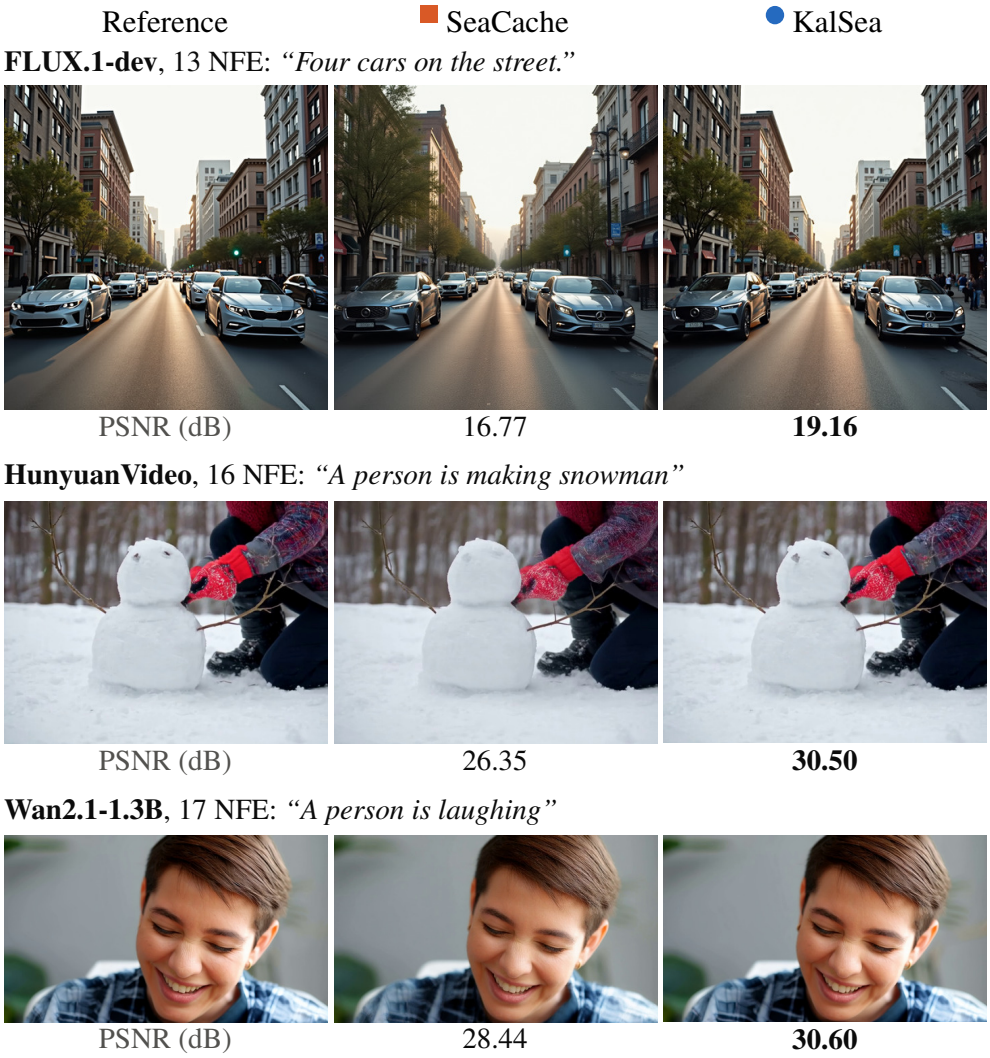


Figure 5.4: One example per model at its lower budget (reference, SeaCache, KalSea): FLUX.1-dev ($\delta = 0.6$, 13 NFE, "Four cars on the street.", PSNR 16.77 vs. 19.16 dB), HunyuanVideo ($\delta = 0.48$, 16 NFE, "A person is making snowman", 26.35 vs. 30.50 dB) and Wan2.1-1.3B ($\delta = 0.35$, 17 NFE, "A person is laughing", 28.44 vs. 30.60 dB); video: frame 32 of 65.

5.6.3 Ablations

Table 5.4 reports paired PSNR differences to KalSea at $\delta = 0.3$ (20.8 NFE) and $\delta = 0.6$ (13 NFE); differences below 0.1 dB are within run-to-run noise.

Anatomy of the corrector. Without PCGS, PSNR drops by 0.83 and 0.61 dB (Kalman row). Across budgets the predictor alone loses its advantage over reuse as gaps grow, while PCGS improves on it at every budget. The gain comes from re-integrating the latent: feeding the smoothed residuals only into the filter state is no better than no correction (−0.86 vs. −0.83 dB at 20.8 NFE). Simpler gain shapes, such as a linear interpolation from r_a to r_b, do as well as the RTS gains. The batched re-integration on the stored inputs h_i^{in} loses 0.22 dB

Table 5.4: Ablations (FLUX.1-dev, DrawBench-200, held out). First row: PSNR of KalSea; other rows: paired PSNR difference to KalSea (dB, 95% bootstrap CI). All rows share the SEA gate and threshold, with NFE within 0.06 (0.00) of KalSea at $\delta = 0.3$ (0.6).

Variant	20.8 NFE ($\delta = 0.3$)	13 NFE ($\delta = 0.6$)
KalSea (Kalman + PCGS)	29.62 dB	22.53 dB
(a) Filler on skipped steps, no correction		
Reuse (SeaCache)	−2.73 [−3.12, −2.34]	−1.27 [−1.52, −1.02]
Linear extrapolation	−1.11 [−1.34, −0.89]	−0.73 [−0.92, −0.54]
Kalman (KalSea w/o PCGS)	−0.83 [−1.02, −0.63]	−0.61 [−0.76, −0.48]
(b) Correction of the Kalman forecast		
Filter state only	−0.86 [−1.06, −0.68]	−0.63 [−0.78, −0.50]
Linear interpolation	+0.14 [+0.04, +0.25]	+0.00 [−0.05, +0.06]
Forecast + lin. innov.	+0.14 [+0.04, +0.25]	+0.02 [−0.03, +0.07]
Batched re-integration	−0.22 [−0.34, −0.09]	−0.06 [−0.13, +0.01]
No gate refresh	+0.03 [−0.04, +0.10]	+0.01 [−0.01, +0.03]
(c) Other forecaster, with PCGS		
Per-element filter	+0.01 [−0.15, +0.14]	−0.01 [−0.15, +0.13]
Damping $\tau = 0.1$	−0.08 [−0.29, +0.12]	+0.05 [−0.11, +0.22]
Calibration-free	−0.02 [−0.11, +0.08]	+0.02 [−0.13, +0.17]
(d) PCGS on other fillers		
Reuse + PCGS	−1.03 [−1.39, −0.67]	−0.53 [−0.76, −0.29]
Linear extrap. + PCGS	+0.04 [−0.08, +0.16]	+0.07 [−0.05, +0.18]

$[0.09, 0.34]$ at 20.8 NFE, and the gate refresh has no measurable effect.

Other forecasters and fillers. With PCGS, the forecaster's parametrisation hardly matters: the per-element filter, the damping $\tau = 0.1$ of SpecKCast and a calibration-free filter all stay within 0.1 dB of KalSea, so the frequency decomposition is unnecessary for the residual and the per-element filter is used for video. Nor is PCGS tied to a Kalman forecaster: it improves reuse by 1.69 and 0.74 dB and linear extrapolation by 1.15 and 0.80 dB; linear extrapolation with PCGS then ties KalSea, while reuse with PCGS stays 1.03 and 0.53 dB below.

5.6.4 Limitations

Matched NFE is not matched latency: PCGS adds output-layer evaluations and 0.02 GiB over KalSea w/o PCGS, and the filter adds element-wise work, so at equal NFE KalSea is 3–5% slower than SeaCache on FLUX.1-dev and at most 1.4% on video. The metrics measure fidelity to the 50-step sampler, which is the goal of a cache, not sample quality. The image results cover a single model. On Wan2.1 the same-setup PSNR gain is only 0.33 dB at 17 NFE and negative at 10 NFE on 20 prompts. Finally, PCGS attaches the residual of the uncorrected latent to the corrected one, which is accurate only for small corrections; with gaps of up to twelve steps its gain is modest (0.31 dB at 9 NFE).

Table 5.5: Conceptual comparison of the two methods developed in this project.

	ReSPect	KalSea
Gate (when)	SEA gate, reference refreshed every step	SEA gate (unchanged), live noise level, refreshed after correction
Filler (what)	Ridge Chebyshev (M=4) + first-order Taylor, w=0.5	Damped-trend Kalman filter (DCT bands or per element)
Target	block-stack residual	block-stack residual
Loop	open: past runs never corrected	closed: PCGS smooths and re-integrates every gap
Theory	skip-length-independent Chebyshev bound; analytic blend and warm-up	RTS smoother gain in closed form; first-order optimal re-integration
Extra cost	1.7–4.3% wall clock over reuse	one Emb + Out per skipped step; 3–5% (images), ≤1.4% (video) over SeaCache
State	K residuals + $(M{+}1) \times F$ coefficients	2 values per element + small covariances; gap buffer
Hyper-parameters	M, K, λ, w, warm-up (fixed by analysis)	τ=0.5, R=10^{-3}; PCGS adds none

5.7 Comparison of ReSPect and KalSea

Table 5.5 compares the two methods of this project conceptually. Both use the same SEA gate and both forecast the block-stack residual rather than the hidden state. They differ in the forecaster (a global regression versus a recursive state-space filter) and, more fundamentally, in the loop: ReSPect predicts each skipped run from the past and never revisits it, whereas KalSea's state-space model provides the covariances needed to smooth the run once the next step has been observed, and PCGS re-integrates it.

A numerical comparison is possible on FLUX.1-dev, where both studies use the same model, resolution, step count, guidance and DrawBench-200 prompts, at matched NFE:

- at 18.1 NFE, ReSPect ($\delta = 0.37$) reaches 26.24 dB (Table 4.4) and KalSea 27.72 dB (Table 5.2);
- at 13 NFE, ReSPect ($\delta = 0.6$) reaches 21.63 dB (Table 4.1) and KalSea 22.53 dB.

The two studies were run in different harnesses, and their SeaCache re-runs differ slightly (25.45 dB at 18.1 NFE in the ReSPect harness; about 25.33 dB implied by KalSea's paired gain of 2.39 dB), so the cross-study difference is indicative rather than a controlled comparison. Within each study the ordering is clear: ReSPect improves on SeaCache's reuse by +0.79 dB at 18.1 NFE, and KalSea by +2.39 dB, of which PCGS contributes 0.81 dB. The ablations of Table 5.4 also explain why the two methods are complementary rather than competing: PCGS improved every filler on which it was tested (reuse, linear extrapolation and the Kalman forecast), and simple gain shapes such as a linear interpolation of the innovation worked as well as the RTS gains, so the closed loop of KalSea could in principle also be

wrapped around ReSPect's forecaster; this combination has not been evaluated yet.

Chapter 6

Applications in Mechanical Engineering

ReSPect and KalSea were developed and evaluated on text-to-image and text-to-video models. Diffusion models, however, are increasingly used as engineering tools (Section 2.7), and in engineering the sampling cost is often more critical than in media generation: a designer explores many load cases, an optimisation loop calls the generator repeatedly, and physics-guided samplers use hundreds to thousands of steps. This chapter applies both caches to two recent open-weight diffusion models from mechanical engineering:

1. **Structural topology optimisation** with *Optimize Any Topology* (OAT, NeurIPS 2025) [7], a conditional latent diffusion model that generates minimum-compliance structures for arbitrary domains, loads, supports and volume fractions;
2. **Flow-field reconstruction around a cylinder** with *DiffusionPDE* (NeurIPS 2024) [11], which solves Navier–Stokes forward and inverse problems from 1% sparse observations by guiding a diffusion sampler with the observation loss and a PDE residual.

Both models were selected because their code and weights are public, because they represent the two classical pillars of mechanical engineering (solid and fluid mechanics), and because they stress the caches in opposite ways: OAT samples with only 20 DDIM steps on a small latent, whereas DiffusionPDE runs 2000 guided second-order steps whose guidance gradients are back-propagated through the network.

Protocol. The models are used exactly as released, with their own samplers, settings, test data and evaluation code. The rows of the original (uncached) models in the result tables are *copied from the original papers*; they were not re-run. Only the cached samplers (ReSPect and KalSea) were run, on eight NVIDIA B200 GPUs. Before any result was produced, the cached samplers were validated against exact invariants (Section 6.1.6).

6.1 Carrying the caches over to engineering denoisers

ReSPect and KalSea rest on three ingredients: a split of the denoiser into a cheap embedding, an expensive block stack and cheap output layers (Section 3.2); a gate input that is available before the block stack runs (Section 3.4); and, for PCGS, a sampler update that can be replayed with corrected network outputs (Section 5.3). Engineering diffusion models differ from the media DiTs in all three respects: their denoisers are convolutional U-Nets, they use

DDIM or EDM samplers with classifier-free or gradient guidance. This section describes how each ingredient generalises.

6.1.1 Residual caching on a U-Net

A U-Net has no single residual stream: the encoder passes skip connections to the decoder, and the channel width changes along the network. The block-stack residual is nevertheless well defined if Emb is chosen as the layers up to the first full-resolution feature map, and Out as the layers after the last full-resolution feature map of the decoder, provided both maps have the same width. The block stack F then contains *all* down-sampling, middle and up-sampling blocks together with their internal skip connections, and on a skipped step every one of them is bypassed:

$$h^{\text{in}} = \text{Emb}(x), \qquad h^{\text{out}} = F(h^{\text{in}}, c), \qquad r = h^{\text{out}} - h^{\text{in}}, \qquad \hat{v} = \text{Out}(h^{\text{in}} + \hat{r}). \tag{6.1}$$

For OAT's U-Net, Emb is the input convolution and Out the final group normalisation, SiLU and output convolution; both ends have 320 channels at the 64×64 latent resolution. For DiffusionPDE's DDPM++ U-Net [16], the input convolution has 128 channels but the decoder output has 256, so Emb also includes the first full-resolution residual block ($128 \to 256$ channels); this keeps the residual formulation, and with it the error identity Eq. (3.8), exact. On computed steps the output layers receive h^{out} itself rather than $h^{\text{in}} + r$, and residuals are kept in float32, so that a computed step reproduces the original network bit for bit.

6.1.2 Gate input

U-Net residual blocks inject the timestep (and, in OAT, the problem) embedding inside the block rather than through adaLN. The gate therefore reads the *timestep-modulated input of the first block of F*: the output of its first convolution with the projected embedding added,

$$I = \text{conv}_1\big(\text{SiLU}(\text{GN}(h^{\text{in}}))\big) + W_e\,\text{SiLU}(e), \tag{6.2}$$

for diffusers' ResNet blocks (OAT), and the normalised, embedding-shifted input for EDM's blocks (DiffusionPDE). This costs one convolution at full resolution, a small fraction of the block stack. The input is laid out on its spatial grid and filtered with the SEA filter of Section 3.4 with $\beta = 2$, since both applications produce two-dimensional fields.

6.1.3 Signal and noise coefficients

The SEA filter needs (a_t, b_t) for the current step. For OAT's variance-preserving DDIM schedule these are $(\sqrt{\bar{\alpha}_t}, \sqrt{1 - \bar{\alpha}_t})$, read from the scheduler at the current timestep. For

EDM's parametrisation $x = x_0 + \sigma\epsilon$ they are $(1/\sqrt{1+\sigma^2}, \sigma/\sqrt{1+\sigma^2})$; because of the gain normalisation of Eq. (3.13), a common scaling of (a_t, b_t) does not change the filter.

6.1.4 Classifier-free guidance and the corrector

- **DDIM with classifier-free guidance (OAT).** Every step evaluates the U-Net twice, conditioned on the problem (c) and unconditioned (u), and the DDIM update uses the guided output $u + g(c - u)$ with $g = 2$. As for Wan2.1 in KalSea, the two calls are two residual streams with their own forecasters under one shared gate, which reads the conditional stream. The DDIM update is affine in the guided output (Eq. (3.3)), so PCGS replays the gap with the scheduler's own `step` function, using the smoothed residuals of both streams, and continues from the corrected latent.
- **Guided Heun (DiffusionPDE).** Each step makes two network calls, at (x_i, σ_i) and at the Euler point $(\tilde{x}_{i+1}, \sigma_{i+1})$, and then subtracts guidance gradients that are back-propagated through both calls. The update is no longer linear in the outputs. Because our harness owns the sampling loop, PCGS replays the complete guided Heun update of every skipped step with the smoothed residuals of both calls, and then recomputes step b from the corrected latent with the residuals observed at b (no additional block-stack evaluation). The two calls of a step are again two residual streams, and the gate decides once per step.

When the harness owns the loop, continuing from the corrected latent directly is mathematically identical to returning the velocity offset of Eq. (5.8); the offset form is only needed when the cache must leave a third-party sampler unchanged, as in the KalSea release.

Guidance on skipped steps. In DiffusionPDE the guidance gradient $\nabla_x \mathcal{L}(D(x))$ is taken through the denoiser. On a skipped step the forecast residual is a constant, so the gradient flows through Emb, Out and the EDM skip connection $c_{\text{skip}}x$ only. Caching therefore saves both the forward *and* the backward pass through the block stack on skipped steps, but it also approximates the guidance Jacobian; the experiments below show that the closed loop of KalSea is essential in this setting.

6.1.5 Implementation

Both applications share one caching core (`cachecore.py`, Listing A.10): the SEA gate (accumulate-and-reset with the reference refreshed every step), the fillers and the PCGS smoother. The fillers are imported *unchanged* from the two code releases: `SpectrumResidualForecaster` from ReSPect ($M = 4$, $K = 100$, $\lambda = 0.1$, $w = 0.5$, warm-up 4) and the per-element, calibration-free `KalSeaForecaster` from KalSea-v0.2 ($\tau = 0.5$, $R = 10^{-3}$, step-index time). The smoother gains are computed exactly as in the KalSea release. Two

Table 6.1: How the caches are mapped onto the two engineering diffusion models.

	OAT (topology optimisation)	DiffusionPDE (Navier–Stokes)
Denoiser	conditional U-Net (diffusers `UNet2DModel`, channels 320/640/1280/1280) + problem encoder	EDM DDPM++ U-Net (SongUNet), 2 channels (a, u)
Sample	$1 \times 64 \times 64$ latent of a neural-field autoencoder	$2 \times 128 \times 128$ field
Sampler	20 DDIM steps, v-prediction, zero terminal SNR, CFG $g = 2$	EDM Heun, 2000 steps, $\sigma \in [0.002, 80]$, $\rho = 7$, gradient guidance
Network calls (uncached)	$20 \times 2 = 40$ per sample	$2 \times 1999 + 1 = 3999$ per solution (each with a backward pass)
Emb / Out	input conv / GN–SiLU–conv	input conv + first block / GN–SiLU–conv
h^{in}, h^{out}	320 channels, 64×64	256 channels, 128×128
Streams	conditional, unconditional	first and second Heun call
SEA coefficients	$(\sqrt{\bar{\alpha}_t}, \sqrt{1 - \bar{\alpha}_t})$, $\beta = 2$	$(1, \sigma)$ up to scale, $\beta = 2$
PCGS replay	scheduler `step` on the guided output	full guided Heun step

drivers, `oat_cache.py` and `pde_cache.py`, split the respective denoisers and own the sampling loops. Table 6.1 summarises the mapping.

6.1.6 Validation

Before producing any result, both cached samplers were checked against exact invariants, in the spirit of the KalSea release (Section 5.5):

- **OAT.** The split $\text{Out}(F(\text{Emb}(x)))$ is bitwise identical to the original `CTOPUNet` forward; with caching disabled, our sampling loop (20 DDIM steps, guidance 2.0) reproduces the decoded structures of OAT's own `OATPipeline.inference` bitwise; and with $\delta = 0$ (nothing skipped) both ReSPect and KalSea reproduce the uncached latents bitwise. OAT's evaluation script, run on the SIMP ground-truth structures, returns a compliance error of 0% and a pass rate of 100%.
- **DiffusionPDE.** The split is bitwise identical to the original `EDMPrecond` denoiser at $\sigma \in \{80, 1, 0.01\}$; with $\delta = 0$, KalSea reproduces the uncached guided sampler bitwise; with an arbitrarily large δ (only the first and last steps computed), PCGS corrects the single remaining gap and the result stays finite.

6.2 Application I: structural topology optimisation with OAT

6.2.1 Problem

Given a design domain Ω discretised into finite elements, a set of supports and point loads $\mathbf{f}$, and a target volume fraction $V^\star$, minimum-compliance topology optimisation seeks a density field $\rho_e \in [0, 1]$ per element that solves

$$\min_{\boldsymbol{\rho}} \; C(\boldsymbol{\rho}) = \mathbf{f}^\top \mathbf{u}(\boldsymbol{\rho}) \quad \text{s.t.} \quad \mathbf{K}(\boldsymbol{\rho})\,\mathbf{u} = \mathbf{f}, \qquad \frac{1}{|\Omega|}\sum_e \rho_e v_e = V^\star, \qquad 0 \le \rho_e \le 1, \quad (6.3)$$

where $\mathbf{K}(\boldsymbol{\rho})$ is the global stiffness matrix assembled with the SIMP interpolation $E(\rho_e) = E_{\min} + \rho_e^p (E_0 - E_{\min})$ [71–73]. Classical solvers need a finite-element analysis in each of tens to hundreds of iterations. OAT replaces the iterations by a single generative pass that predicts a near-optimal layout directly from the problem description.

6.2.2 The OAT model

OAT [7] consists of a resolution- and shape-agnostic *neural-field autoencoder* (NFAE), whose implicit decoder renders a density field at any resolution and aspect ratio, and a *conditional latent diffusion model* trained on OpenTO, a dataset of 2.2 million optimised structures. The diffusion model (`OpenTO/LDM`) is a diffusers U-Net on a $1 \times 64 \times 64$ latent with block widths 320, 640, 1280 and 1280, attention in the two lowest-resolution levels, and a *problem encoder* that embeds the boundary conditions and loads (as point sets) and global problem descriptors such as the target volume fraction; the problem embedding is added to the timestep embedding. The released sampler is DDIM with v-prediction on a cosine schedule rescaled to zero terminal SNR [18]. The decoded density field is evaluated by a GPU finite-element solver (pyEDGE) shipped with the code.

6.2.3 Experimental protocol

We follow the protocol of the OAT paper for its fully general OpenTO benchmark (its Table 4) exactly:

- **Problems.** The complete OpenTO test split, 5000 problems with arbitrary domains, aspect ratios, resolutions, loads, supports and volume fractions, each with a SIMP-optimised ground truth.
- **Sampler.** 20 deterministic DDIM steps with classifier-free guidance $g = 2.0$, 10 samples per problem (50,000 structures per configuration). The 10 samples of a problem form one batch and share one gate schedule.
- **Metrics.** Computed with OAT's own evaluation script (`RunComplianceTest.py`, run unchanged): the compliance error $\mathrm{CE} = (C - C_{\mathrm{SIMP}})/C_{\mathrm{SIMP}} \times 100\%$ from a

Table 6.2: Topology optimisation with OAT on the full OpenTO test split (5000 problems, 10 samples each, 20 DDIM steps, guidance 2.0). Evals: U-Net evaluations per sample (two per computed step). †Copied from the OAT paper [7, Table 4] (not re-run). CE*, VFE*: over non-failed samples; failure: CE > 100%.

Method	Evals	Failure (%)↓	CE* (%)↓	Median CE* (%)↓	VFE* (%)
OAT (uncached)†	40	39.22	8.17	3.75	−0.12
ReSPect ($\delta = 0.15$)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
KalSea ($\delta = 0.15$)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
ReSPect ($\delta = 0.3$)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
KalSea ($\delta = 0.3$)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

[TBD] (figure generated after the evaluation)

Figure 6.1: Worked examples: OpenTO test problems 0 and 1.

finite-element analysis of the generated density field, the volume-fraction error VFE, and the *failure rate*, the fraction of samples with CE > 100%. CE* and VFE* are averaged over non-failed samples, and the median CE* is reported because the mean is dominated by a few outliers.

- **Methods and budgets.** ReSPect and KalSea at $\delta = 0.15$ (about 14 of 20 steps computed) and $\delta = 0.3$ (10 of 20 steps computed, i.e. half the U-Net evaluations). The row of the original OAT is copied from its paper.

6.2.4 Results

Table 6.2 reports the full evaluation and Fig. 6.1 shows the first two test problems.

6.3 Application II: flow past a cylinder with DiffusionPDE

6.3.1 Problem

Many fluid-mechanics problems are *partially observed*: a few sensors measure the flow, and the full field is needed. DiffusionPDE [11] learns the joint distribution of a pair of

fields (a, u), an earlier and a later state of a flow, with a diffusion model, and solves *forward* problems (observe a sparsely, recover u) and *inverse* problems (observe u sparsely, recover a). For the bounded Navier–Stokes case used here, a and u are velocity-magnitude fields of the flow past a cylinder at two time instants on a 128×128 grid; 1% of the grid points of the observed field are known at random, together with the cylinder region.

6.3.2 The DiffusionPDE sampler

The model is an EDM [16] DDPM++ U-Net trained on the joint two-channel field. Sampling uses 2000 Heun steps on the EDM schedule ($\sigma_{\max} = 80$, $\sigma_{\min} = 0.002$, $\rho = 7$). After each step the latent is moved along the negative gradients of the observation losses and, during the last 20% of the steps, of a finite-difference PDE residual, with the weights ζ of the released configuration; the gradients are taken with respect to the current latent through the network, in the manner of diffusion posterior sampling [79]. Each solution therefore costs 3999 network calls, each with a backward pass.

6.3.3 Experimental protocol

- **Cases.** The two bounded Navier–Stokes test cases released with DiffusionPDE, each solved as a forward and as an inverse problem, with the released configurations (2000 steps, seed 0, $\zeta_{\text{obs}} = 0.5$, $\zeta_{\text{pde}} = 1$).
- **Metric.** The relative L^2 error of the recovered field with respect to the simulated ground truth, $\|u - u_{\text{GT}}\|_2 / \|u_{\text{GT}}\|_2$ for the forward problem (and of a for the inverse problem), as computed by the released scripts.
- **Methods and budgets.** ReSPect and KalSea at $\delta = 0.01$ and $\delta = 0.02$. A calibration run on forward case 1 showed that the error grows quickly for larger thresholds (KalSea: 10.1% at $\delta = 0.03$, 12.2% at 0.1, 24.6% at 0.3, with 1041, 499 and 181 network calls).
- **Reference.** The error of the uncached sampler is copied from the DiffusionPDE paper: 3.9 ± 0.2% (forward) and 2.7 ± 0.2% (inverse), averaged over 1000 test scenes [11]. Only two of these scenes are released, so the comparison of our two cases with the paper's mean is indicative only.

6.3.4 Results

Table 6.3 summarises the 16 cached solutions. With KalSea at $\delta = 0.01$, the number of network calls drops from 3999 to about 2300 (1.7× fewer), and at $\delta = 0.02$ to about 1550 (2.6× fewer). On the forward problem of case 1, KalSea recovers the flow with 3.3% error at both budgets, below the paper's mean of 3.9% for the uncached sampler, and on the inverse problem it stays at 3.6–5.2% for both cases. ReSPect, at the same thresholds (and within 3% of the same number of network calls), is worse in every one of the eight comparisons, most

Table 6.3: Bounded Navier–Stokes (flow past a cylinder) with DiffusionPDE, 1% observations. Calls: network evaluations per solution (mean of the two cases; uncached: 3999). Errors: relative L^2 error of the recovered field w.r.t. the simulation. †Copied from the DiffusionPDE paper (mean ± std over 1000 scenes; not re-run).

		Forward (recover u)			Inverse (recover a)		
Method	δ	Calls	Case 1	Case 2	Calls	Case 1	Case 2
DiffusionPDE (uncached)†	–	3999	3.9 ± 0.2%		3999	2.7 ± 0.2%	
ReSPect	0.01	2301	9.24%	13.34%	2277	5.78%	6.88%
KalSea	0.01	2360	**3.32%**	**7.02%**	2265	**5.21%**	**5.23%**
ReSPect	0.02	1503	16.81%	16.35%	1522	8.17%	8.59%
KalSea	0.02	1552	**3.25%**	**14.58%**	1533	**4.98%**	**3.59%**

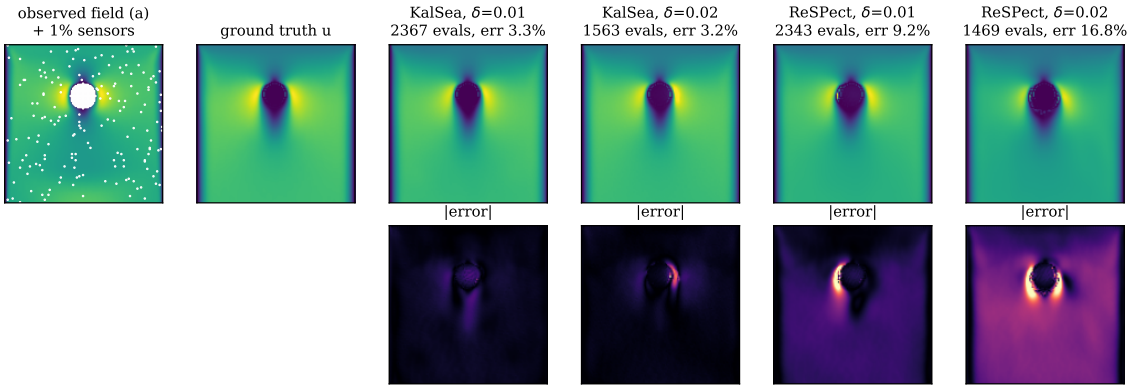


Figure 6.2: Forward problem, case 1: from 1% observations of the earlier velocity field a (white dots) and the cylinder, the later field u is recovered. Top: ground truth and the solutions of KalSea and ReSPect at $\delta = 0.01$ and 0.02 with their network calls and errors; bottom: absolute error maps (same colour scale). ReSPect's errors concentrate in the wake of the cylinder.

clearly on the forward problem (9.2–16.8% vs. 3.3–14.6%). Unlike on the media models, where ReSPect's open-loop forecast was competitive, here the closed loop is essential: the gradient guidance on skipped steps is approximate, and KalSea's corrector re-integrates every gap with the full guided update, repairing the drift that both the forecast and the approximate guidance introduce. The hardest setting is forward case 2 at $\delta = 0.02$, where KalSea's error rises to 14.6%; the budget at which caching is safe therefore depends on the case, which motivates a gate that also accounts for the guidance (Chapter 7). Figures 6.2 and 6.3 show the recovered fields and their errors.

Wall-clock time. The cached runs were executed concurrently with the OAT runs on shared GPUs, so their wall-clock times (185–248 s per solution) are not a clean measurement and are not compared with the paper's timing (140 s on an A40). The saving in network calls carries over to forward and backward passes alike, but a skipped step still evaluates Emb (which includes one full-resolution block), Out and their gradients, so the time saved is smaller than the reduction in calls.

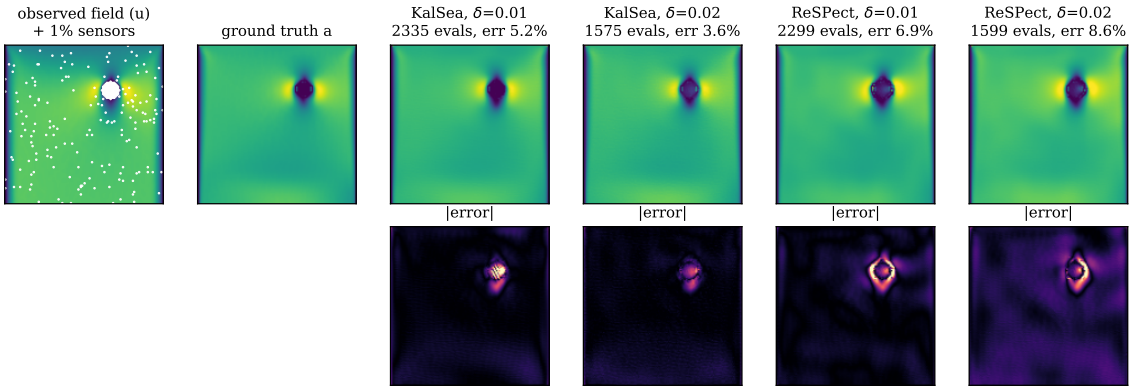


Figure 6.3: Inverse problem, case 2: from 1% observations of the later field u, the earlier field a is recovered; layout as in Fig. 6.2.

6.4 Reproduction

The pretrained weights (OAT: `OpenTO/NFAE` and `OpenTO/LDM` from Hugging Face; DiffusionPDE: the released pretrained models) and the test data (OpenTO test split; DiffusionPDE bounded Navier–Stokes cases) are those released by the authors. The runs are reproduced by

```
# Application I: OAT, full OpenTO test split, one shard per GPU (g = 0..7)
python oat_cache.py --method kalsea --delta 0.3 --steps 20 --guidance 2.0 \
    --n_samples 10 --shard $g --num_shards 8 --out results/oat
python eval_oat.py --method kalsea --delta 0.3     # OAT's RunComplianceTest.py
# Application II: DiffusionPDE, bounded Navier-Stokes
python pde_cache.py --case 1 --problem forward --steps 2000 \
    --methods respect,kalsea --deltas 0.01,0.02
```

6.5 Discussion

- **Short versus long schedules.** OAT is designed for few-step sampling, so the room for skipping is limited to a few of its 20 steps. DiffusionPDE, at the other extreme, takes 2000 small guided steps; there the gate skips about 40–60% of the network calls. Together the two applications bracket the regimes in which caching is used.
- **Physics-based acceptance.** For engineering use, a cached sample is acceptable only if its physical performance is preserved. Both studies therefore report engineering metrics, compliance from a finite-element analysis and the error with respect to a flow simulation, rather than visual similarity alone.
- **Open versus closed loop under guidance.** The DiffusionPDE results give the clearest evidence in this report for the benefit of the corrector: when guidance gradients are approximated on skipped steps, an open-loop forecast lets the error accumulate, while KalSea's re-integration of every gap keeps the solution close to the observations and the PDE.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project set out to make diffusion models faster to sample without retraining them, by improving training-free feature caching. A cache makes two decisions at every step, *when* to evaluate the network and *what* to write on the steps it skips. The literature review (Chapter 2) showed that existing work improved these decisions in isolation, forecast the full hidden state rather than the part that is actually unknown, tuned the forecasters' hyper-parameters empirically, and, without exception, left the errors made on skipped steps uncorrected. The project produced two methods that address these gaps.

ReSPect (Chapter 4) couples SeaCache's spectrally aligned gate, derived from the linear-MMSE (Wiener) denoiser, with a forecaster of the block-stack *residual*, the only part of a skipped step's features that is not known exactly. The forecaster blends a ridge-regularised Chebyshev fit of the whole residual history with a first-order discrete Taylor step. We showed that copy-reuse has an error bound linear in the skip length, that Taylor extrapolation attains its ideal $(P{+}1)$-order rate only under a node-spacing condition that a dynamic gate does not guarantee (with a sharp counterexample), and that the Chebyshev truncation error is independent of the skip length; we derived the blend weight and the warm-up from bias–variance and degrees-of-freedom arguments. At matched or lower compute, ReSPect achieved speedups of 3.30×, 2.63× and 2.77× on FLUX.1-dev, Wan2.1-1.3B and HunyuanVideo, and improved on the strongest compared baseline by up to 3.46, 11.4 and 9.9 dB PSNR respectively.

KalSea (Chapter 5) closes the loop that every previous filler leaves open. A damped-trend Kalman filter forecasts the residual on skipped steps, and when the next step is computed, predictor–corrector gap smoothing (PCGS) uses the Rauch–Tung–Striebel smoother of the filter's own model to correct the residuals of the whole gap, re-integrates the gap through the output layers only, and continues sampling from the corrected latent. PCGS adds no transformer evaluations and no hyper-parameters; we derived its gain in closed form and extended it to multistep solvers, classifier-free guidance and two-stream video-and-audio generation. On FLUX.1-dev, KalSea reached 27.72 dB at 18.1 of 50 evaluations (SeaCache: 26.29 dB at 18.4) and 22.53 dB at 13 (SeaCache: 21.33 dB); PCGS alone contributed 0.81 and 0.61 dB and also improved reuse and linear extrapolation. On HunyuanVideo it improved

on SeaCache in the same setup by 3.17 and 1.92 dB, and on Wan2.1 it exceeded SeaCache's published results by 5.07 and 2.07 dB. The released implementation supports five backbones, including FLUX.2 [dev] and MiniMax-H3, and is verified against exact invariants.

Applications in mechanical engineering (Chapter 6). We carried both caches over to two open-weight diffusion models from mechanical engineering: structural topology optimisation with Optimize Any Topology and the reconstruction of a flow past a cylinder with DiffusionPDE. This required generalising the residual split to U-Net denoisers, the gate to timestep-modulated convolutional inputs, the SEA coefficients to DDIM and EDM schedules, and PCGS to DDIM with classifier-free guidance and to gradient-guided second-order samplers; all of it was validated against exact invariants before use. On DiffusionPDE, KalSea reduced the network evaluations of the 2000-step guided sampler by 1.7–2.6× and was more accurate than ReSPect in all eight forward and inverse comparisons, recovering the flow of the first released case with 3.3% error (the paper reports 3.9% on average for the uncached sampler). The OAT evaluation follows the original paper's protocol on its full 5000-problem test split, with compliance computed by the model's own finite-element analysis.

Taken together, the project shows that the two decisions of a cache should be designed jointly, that forecasting the residual rather than the hidden state is both theoretically and practically preferable, and that a cache need not be open loop: the observation made at each computed step can be used not only to predict the future but also to repair the past, at almost no cost.

7.2 Future work

- **Extend the engineering evaluation.** Evaluate DiffusionPDE on all 1000 test scenes of each PDE (Darcy, Poisson, Helmholtz, Navier–Stokes) as the scenes become available, and measure clean wall-clock speedups on dedicated GPUs.
- **Guidance-aware gating.** On DiffusionPDE the safe skipping budget depended on the case; a gate that also measures the change of the guidance signal could choose it automatically.
- **Combine the two methods.** PCGS improved every filler on which it was tested; wrapping it around ReSPect's Chebyshev–Taylor forecaster, with gains from a linear interpolation of the innovation, is a natural next experiment.
- **Choose the computed steps jointly with the corrector.** With a closed loop, the gate could account for the fact that the error of a gap will be corrected, for example by using the filter's predicted uncertainty to decide when to compute.

- **Correct caches inside the block stack.** Many caches reuse features of individual blocks, attention or MLP layers; extending smoothing and re-integration to such intermediate features is open.
- **Larger corrections.** PCGS attaches the residual of the uncorrected latent to the corrected one, which is accurate only for small corrections; long gaps could benefit from a cheap first-order update of the residual itself.
- **Guided and physics-constrained samplers.** Engineering samplers often add gradient guidance or constraint projections. Understanding how skipped steps affect such guidance, and how correction interacts with it, would make caching safe to use in design and simulation workflows.

References

[1] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning (ICML)*, 2015.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[3] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[4] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[6] François Mazé and Faez Ahmed. Diffusion models beat GANs on topology optimization. In *AAAI Conference on Artificial Intelligence*, 2023.

[7] Amin Heyrani Nobari, Lyle Regenwetter, Cyril Picard, Ligong Han, and Faez Ahmed. Optimize any topology: A foundation model for shape- and resolution-free structural topology optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.23667.

[8] Jan-Hendrik Bastek and Dennis M. Kochmann. Inverse design of nonlinear mechanical metamaterials via video denoising diffusion models. *Nature Machine Intelligence*, 5: 1466–1475, 2023.

[9] Xiang Xu, Joseph G. Lambourne, Pradeep Kumar Jayaraman, Zhengqing Wang, Karl D. D. Willis, and Yasutaka Furukawa. BrepGen: A B-rep generative diffusion model with structured latent geometry. *ACM Transactions on Graphics (SIGGRAPH)*, 43(4), 2024.

[10] Dule Shu, Zijie Li, and Amir Barati Farimani. A physics-informed diffusion model for high-fidelity flow field reconstruction. *Journal of Computational Physics*, 478:111972, 2023.

[11] Jiahe Huang, Guandao Yang, Zichen Wang, and Jeong Joon Park. DiffusionPDE: Generative PDE-solving under partial observation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.17763.

[12] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023.

[13] Jiwoo Chung, Sangeek Hyun, MinKyu Lee, Byeongju Han, Geonho Cha, Dongyoon Wee, Youngjun Hong, and Jae-Pil Heo. SeaCache: Spectral-evolution-aware cache for accelerating diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14283–14294, 2026. arXiv:2602.18993.

[14] Jiaqi Han, Juntong Shi, Puheng Li, Haotian Ye, Qiushan Guo, and Stefano Ermon. Adaptive spectral feature forecasting for diffusion sampling acceleration. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 43320–43330, 2026. Spectrum; arXiv:2603.01623.

[15] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[16] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 26565–26577, 2022.

[17] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*, 2021.

[18] Shanchuan Lin, Bingchen Liu, Jiashi Li, and Xiao Yang. Common diffusion noise schedules and sample steps are flawed. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024.

[19] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[20] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2022. arXiv:2207.12598.

[21] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.

[22] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, pages 12606–12633, 2024.

[23] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[24] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[26] Black Forest Labs. FLUX.1 [dev]. `https://github.com/black-forest-labs/flux`, 2024. Open-source code and model weights.

[27] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[28] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[29] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 49842–49869, 2023.

[30] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5775–5787, 2022.

[31] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22(4):730–751, 2025.

[32] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022.

[33] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, pages 32211–32252, 2023.

[34] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023.

[35] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *European Conference on Computer Vision (ECCV)*, pages 87–103, 2024.

[36] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[37] Xiuyu Li, Yijiang Liu, Long Lian, Huanrui Yang, Zhen Dong, Daniel Kang, Shanghang Zhang, and Kurt Keutzer. Q-Diffusion: Quantizing diffusion models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17535–17545, 2023.

[38] Tianchen Zhao, Tongcheng Fang, Haofeng Huang, Enshu Liu, Rui Wan, Widyadewi Soedarmadji, Shiyao Li, Zinan Lin, Guohao Dai, Shengen Yan, et al. ViDiT-Q: Efficient and accurate quantization of diffusion transformers for image and video generation. *arXiv preprint arXiv:2406.02540*, 2024.

[39] Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15762–15772, 2024.

[40] Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, Christian Rupprecht, Daniel Cremers, Peter Vajda, and Jialiang Wang. Cache me if you can: Accelerating diffusion models through block caching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6211–6220, 2024.

[41] Pengtao Chen, Mingzhu Shen, Peng Ye, Jianjian Cao, Chongjun Tu, Christos-Savvas Bouganis, Yiren Zhao, and Tao Chen. Δ-DiT: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.

[42] Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.

[43] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations (ICLR)*, 2025.

[44] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-time video generation with pyramid attention broadcast. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2408.12588.

[45] Zhengyao Lv, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K. Wong. FasterCache: Training-free video diffusion model acceleration with high quality. In *International Conference on Learning Representations (ICLR)*, 2025.

[46] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 133282–133304, 2024.

[47] Huanpeng Chu, Wei Wu, Guanyu Feng, and Yutao Zhang. OmniCache: A trajectory-oriented global perspective on training-free cache reuse for diffusion transformer models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16302–16312, 2025.

[48] Kumara Kahatapitiya, Haozhe Liu, Sen He, Ding Liu, Menglin Jia, Chenyang Zhang, Michael S. Ryoo, and Tian Xie. Adaptive caching for faster video generation with diffusion transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15240–15252, 2025.

[49] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7353–7363, 2025.

[50] Zehong Ma, Longhui Wei, Feng Wang, Shiliang Zhang, and Qi Tian. MagCache: Fast video generation with magnitude-aware cache. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 38592–38624, 2025.

[51] Jiazi Bu, Pengyang Ling, Yujie Zhou, Yibin Wang, Yuhang Zang, Dahua Lin, and Jiaqi Wang. DiCache: Let diffusion model determine its own cache. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2508.17356.

[52] Jiacheng Liu, Peiliang Cai, Qinming Zhou, et al. FreqCa: Accelerating diffusion models via frequency-aware caching. *arXiv preprint arXiv:2510.08669*, 2025.

[53] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15853–15863, 2025. arXiv:2503.06923.

[54] Liang Feng, Shikang Zheng, Jiacheng Liu, Yuqi Lin, Qinming Zhou, Peiliang Cai, Xinyu Wang, Junjie Chen, Chang Zou, Yue Ma, and Linfeng Zhang. HiCache: A plug-in scaled-Hermite upgrade for Taylor-style cache-then-forecast diffusion acceleration. In *International Conference on Learning Representations (ICLR)*, 2026.

[55] Shikang Zheng, Liang Feng, Xinyu Wang, Qinming Zhou, Peiliang Cai, Chang Zou, Jiacheng Liu, Yuqi Lin, Junjie Chen, Yue Ma, and Linfeng Zhang. Forecast then calibrate: Feature caching as ODE for efficient diffusion transformers. In *AAAI Conference on Artificial Intelligence*, pages 13449–13457, 2026.

[56] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[57] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice, Extended Edition*. SIAM, 2019.

[58] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Fei Ren, Shaobo Wang, Kaixin Li, and Linfeng Zhang. SpeCa: Accelerating diffusion transformers with speculative feature caching. In *ACM International Conference on Multimedia*, pages 10024–10033, 2025.

[59] Xurui Peng, Chenqian Yan, Hong Liu, Rui Ma, Fangmin Chen, Xing Wang, Zhihua Wu, Songwei Liu, and Mingbao Lin. ERTACache: Error rectification and timesteps adjustment for efficient diffusion. In *International Conference on Learning Representations (ICLR)*, 2026.

[60] Zhirong Shen, Rui Huang, Chang Zou, Shikang Zheng, Jiacheng Liu, Peiliang Cai, Zhengyi Shi, Yaosong Du, Liang Feng, Xiaobing Tu, Jinkui Ren, Xiantao Zhang, and Linfeng Zhang. Accelerating diffusion transformers with Gaussian process rectified feature cache. In *European Conference on Computer Vision (ECCV)*, 2026.

[61] Javad Rajabi, Hakki Can Karaimer, David B. Lindell, Babak Taati, and Michael S. Brown. SpecKCast: Spectral Kalman forecasting for diffusion acceleration. *arXiv preprint arXiv:2610.26837*, 2026.

[62] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[63] H. E. Rauch, F. Tung, and C. T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[64] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.

[65] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[66] Everette S. Gardner and Ed McKenzie. Forecasting trends in time series. *Management Science*, 31(10):1237–1246, 1985.

[67] Raman K. Mehra. Approaches to adaptive filtering. *IEEE Transactions on Automatic Control*, 17(5):693–698, 1972.

[68] Michael Schober, Simo Särkkä, and Philipp Hennig. A probabilistic model for the numerical solution of initial value problems. *Statistics and Computing*, 29(1):99–122, 2019.

[69] Filip Tronarp, Hans Kersting, Simo Särkkä, and Philipp Hennig. Probabilistic solutions to ordinary differential equations as nonlinear Bayesian filtering: A new perspective. *Statistics and Computing*, 29(6):1297–1315, 2019.

[70] Lyle Regenwetter, Amin Heyrani Nobari, and Faez Ahmed. Deep generative models in engineering design: A review. *Journal of Mechanical Design*, 144(7):071704, 2022.

[71] Martin Philip Bendsøe and Noboru Kikuchi. Generating optimal topologies in structural design using a homogenization method. *Computer Methods in Applied Mechanics and Engineering*, 71(2):197–224, 1988.

[72] Ole Sigmund. A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization*, 21(2):120–127, 2001.

[73] Erik Andreassen, Anders Clausen, Mattias Schevenels, Boyan S. Lazarov, and Ole Sigmund. Efficient topology optimization in MATLAB using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, 2011.

[74] Giorgio Giannone, Akash Srivastava, Ole Winther, and Faez Ahmed. Aligning optimization trajectories with diffusion models for constrained design generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[75] Amin Heyrani Nobari, Giorgio Giannone, Lyle Regenwetter, and Faez Ahmed. NITO: Neural implicit fields for resolution-free topology optimization. *arXiv preprint arXiv:2402.05073*, 2024.

[76] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[77] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[78] Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *arXiv preprint arXiv:2309.01745*, 2023.

[79] Hyungjin Chung, Jeongsol Kim, Michael T. McCann, Marc L. Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023.

[80] Norbert Wiener. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT Press, 1949.

[81] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[82] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 586–595, 2018.

[83] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L. Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 36479–36494, 2022.

[84] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21807–21818, 2024.

[85] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

Appendix A

Code

This appendix describes the organisation of the code written in this project and reproduces its key parts. The listings are included directly from the source files (copied unchanged into the `code/` folder of the thesis), so they are exact excerpts; the line numbers are those of the original files.

A.1 Repository layout

Both code releases follow the layout of SeaCache: one self-contained folder per backbone, so that each folder can be copied into the corresponding official repository (HunyuanVideo, Wan2.1) or run directly (diffusers models).

```
ReSpect/                          ReSPect release
  FLUX/  HunyuanVideo/  Wan2.1/
    respect_generate.py           patched transformer forward + generation script
    spectrum_forecaster.py        Chebyshev ridge + discrete Taylor forecaster
    util_seacache.py              SEA (Wiener) filter, relative L1 distance

KalSea-v0.2/                      KalSea release
  FLUX/  FLUX2/  HunyuanVideo/  Wan2.1/  MiniMax-H3/
    kalsea_generate.py            generation script (switches --no_pcgs/--reuse/--no_cache/--delta)
    kalsea_forecaster.py          damped-trend Kalman residual forecaster
    pcgs.py                       predictor-corrector gap smoothing (FLUX, FLUX2, MiniMax-H3)
    kalsea_hunyuan.py, kalsea_wan.py   gate + forecaster + PCGS for the official video repos
    util_seacache.py              SEA filter (identical in every folder)

mech_apps/cache/                  applications of Chapter 6
  cachecore.py                    shared gate, fillers and PCGS smoother
  oat_cache.py                    Optimize Any Topology (topology optimisation)
  pde_cache.py                    DiffusionPDE (bounded Navier-Stokes)
```

A.2 The SEA filter

The filter of Eqs. (3.12) and (3.13): a separable Wiener gain per axis under a power-law prior, normalised to unit mean gain, applied in the Fourier domain. The gate's distance is the relative ℓ_1 change.

Listing A.1: SEA filter (`util_seacache.py`).

```
23 def apply_sea_from_ab(
24     x: torch.Tensor,
25     a: float,  # a_t
26     b: float,  # b_t
27     power_exp: float = 2.0,
```

```
    power_const: float = 1.0,
    dims=None,  # e.g., images: (-2, -3); videos THW: (-4, -3, -2)
    eps: float = 1e-16,
    norm_mode: str = "mean",  # "peak" | "mean"
    *,
    real: bool = False,
) -> torch.Tensor:
    """
    Apply an N-D separable Wiener filter using (a_t, b_t).

    - If multiple dims are given, a 1D Wiener gain is built per axis from |f|,
      and their product forms a separable N-D filter.
    - Normalization ("peak"/"mean") is applied to the full N-D filter
    """
    orig_dtype = x.dtype
    x = x.contiguous()
    x32 = x.to(torch.float32)

    # --- Auto-select dims if not provided ---
    if dims is None:
        if x32.ndim <= 2:
            dims = tuple(range(x32.ndim))
        else:
            dims = tuple(range(-2, -x32.ndim, -1))

    # --- FFT ---
    if real:
        X = torch.fft.rfftn(x32, dim=dims)
    else:
        X = torch.fft.fftn(x32, dim=dims)

    # --- Choose filter H ---
    H = None
    for i, ax in enumerate(dims):
        N = x32.shape[ax]
        if real and (i == len(dims) - 1):
            f = torch.fft.rfftfreq(N, device=x32.device, dtype=torch.float32)
        else:
            f = torch.fft.fftfreq(N, device=x32.device, dtype=torch.float32)

        rad = torch.abs(f)
        Sx0 = power_const / ((rad ** power_exp) + eps)
        H1 = (a * Sx0) / (a * a * Sx0 + (b * b) + eps)

        shape_i = [1] * x32.ndim
        shape_i[ax] = H1.shape[0]
        H1 = H1.reshape(shape_i)  # [1, ..., Ni', ..., 1]
        H = H1 if H is None else (H * H1)

    # --- Normalize  ---
    nm = norm_mode.lower()
    if nm == "peak":
        maxv = torch.amax(H)
        if torch.isfinite(maxv) and maxv > 0:
            H = H / maxv
    elif nm == "mean":
        if real:
            # Use weighted mean along the last FFT axis to match full-spectrum mean.
            N_last = int(x32.shape[dims[-1]])
```

```
            w_last = _rfft_full_mean_weights_1d(
                N_last, device=x32.device, dtype=torch.float32
            )  # [N//2+1]
            wshape = [1] * x32.ndim
            wshape[dims[-1]] = w_last.numel()
            W = w_last.view(*wshape)

            denom = torch.sum(W) * float(torch.prod(torch.tensor([x32.shape[d] for d in dims[:-1]])))
            meanv = torch.sum(H * W) / denom
        else:
            meanv = torch.mean(H)
        if torch.isfinite(meanv) and meanv > 0:
            H = H / meanv

    # --- Apply SEA filter and iFFT ---
    Y = X * H
    if real:
        s = [x32.shape[d] for d in dims]
        y = torch.fft.irfftn(Y, s=s, dim=dims)
    else:
        y = torch.fft.ifftn(Y, dim=dims).real

    return y.to(orig_dtype)
```

Listing A.2: Relative ℓ_1 distance (`util_seacache.py`).

```
def rel_l1(a: torch.Tensor, b: torch.Tensor, eps: float = 1e-16) -> float:
    num = (a - b).abs().mean()
    den = b.abs().mean() + eps
    return float((num / den).detach().cpu())
```

A.3 The gate in the patched FLUX.1 forward

The accumulate-and-reset rule of Eq. (3.14) as implemented in KalSea's patched forward of `FluxTransformer2DModel`: the first block's modulated input is laid out on its token grid, SEA-filtered for the current step, and compared with the stored reference, which is refreshed at every step.

Listing A.3: SEA gate in the patched FLUX.1 forward (`KalSea-v0.2/FLUX/kalsea_generate.py`).

```
    # ---- SeaCache gating ----
    should_calc = True
    step_idx = int(getattr(self, "cnt", 0))  # index of the current sampler step (cnt is reset after the last step)
    if getattr(self, "enable_seacache", False):
        inp = hidden_states
        temb_ = temb
        modulated_inp, gate_msa, shift_mlp, scale_mlp, gate_mlp = self.transformer_blocks[0].norm1(inp, emb=temb_)
        # Track change based on the first block's norm1 modulation
        if self.cnt == 0 or self.cnt == self.num_steps - 1 or self.previous_modulated_input is None:
            should_calc = True
            self.accumulated_rel_l1_distance = 0.0
        else:
```

```
            # Apply SEA filtering before computing distance
            modulated_inp = modulated_inp.reshape(
                modulated_inp.shape[0],
                int(img_ids[:, 1].max().item() + 1),
                int(img_ids[:, 2].max().item() + 1),
                modulated_inp.shape[-1],
            )
            modulated_inp = apply_sea_with_scheduler(
                modulated_inp,
                self.scheduler,
                getattr(self, "cnt", 0),
                power_exp=2.0,
                dims=(-2, -3),
                norm_mode="mean",
            )
            modulated_inp = modulated_inp.reshape(modulated_inp.shape[0], -1, modulated_inp.shape[-1])
            self.accumulated_rel_l1_distance += rel_l1(modulated_inp, self.previous_modulated_input)

            if self.accumulated_rel_l1_distance < float(self.seacache_thresh):
                should_calc = False
            else:
                should_calc = True
                self.accumulated_rel_l1_distance = 0.0

        self.previous_modulated_input = modulated_inp
        self.cnt += 1
        if self.cnt == self.num_steps:
            # Reset step counter at the end of a full trajectory
            self.cnt = 0
```

A.4 The ReSPect forecaster

The Chebyshev design matrix (three-term recurrence), the ridge fit by Cholesky factorisation and the forecast, the first-order discrete Taylor step, and the residual forecaster that blends the two with $w = 0.5$ after the warm-up (Sections 4.4 and 4.6).

Listing A.4: Chebyshev design matrix (`spectrum_forecaster.py`).

```
    def _build_design(self, taus: torch.Tensor) -> torch.Tensor:
        taus = taus.reshape(-1, 1)
        k = taus.shape[0]
        t0 = torch.ones((k, 1), device=taus.device, dtype=taus.dtype)
        if self.M == 0:
            return t0
        t1 = taus
        cols = [t0, t1]
        for _ in range(2, self.M + 1):
            tm = 2 * taus * cols[-1] - cols[-2]
            cols.append(tm)
        return torch.cat(cols[: self.M + 1], dim=1)
```

Listing A.5: Ridge fit and Chebyshev forecast (`spectrum_forecaster.py`).

```
    def _fit_if_needed(self) -> None:
        if self._coef is not None:
```

```
            return
        assert self.ready()
        taus = self._taus(self.t_buf)
        X = self._build_design(taus).to(torch.float32)
        H = self._H_buf
        _, P = X.shape
        assert P == self.P
        lamI = self.lam * torch.eye(P, device=X.device, dtype=X.dtype)
        Xt = X.transpose(0, 1)
        XtX = Xt @ X + lamI
        try:
            L = torch.linalg.cholesky(XtX.to(torch.float32))
        except Exception:
            jitter = 1e-6 * XtX.diag().mean()
            L = torch.linalg.cholesky(XtX + jitter * torch.eye(P, device=X.device))

        if self.fit_chunks <= 1:
            XtH = Xt @ H.to(torch.float32)
            C = torch.cholesky_solve(XtH.to(torch.float32), L).to(DTYPE)
        else:
            # Chunked solve over the feature dim: each column-block is an
            # independent ridge solve with the same Cholesky factor, so the
            # result matches the unchunked fit while peak memory drops by
            # ~fit_chunks (the full fp32 upcast of H never materializes).
            D = H.shape[1]
            n_chunks = min(self.fit_chunks, D)
            blocks = []
            for s, e in _chunk_ranges(D, n_chunks):
                Hb = H[:, s:e].to(torch.float32)
                XtHb = Xt @ Hb
                Cb = torch.cholesky_solve(XtHb.to(torch.float32), L).to(DTYPE)
                blocks.append(Cb)
                del Hb, XtHb, Cb
            C = torch.cat(blocks, dim=1)
            del blocks
        self._coef = C
        self._XtX_fac = L
        self._tau_cache = taus
        self._X_cache = X.to(DTYPE)

    @torch.no_grad()
    def predict(self, t_star: float | torch.Tensor) -> torch.Tensor:
        assert self._shape is not None
        device = self.t_buf.device
        t_star = torch.as_tensor(t_star, dtype=DTYPE, device=device)
        self._fit_if_needed()
        tau_star = self._taus(t_star)
        x_star = self._build_design(tau_star[None])
        h_flat = x_star @ self._coef
        return _unflatten(h_flat, self._shape)
```

Listing A.6: Discrete Taylor step (`spectrum_forecaster.py`).

```
    def _local_taylor_discrete(self, t_star: torch.Tensor) -> torch.Tensor:
        H = self.cheb._H_buf
        t = self.cheb.t_buf
        h_i = H[-1]
        t_i = t[-1]
        if t.numel() < 2:
```

```
            return h_i.clone()
        h_im1 = H[-2]
        t_im1 = t[-2]
        dh1 = h_i - h_im1
        dt_last = (t_i - t_im1).clamp_min(1e-8)
        k = ((t_star - t_i) / dt_last).to(h_i.dtype)
        out = h_i + k * dh1
        if self.taylor_order >= 2 and t.numel() >= 3:
            h_im2 = H[-3]
            d2 = h_i - 2 * h_im1 + h_im2
            out = out + 0.5 * k * (k - 1.0) * d2
        if self.taylor_order >= 3 and t.numel() >= 4:
            h_im3 = H[-4]
            d3 = h_i - 3 * h_im1 + 3 * h_im2 - h_im3
            out = out + (k * (k - 1.0) * (k - 2.0) / 6.0) * d3
        return out

    @torch.no_grad()
```

Listing A.7: Residual forecaster with blend and warm-up (`spectrum_forecaster.py`).

```
class SpectrumResidualForecaster:
    """Forecast a feature tensor with Spectrum (Chebyshev + discrete Taylor).

    Matches Spectrum defaults: M=4, K=100, lam=0.1, w=0.5, taylor_order=1.
    Chebyshev is mixed in only after enough observations so a 1-point ridge
    fit cannot pollute early skipped steps (those fall back to Taylor, which
    is last-value copy with 1 obs and linear extrapolation with 2+).
    """

    def __init__(
        self,
        m: int = 4,
        k: int = 100,
        lam: float = 0.1,
        w: float = 0.5,
        taylor_order: int = 1,
        device=None,
        t_max: float = 50.0,
        min_cheb_obs: int = 4,
        fit_chunks: int = 1,
    ):
        cheb = ChebyshevForecaster(
            M=m,
            K=k,
            lam=lam,
            device=device,
            t_max=t_max,
            fit_chunks=fit_chunks,
        )
        self.spectrum = Spectrum(
            cheb,
            taylor_order=taylor_order,
            enable_blend=True,
            prefer="cheb",
            w=w,
        )
        self.min_cheb_obs = int(min_cheb_obs)
        self._feat_shape = None
```

```python

    @property
    def n_obs(self) -> int:
        return self.spectrum.n_obs

    def update(self, t, h: torch.Tensor) -> None:
        self._feat_shape = tuple(h.shape)
        self.spectrum.update(t, h.reshape(-1))

    @torch.no_grad()
    def predict(self, t) -> torch.Tensor:
        device = self.spectrum.cheb.t_buf.device
        t_star = torch.as_tensor(t, dtype=DTYPE, device=device)
        h_taylor = self.spectrum._local_taylor_discrete(t_star)
        if self.n_obs >= self.min_cheb_obs:
            h_cheb = self.spectrum.cheb.predict(t_star)
            w = float(self.spectrum.w)
            out = (1.0 - w) * h_taylor + w * h_cheb
        else:
            out = h_taylor
        return out.reshape(self._feat_shape).to(dtype=DTYPE)
```

A.5 The KalSea forecaster

Prediction at every step with the damped-trend transition and integrated-Wiener process noise (Eqs. (5.1) and (5.2)), the Kalman update on computed steps with online adaptation of the process-noise intensity (Eq. (5.3)), and the forecast on skipped steps.

Listing A.8: Kalman predict / update / forecast (`KalSea-v0.2/FLUX2/kalsea_forecaster.py`).

```python
    def _predict_step(self, j: int):
        """Advance the prior from step j-1 to j (paper Eq. 9-11)."""
        self.j_last = j
        if j == 0 or self.m is None:
            return
        dl = self.delta[:, min(j, self.N - 1)]
        rho = np.exp(-dl / self.tau)
        self.m = self.m + self._band_vec(dl) * self.nu
        self.nu = self._band_vec(rho) * self.nu
        q = self.q0 * self.qscale
        for l in range(self.L):
            A = np.array([[1.0, dl[l]], [0.0, rho[l]]])
            Q = q[l] * np.array([[dl[l] ** 3 / 3, dl[l] ** 2 / 2], [dl[l] ** 2 / 2, dl[l]]])
            self.P[l] = A @ self.P[l] @ A.T + Q

    def _advance_to(self, j: int):
        while self.j_last < j:
            self._predict_step(self.j_last + 1)

    @torch.no_grad()
    def update(self, t, h: torch.Tensor) -> None:
        """Observation at a computed step: Kalman correction of (value, derivative) of every element.
        """
        j = int(round(float(t)))
        if not self._ready:
            self._setup(h)
```

```
        self._advance_to(j)
        Z = self._to_state(h)
        if self.m is None:
            if self._p_scale_cfg is not None and self.bm is not None:
                p_scale = np.array(self._p_scale_cfg, dtype=np.float64)
            elif self._p_scale_cfg is not None:
                p_scale = np.array([float(np.mean(self._p_scale_cfg))])
            elif self.bm is None:  # online estimate of the signal power from the first observation
                p_scale = np.array([float((Z ** 2).mean())])
            else:
                p_scale = torch.zeros(self.L, device=self.dev).scatter_add_(
                    0, self.bm.reshape(-1), (Z ** 2).sum(0).reshape(-1)).cpu().numpy() / self.d
            p_scale = np.maximum(p_scale, 1e-12)
            self.p_scale = p_scale
            self.scale = self._band_vec(1.0 / np.sqrt(p_scale))
            self.m = Z * self.scale
            self.nu = torch.zeros_like(self.m)
            for l in range(self.L):
                self.P[l] = np.diag([self.r_obs, self.p22_init[l]])
        else:
            z = Z * self.scale
            e = z - self.m
            Sv = self.P[:, 0, 0] + self.r_obs
            K = self.P[:, :, 0] / Sv[:, None]                              # [L, 2]  (H = [1, 0])
            if self.adapt_q:
                err = self._band_err(e)
                g = np.clip(err / np.maximum(self.P[:, 0, 0] + self.r_obs, 1e-9), 0.1, 10.0)
                self.qscale = np.clip(self.qscale * g ** 0.2, 0.1, 10.0)
            self.m = self.m + self._band_vec(K[:, 0]) * e
            self.nu = self.nu + self._band_vec(K[:, 1]) * e
            for l in range(self.L):
                IKH = np.eye(2) - np.outer(K[l], np.array([1.0, 0.0]))
                self.P[l] = IKH @ self.P[l]
        self._n_obs += 1

    @torch.no_grad()
    def predict(self, t) -> torch.Tensor:
        """Forecast at a skipped step: predicted values -> (IDCT) -> [B, Nt, D] (bf16)."""
        j = int(round(float(t)))
        assert self._ready and self.m is not None, "predict() called before any observation"
        self._advance_to(j)
        z = self.m / self.scale
        return self._from_state(z).to(torch.bfloat16)
```

A.6 Predictor–corrector gap smoothing

The smoother gains of Eq. (5.4) computed from the filter's own prior covariances and transition matrices, the smoothed residuals of Eq. (5.6), the re-integration of Eq. (5.7) with the scheduler's sigmas, the continuation of Eq. (5.8) and the gate refresh.

Listing A.9: PCGS smoother and correction (`KalSea-v0.2/FLUX/pcgs.py`).

```
    def _kalman_gains(self, a, b, steps):
        fc = self.tr.kalsea_forecaster
        Pb = self.upd["Pb"]
        S = Pb[:, 0, 0] + fc.r_obs                              # innovation variance per band  [L]
```

```
        L = Pb.shape[0]
        Phi = np.tile(np.eye(2), (L, 1, 1))
        gains = {}
        for i in range(b - 1, a, -1):
            k = i + 1
            dl = fc.delta[:, min(k, fc.N - 1)]
            rho = np.exp(-dl / fc.tau)
            A = np.zeros((L, 2, 2))
            A[:, 0, 0], A[:, 0, 1], A[:, 1, 1] = 1.0, dl, rho
            Phi = Phi @ A                                   # Phi(b, i) = Phi(b, i+1) A_{i+1}
            if i in steps:
                Pi = self.prior_P[i]
                gains[i] = (Pi[:, 0, :] * Phi[:, 0, :]).sum(-1) / S   # [P_i Phi^T]_00 / S
        return gains

    def _smooth(self, a, b):
        """Smoothed residuals (bf16) of the gap records."""
        assert self.upd is not None and self.upd["step"] == b, "PCGS: no forecaster update captured at the computed step"
        fc = self.tr.kalsea_forecaster
        gains = self._kalman_gains(a, b, {rec["step"] for rec in self.gap})
        E = self.upd["e_rel"] / fc.scale                    # innovation in DCT / element coordinates, absolute units
        out = []
        for rec in self.gap:
            corr = fc._from_grid(fc._inv(fc._band_vec_to_map(gains[rec["step"]]) * E))
            out.append((rec["pred"].float() + corr).to(rec["pred"].dtype))
        return out

    # ------------------------------------------------------------------------------------------
    def _out(self, h, temb):
        self.st["n_out_calls"] += int(h.shape[0])
        return self.tr.proj_out(self.tr.norm_out(h, temb))

    def _correct(self, b, x_b, temb_b, r_b, output, img_ids):
        tr = self.tr
        a = self.hist[0]
        steps = [rec["step"] for rec in self.gap]
        assert steps == list(range(a + 1, b)), f"PCGS gap bookkeeping: a={a} b={b} steps={steps}"
        rt = self._smooth(a, b)
        sg = self.sched.sigmas
        x = self.gap[0]["x"]                                # x_{a+1}: exact (step a was computed)
        for rec, r_i in zip(self.gap, rt):
            i = rec["step"]
            h = tr.x_embedder(x)
            self.st["n_emb_calls"] += 1
            v = self._out(h + r_i, rec["temb"])
            dt = sg[i + 1] - sg[i]                          # same tensors / arithmetic as FlowMatchEulerDiscreteScheduler.step
            x = (x.to(torch.float32) + dt * v).to(v.dtype)
        x_corr = x
        self.st["gap_len"].append(len(steps))
        if torch.equal(x_corr, x_b):
            return output
        self.st["n_corr"] += 1
        dxf = x_corr.float() - x_b.float()
        dt_b = float(sg[b + 1] - sg[b])
        h_new = tr.x_embedder(x_corr)                       # residual held fixed: h_out' =
```

```
    x_embedder(x_b') + r_b
        self.st["n_emb_calls"] += 1
        v_new = self._out(h_new + r_b, temb_b)
        out = (v_new.float() + dxf / dt_b).to(output.dtype)
        if self.gate_refresh and b < self.num_steps - 1 and getattr(tr, "enable_seacache", False) \
                and tr.previous_modulated_input is not None:
            self._refresh_gate(b, h_new, temb_b, img_ids)
        return out

    def _refresh_gate(self, b, h_new, temb_b, img_ids):
        """Recompute the SEA gate's reference (previous filtered modulated input) from the corrected
    latent."""
        tr = self.tr
        mod = tr.transformer_blocks[0].norm1(h_new, emb=temb_b)[0]
        Hg, Wg = int(img_ids[:, 1].max().item() + 1), int(img_ids[:, 2].max().item() + 1)
        mod = mod.reshape(mod.shape[0], Hg, Wg, mod.shape[-1])
        mod = apply_sea_with_scheduler(mod, tr.scheduler, b, power_exp=2.0, dims=(-2, -3), norm_mode="
    mean")
        tr.previous_modulated_input = mod.reshape(mod.shape[0], -1, mod.shape[-1])
        self.st["n_gate_refresh"] += 1
```

A.7 Caching core for the engineering applications

The shared core of Section 6.1.5: the gate, the residual streams with their fillers (imported unchanged from the two releases), and a generic PCGS smoother for samplers owned by the application driver.

Listing A.10: Caching core for the applications (`mech_apps/cache/cachecore.py`).

```
"""Shared caching core for the mechanical-engineering applications of the thesis.

A cache makes two decisions per sampler step i (see the thesis, Chapters 4 and 5):
  * WHEN to compute the block stack F: the SEA gate (SeaCache's spectrally aligned gate), unchanged.
  * WHAT to write on a skipped step:
      - 'seacache' : reuse of the last computed residual (SeaCache),
      - 'respect'  : ReSPect's Chebyshev + discrete-Taylor residual forecaster (
    SpectrumResidualForecaster,
                     imported unchanged from the ReSpect repository),
      - 'kalsea_nopcgs' : KalSea's damped-trend Kalman forecaster (KalSeaForecaster, per-element,
                     calibration-free, imported unchanged from KalSea-v0.2/FLUX2),
      - 'kalsea'   : the Kalman forecaster + predictor-corrector gap smoothing (PCGS).

The application drivers (oat_cache.py, pde_cache.py) own the sampler loop and call
  Stream.observe / Stream.forecast / Stream.smooth_gap and SEAGate.decide.
"""
from __future__ import annotations

import numpy as np
import torch

from util_seacache import apply_sea_from_ab, rel_l1
from spectrum_forecaster import SpectrumResidualForecaster
from kalsea_forecaster import KalSeaForecaster

METHODS = ("reference", "fewer_steps", "seacache", "respect", "kalsea_nopcgs", "kalsea")

```

```python

class SEAGate:
    """Accumulate-and-reset gate on the SEA-filtered, timestep-modulated input of the first block.

    The reference input is refreshed at every step (computed or skipped); the first and last steps are always computed.
    """

    def __init__(self, delta: float, num_steps: int, power_exp: float = 2.0, dims=(-1, -2)):
        self.delta, self.N, self.beta, self.dims = float(delta), int(num_steps), float(power_exp), tuple(dims)
        self.acc, self.prev, self.distances = 0.0, None, []

    def filtered(self, I: torch.Tensor, a: float, b: float) -> torch.Tensor:
        return apply_sea_from_ab(I.float(), a, b, power_exp=self.beta, dims=self.dims, norm_mode="mean")

    def decide(self, i: int, I_f: torch.Tensor) -> bool:
        if i == 0 or i == self.N - 1 or self.prev is None:
            compute, d = True, 0.0
            self.acc = 0.0
        else:
            d = rel_l1(I_f, self.prev)
            self.acc += d
            compute = self.acc >= self.delta
            if compute:
                self.acc = 0.0
        self.distances.append(d)
        self.prev = I_f
        return compute

    def refresh(self, I_f: torch.Tensor):
        """PCGS step 4: replace the stored reference by the filtered input of the corrected latent."""
        self.prev = I_f


class Stream:
    """One residual stream (e.g. one network call per sampler step) with its filler and PCGS bookkeeping."""

    def __init__(self, method: str, num_steps: int):
        self.method, self.N = method, int(num_steps)
        self.fc = None
        if method == "respect":
            self.fc = SpectrumResidualForecaster(m=4, k=100, lam=0.1, w=0.5, t_max=float(num_steps), min_cheb_obs=4)
        elif method in ("kalsea", "kalsea_nopcgs"):
            self.fc = KalSeaForecaster(calib=None, num_steps=num_steps, tau=0.5, r_obs=1e-3, time_mode="step",
                                       use_dct=False, adapt_q=True)
        self.last = None            # last computed residual (SeaCache reuse, and fallback)
        self.shape = None
        self.pcgs = method == "kalsea"
        self.gap = []               # skipped steps since the last computed step: dict(step, pred, ctx)
        self.prior_P = {}
        self.upd = None
        self.hist = None            # last computed step index

    # -- shapes: KalSeaForecaster wants a 3-D tensor [B, Nt, D]; any per-element layout works
```

```
    ----------------
    def _flat(self, r):
        self.shape = r.shape
        return r.reshape(r.shape[0], r.shape[1], -1)

    def _unflat(self, r):
        return r.reshape(self.shape)

    # -- computed step
    ---------------------------------------------------------------------------------------
    @torch.no_grad()
    def observe(self, i: int, r: torch.Tensor):
        r = r.detach()
        if self.pcgs and self.gap and self.fc is not None and self.fc._ready and self.fc.m is not None:
            fc = self.fc
            fc._advance_to(i)
            Z = fc._to_state(self._flat(r))
            self.upd = dict(step=i, Pb=fc.P.copy(), e_rel=Z * fc.scale - fc.m)
        if self.fc is not None:
            x = self._flat(r) if isinstance(self.fc, KalSeaForecaster) else r
            self.fc.update(i, x)
        self.last = r

    # -- skipped step
    ---------------------------------------------------------------------------------------
    @torch.no_grad()
    def forecast(self, i: int, ctx=None) -> torch.Tensor:
        if self.fc is None or self.fc.n_obs < 1:
            pred = self.last
        elif isinstance(self.fc, KalSeaForecaster):
            pred = self._unflat(self.fc.predict(i)).to(self.last.dtype)
            if self.pcgs:
                self.prior_P[i] = self.fc.P.copy()
        else:
            pred = self.fc.predict(i).to(self.last.dtype)
        if self.pcgs:
            self.gap.append(dict(step=i, pred=pred, ctx=ctx))
        return pred

    # -- PCGS smoother
    ---------------------------------------------------------------------------------------
    def _gains(self, a, b, steps):
        fc = self.fc
        Pb = self.upd["Pb"]
        S = Pb[:, 0, 0] + fc.r_obs
        L = Pb.shape[0]
        Phi = np.tile(np.eye(2), (L, 1, 1))
        gains = {}
        for i in range(b - 1, a, -1):
            dl = fc.delta[:, min(i + 1, fc.N - 1)]
            rho = np.exp(-dl / fc.tau)
            A = np.zeros((L, 2, 2))
            A[:, 0, 0], A[:, 0, 1], A[:, 1, 1] = 1.0, dl, rho
            Phi = Phi @ A                                  # Phi(b, i) = Phi(b, i+1) A_{i+1}
            if i in steps:
                gains[i] = float(((self.prior_P[i][:, 0, :] * Phi[:, 0, :]).sum(-1) / S)[0])
        return gains

    @torch.no_grad()
```

```
    def smooth_gap(self, b: int):
        """Smoothed residuals r~_i = r^_i + g_i e_b for the gap closed by the computed step b (RTS gains)."""
        if not (self.pcgs and self.gap and self.hist is not None and self.upd is not None and self.upd["step"] == b):
            return None
        a = self.hist
        steps = [g["step"] for g in self.gap]
        assert steps == list(range(a + 1, b)), (a, b, steps)
        gains = self._gains(a, b, set(steps))
        E = self._unflat(self.upd["e_rel"] / self.fc.scale)
        return [dict(step=g["step"], r=(g["pred"].float() + gains[g["step"]] * E).to(g["pred"].dtype),
                     gain=gains[g["step"]], ctx=g["ctx"]) for g in self.gap]

    def close(self, b: int):
        self.gap, self.prior_P, self.upd = [], {}, None
        self.hist = b
```